

LECTURE NOTES

ON

ARTIFICIAL INTELLIGENCE

PREPARED BY

DR. PRASHANTA KUMAR PATRA

COLLEGE OF ENGINEERING AND TECHNOLOGY,
BHUBANESWAR

ARTIFICIAL INTELLIGENCE SYLLABUS

Module 1

12Hrs

What is Artificial Intelligence? AI Technique, Level of the Model, Problem Spaces, and Search: Defining the Problem as a State Space Search, Production Systems, Problem Characteristics, Production System Characteristics, Issues in the Design of Search Programs. Heuristic Search Techniques: Generate-and-Test, Hill Climbing, Best-first Search, Problem Reduction, Constraint Satisfaction, Means-ends

Analysis, Knowledge Representation: Representations and Mappings, Approaches to Knowledge Representation, Using Predicate Logic: Representing Simple Facts in Logic, Representing Instance and ISA Relationships, Computable Functions and Predicates, Resolution, Natural Deduction. Using Rules: Procedural Versus Declarative Knowledge, Logic Programming, Forward Versus Backward Reasoning,

Matching, Control Knowledge. Symbolic Reasoning Under Uncertainty: Introduction to Nonmonotonic Reasoning, Logics for Nonmonotonic Reasoning, Implementation Issues, Augmenting a Problem-solver, Depth-first Search, Breadthfirst Search. Weak and Strong Slot-and-Filler Structures: Semantic Nets, Frames, Conceptual Dependency Scripts, CYC.

Module 2 10Hrs

Game Playing: The Minimax Search Procedure, Adding Alpha-beta Cutoffs, Iterative Deepening. Planning: The Blocks World, Components of a Planning System, Goal Stack Planning, Nonlinear Planning Using Constraint Posting, Hierarchical Planning. Other Planning Techniques. Understanding: What is Understanding, What Makes Understanding Hard?, Understanding as Constraint Satisfaction. Natural Language Processing: Introduction, Syntactic Processing, Semantic Analysis, Discourse and Pragmatic Processing, Statistical Natural Language Processing, Spell Checking.

Module 3

8Hrs

Learning: Rote Learning, learning by Taking Advice, Learning in Problem-solving, Learning from Examples: Induction, Explanation-based Learning, Discovery, Analogy, Formal Learning Theory, Neural Net Learning and Genetic Learning. Expert Systems: Representing and Using Domain Knowledge, Expert System Shells, Explanation, Knowledge Acquisition.

Text Book:

1. Elaine Rich, Kevin Knight, & Shivashankar B Nair, Artificial Intelligence, McGraw Hill, 3rd ed., 2009

References:

1) Introduction to Artificial Intelligence & Expert Systems, Dan W Patterson, PHI., 2010

2) S Kaushik, Artificial Intelligence, Cengage Learning, 1st ed. 2011

ARTIFICIAL INTELLIGENCE

What is Artificial Intelligence?

It is a branch of Computer Science that pursues creating the computers or machines as intelligent as human beings.

It is the science and engineering of making intelligent machines, especially intelligent computer programs.

It is related to the similar task of using computers to understand human intelligence, but **AI** does not have to confine itself to methods that are biologically observable

Definition: Artificial Intelligence is the study of how to make computers do things, which, at the moment, people do better.

According to the father of Artificial Intelligence, John McCarthy, it is “*The science and engineering of making intelligent machines, especially intelligent computer programs*”.

Artificial Intelligence is a way of **making a computer, a computer-controlled robot, or a software think intelligently**, in the similar manner the intelligent humans think.

AI is accomplished by studying how human brain thinks and how humans learn, decide, and work while trying to solve a problem, and then using the outcomes of this study as a basis of developing intelligent software and systems.

It has gained prominence recently due, in part, to big data, or the increase in speed, size and variety of data businesses are now collecting. AI can perform tasks such as identifying patterns in the data more efficiently than humans, enabling businesses to gain more insight out of their data.

From a **business** perspective AI is a set of very powerful tools, and methodologies for using those tools to solve business problems.

From a **programming** perspective, AI includes the study of symbolic programming, problem solving, and search.

AI Vocabulary

Intelligence relates to tasks involving higher mental processes, e.g. creativity, solving problems, pattern recognition, classification, learning, induction, deduction, building analogies, optimization, language processing, knowledge and many more. Intelligence is the computational part of the ability to achieve goals.

Intelligent behaviour is depicted by perceiving one's environment, acting in complex environments, learning and understanding from experience, reasoning to solve problems and discover hidden knowledge, applying knowledge successfully in new situations, thinking abstractly, using analogies, communicating with others and more.

Science based goals of AI pertain to developing concepts, mechanisms and understanding biological intelligent behaviour. The emphasis is on understanding intelligent behaviour.

Engineering based goals of AI relate to developing concepts, theory and practice of building intelligent machines. The emphasis is on system building.

AI Techniques depict how we represent, manipulate and reason with knowledge in order to solve problems. Knowledge is a collection of ‘facts’. To manipulate these facts by a program, a suitable representation is required. A good representation facilitates problem solving.

Learning means that programs learn from what facts or behaviour can represent. Learning denotes changes in the systems that are adaptive in other words, it enables the system to do the same task(s) more efficiently next time.

Applications of AI refers to problem solving, search and control strategies, speech recognition, natural language understanding, computer vision, expert systems, etc.

Problems of AI:

Intelligence does not imply perfect understanding; every intelligent being has limited perception, memory and computation. Many points on the spectrum of intelligence versus cost are viable, from insects to humans. AI seeks to understand the computations required from intelligent behaviour and to produce computer systems that exhibit intelligence. Aspects of intelligence studied by AI include perception, communicational using human languages, reasoning, planning, learning and memory.

The following questions are to be considered before we can step forward:

1. What are the underlying assumptions about intelligence?
2. What kinds of techniques will be useful for solving AI problems?
3. At what level human intelligence can be modelled?
4. When will it be realized when an intelligent program has been built?

Branches of AI:

A list of branches of AI is given below. However some branches are surely missing, because no one has identified them yet. Some of these may be regarded as concepts or topics rather than full branches.

Logical AI — In general the facts of the specific situation in which it must act, and its goals are all represented by sentences of some mathematical logical language. The program decides what to do by inferring that certain actions are appropriate for achieving its goals.

Search — Artificial Intelligence programs often examine large numbers of possibilities – for example, moves in a chess game and inferences by a theorem proving program. Discoveries are frequently made about how to do this more efficiently in various domains.

Pattern Recognition — When a program makes observations of some kind, it is often planned to compare what it sees with a pattern. For example, a vision program may try to match a pattern of eyes and a nose in a scene in order to find a face. More complex patterns are like a natural language text, a chess position or in the history of some event. These more complex patterns require quite different methods than do the simple patterns that have been studied the most.

Representation — Usually languages of mathematical logic are used to represent the facts about the world.

Inference — Others can be inferred from some facts. Mathematical logical deduction is sufficient for some purposes, but new methods of *non-monotonic* inference have been added to the logic since the 1970s. The simplest kind of non-monotonic reasoning is default reasoning in which a conclusion is to be inferred by default. But the conclusion can be withdrawn if there is evidence to the divergent. For example, when we hear of a bird, we infer that it can fly, but this conclusion can be reversed when we hear that it is a penguin. It is the possibility that a conclusion may have to be withdrawn that constitutes the non-monotonic character of the reasoning. Normal logical reasoning is monotonic, in that the set of conclusions can be drawn from a set of premises, i.e. monotonic increasing function of the premises. Circumscription is another form of non-monotonic reasoning.

Common sense knowledge and Reasoning — This is the area in which AI is farthest from the human level, in spite of the fact that it has been an active research area since the 1950s. While there has been considerable progress in developing systems of *non-monotonic reasoning* and theories of action, yet more new ideas are needed.

Learning from experience — There are some rules expressed in logic for learning. Programs can only learn what facts or behaviour their formalisms can represent, and unfortunately learning systems are almost all based on very limited abilities to represent information.

Planning — Planning starts with general facts about the world (especially facts about the effects of actions), facts about the particular situation and a statement of a goal. From these, planning programs generate a strategy for achieving the goal. In the most common cases, the strategy is just a sequence of actions.

Epistemology — This is a study of the kinds of knowledge that are required for solving problems in the world.

Ontology — Ontology is the study of the kinds of things that exist. In AI the programs and sentences deal with various kinds of objects and we study what these kinds are and what their basic properties are. Ontology assumed importance from the 1990s.

Heuristics — A heuristic is a way of trying to discover something or an idea embedded in a program. The term is used variously in AI. *Heuristic functions* are used in some approaches to search or to measure how far a node in a search tree seems to be from a goal. *Heuristic predicates* that compare two nodes in a search tree to see if one is better than the other, i.e. constitutes an advance toward the goal, and may be more useful.

Genetic programming — Genetic programming is an automated method for creating a working computer program from a high-level problem statement of a problem. Genetic programming starts from a high-level statement of ‘what needs to be done’ and automatically creates a computer program to solve the problem.

Applications of AI

AI has applications in all fields of human study, such as finance and economics, environmental engineering, chemistry, computer science, and so on. Some of the applications of AI are listed below:

- Perception
 - Machine vision
 - Speech understanding
 - Touch (*tactile* or *haptic*) sensation
- Robotics
- Natural Language Processing
 - Natural Language Understanding
 - Speech Understanding
 - Language Generation
 - Machine Translation
- Planning
- Expert Systems
- Machine Learning
- Theorem Proving
- Symbolic Mathematics
- Game Playing

AI Technique:

Artificial Intelligence research during the last three decades has concluded that *Intelligence requires knowledge*. To compensate overwhelming quality, knowledge possesses less desirable properties.

- A. It is huge.
- B. It is difficult to characterize correctly.
- C. It is constantly varying.
- D. It differs from data by being organized in a way that corresponds to its application.
- E. It is complicated.

An AI technique is a method that exploits knowledge that is represented so that:

- The knowledge captures generalizations that share properties, are grouped together, rather than being allowed separate representation.
- It can be understood by people who must provide it—even though for many programs bulk of the data comes automatically from readings.
- In many AI domains, how the people understand the same people must supply the knowledge to a program.
- It can be easily modified to correct errors and reflect changes in real conditions.
- It can be widely used even if it is incomplete or inaccurate.
- It can be used to help overcome its own sheer bulk by helping to narrow the range of possibilities that must be usually considered.

In order to characterize an AI technique let us consider initially OXO or tic-tac-toe and use a series of different approaches to play the game.

The programs increase in complexity, their use of generalizations, the clarity of their knowledge and the extensibility of their approach. In this way they move towards being representations of AI techniques.

Example-1: Tic-Tac-Toe

1.1 The first approach (simple)

The Tic-Tac-Toe game consists of a nine element vector called BOARD; it represents the numbers 1 to 9 in three rows.

1	2	3
4	5	6
7	8	9

An element contains the value 0 for blank, 1 for X and 2 for O. A MOVETABLE vector consists of 19,683 elements (3^9) and is needed where each element is a nine element vector. The contents of the vector are especially chosen to help the algorithm.

The algorithm makes moves by pursuing the following:

1. View the vector as a ternary number. Convert it to a decimal number.
2. Use the decimal number as an index in MOVETABLE and access the vector.
3. Set BOARD to this vector indicating how the board looks after the move. This approach is capable in time but it has several disadvantages. It takes more space and requires stunning

effort to calculate the decimal numbers. This method is specific to this game and cannot be completed.

1.2 The second approach

The structure of the data is as before but we use 2 for a blank, 3 for an X and 5 for an O. A variable called TURN indicates 1 for the first move and 9 for the last. The algorithm consists of three actions:

MAKE2 which returns 5 if the centre square is blank; otherwise it returns any blank non-corner square, i.e. 2, 4, 6 or 8. POSSWIN (p) returns 0 if player p cannot win on the next move and otherwise returns the number of the square that gives a winning move.

It checks each line using products $3*3*2 = 18$ gives a win for X, $5*5*2=50$ gives a win for O, and the winning move is the holder of the blank. GO (n) makes a move to square n setting BOARD[n] to 3 or 5.

This algorithm is more involved and takes longer but it is more efficient in storage which compensates for its longer time. It depends on the programmer's skill.

1.3 The final approach

The structure of the data consists of BOARD which contains a nine element vector, a list of board positions that could result from the next move and a number representing an estimation of how the board position leads to an ultimate win for the player to move.

This algorithm looks ahead to make a decision on the next move by deciding which the most promising move or the most suitable move at any stage would be and selects the same.

Consider all possible moves and replies that the program can make. Continue this process for as long as time permits until a winner emerges, and then choose the move that leads to the computer winning, if possible in the shortest time.

Actually this is most difficult to program by a good limit but it is as far that the technique can be extended to in any game. This method makes relatively fewer loads on the programmer in terms of the game technique but the overall game strategy must be known to the adviser.

Example-2: Question Answering

Let us consider Question Answering systems that accept input in English and provide answers also in English. This problem is harder than the previous one as it is more difficult to specify the problem properly. Another area of difficulty concerns deciding whether the answer obtained is correct, or not, and further what is meant by 'correct'. For example, consider the following situation:

2.1 Text

Rani went shopping for a new Coat. She found a red one she really liked.
When she got home, she found that it went perfectly with her favourite dress.

2.2 Question

1. What did Rani go shopping for?

2. What did Rani find that she liked?
3. Did Rani buy anything?

Method 1

2.3 Data Structures

A set of templates that match common questions and produce patterns used to match against inputs. Templates and patterns are used so that a template that matches a given question is associated with the corresponding pattern to find the answer in the input text. For example, the template who did **x y** generates **x y z** if a match occurs and **z** is the answer to the question. The given text and the question are both stored as strings.

2.4 Algorithm

Answering a question requires the following four steps to be followed:

- Compare the template against the questions and store all successful matches to produce a set of text patterns.
- Pass these text patterns through a substitution process to change the person or voice and produce an expanded set of text patterns.
- Apply each of these patterns to the text; collect all the answers and then print the answers.

2.5 Example

In **question 1** we use the template WHAT DID X Y which generates Rani go shopping for **z** and after substitution we get Rani goes shopping for **z** and Rani went shopping for **z** giving **z** [equivalence] a new coat

In **question 2** we need a very large number of templates and also a scheme to allow the insertion of 'find' before 'that she liked'; the insertion of 'really' in the text; and the substitution of 'she' for 'Rani' gives the answer 'a red one'.

Question 3 cannot be answered.

2.6 Comments

This is a very primitive approach basically not matching the criteria we set for intelligence and worse than that, used in the game. Surprisingly this type of technique was actually used in ELIZA which will be considered later in the course.

Method 2

2.7 Data Structures

A structure called English consists of a dictionary, grammar and some semantics about the vocabulary we are likely to come across. This data structure provides the knowledge to convert English text into a storable internal form and also to convert the response back into English. The structured representation of the text is a processed form and defines the context of the input text by making explicit all references such as pronouns. There are three types of such *knowledge representation* systems: production rules of the form ‘if x then y’, slot and filler systems and statements in mathematical logic. The system used here will be the slot and filler system.

Take, for example sentence:

‘She found a red one she really liked’.

Event2

instance: finding
tense: past
agent: Rani
object: Thing1

Event2

instance: liking
tense: past
modifier: much
object: Thing1

Thing1

instance: coat
colour: red

The question is stored in two forms: as input and in the above form.

2.8 Algorithm

- Convert the question to a structured form using English know how, then use a marker to indicate the substring (like ‘who’ or ‘what’) of the structure, that should be returned as an answer. If a slot and filler system is used a special marker can be placed in more than one slot.
- The answer appears by matching this structured form against the structured text.
- The structured form is matched against the text and the requested segments of the question are returned.

2.9 Examples

Both questions 1 and 2 generate answers via a new coat and a red coat respectively. Question 3 cannot be answered, because there is no direct response.

2.10 Comments

This approach is more meaningful than the previous one and so is more effective. The extra power given must be paid for by additional search time in the knowledge bases. A warning

must be given here: that is – to generate unambiguous English knowledge base is a complex task and must be left until later in the course. The problems of handling pronouns are difficult.

For example:

Rani walked up to the salesperson: she asked where the toy department was.

Rani walked up to the salesperson: she asked her if she needed any help.

Whereas in the original text the linkage of ‘she’ to ‘Rani’ is easy, linkage of ‘she’ in each of the above sentences to Rani and to the salesperson requires additional knowledge about the context via the people in a shop.

Method 3

2.11 Data Structures

World model contains knowledge about objects, actions and situations that are described in the input text. This structure is used to create integrated text from input text. The diagram shows how the system’s knowledge of shopping might be represented and stored. This information is known as a script and in this case is a shopping script. (See figure 1.1 next page)

1.8.2.12 Algorithm

Convert the question to a structured form using both the knowledge contained in Method 2 and the World model, generating even more possible structures, since even more knowledge is being used. Sometimes filters are introduced to prune the possible answers.

To answer a question, the scheme followed is: Convert the question to a structured form as before but use the world model to resolve any ambiguities that may occur. The structured form is matched against the text and the requested segments of the question are returned.

2.13 Example

Both questions 1 and 2 generate answers, as in the previous program. Question 3 can now be answered. The shopping script is instantiated and from the last sentence the path through step 14 is the one used to form the representation. ‘M’ is bound to the red coat-got home. ‘**Rani buys a red coat**’ comes from step 10 and the integrated text generates that she bought a red coat.

2.14 Comments

This program is more powerful than both the previous programs because it has more knowledge. Thus, like the last game program it is exploiting AI techniques. However, we are not yet in a position to handle any English question. The major omission is that of a general reasoning mechanism known as inference to be used when the required answer is not explicitly given in the input text. But this approach can handle, with some modifications, questions of the following form with the answer—Saturday morning Rani went shopping. Her brother tried to call her but she did not answer.

Question: Why couldn’t Rani’s brother reach her?

Answer: Because she was not in.

This answer is derived because we have supplied an additional fact that a person cannot be in two places at once. This patch is not sufficiently general so as to work in all cases and does not provide the type of solution we are really looking for.

Shopping Script: C - Customer, S - Salesperson

Props: M - Merchandize, D - Money-dollars, Location: L - a Store.

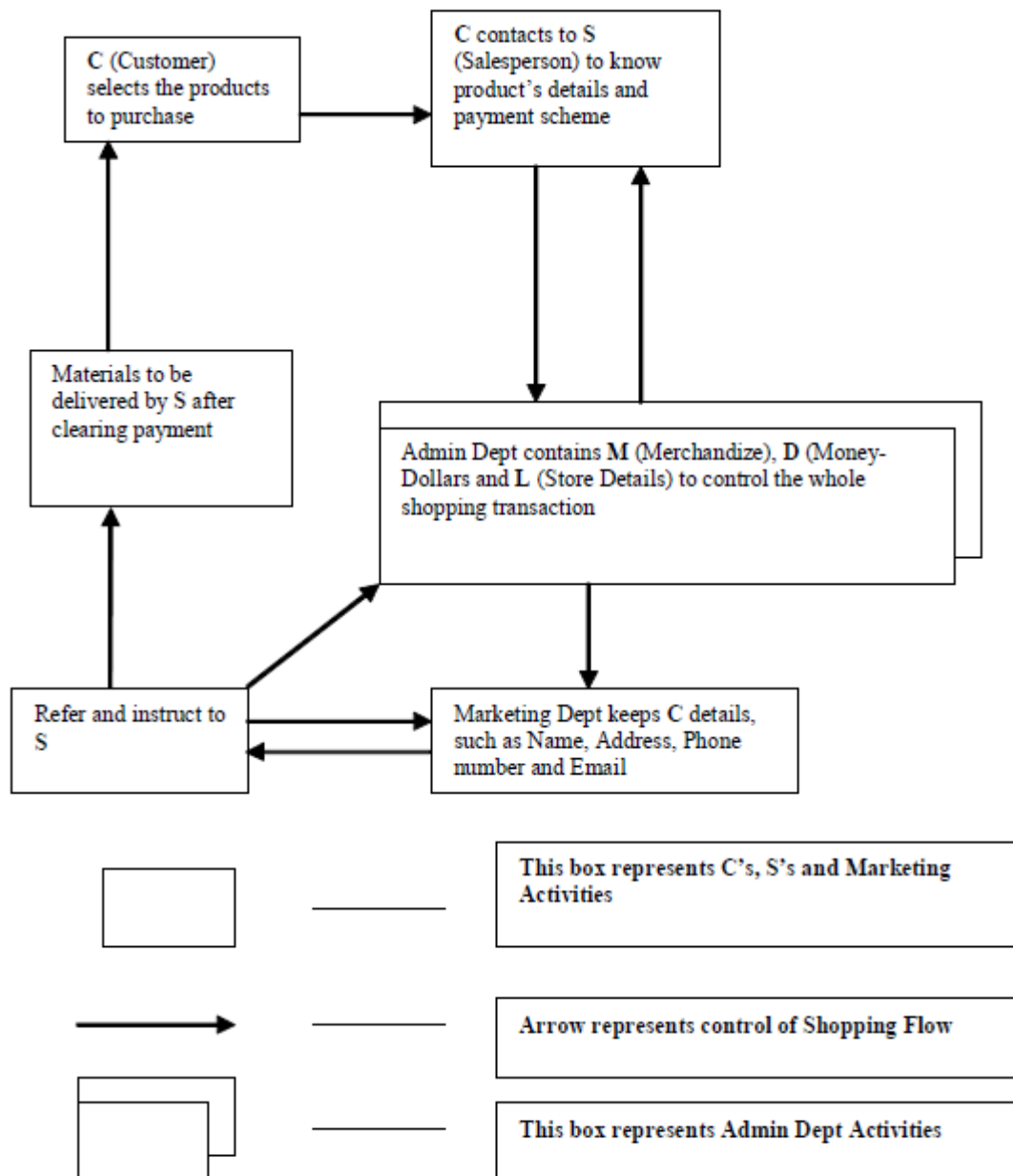


Fig. 1.1 Diagrammatic Representation of Shopping Script

LEVEL OF THE AI MODEL

‘What is our goal in trying to produce programs that do the intelligent things that people do?’

Are we trying to produce programs that do the tasks the same way that people do?

OR

Are we trying to produce programs that simply do the tasks the easiest way that is possible?

Programs in the first class attempt to solve problems that a computer can easily solve and do not usually use AI techniques. AI techniques usually include a search, as no direct method is available, the use of knowledge about the objects involved in the problem area and abstraction on which allows an element of pruning to occur, and to enable a solution to be found in real time; otherwise, the data could explode in size. Examples of these trivial problems in the first class, which are now of interest only to psychologists are EPAM (Elementary Perceiver and Memorizer) which memorized garbage syllables.

The second class of problems attempts to solve problems that are non-trivial for a computer and use AI techniques. We wish to model human performance on these:

1. To test psychological theories of human performance. Ex. PARRY [Colby, 1975] – a program to simulate the conversational behavior of a paranoid person.
2. To enable computers to understand human reasoning – for example, programs that answer questions based upon newspaper articles indicating human behavior.
3. To enable people to understand computer reasoning. Some people are reluctant to accept computer results unless they understand the mechanisms involved in arriving at the results.
4. To exploit the knowledge gained by people who are best at gathering information. This persuaded the earlier workers to simulate human behavior in the SB part of AISB simulated behavior. Examples of this type of approach led to GPS (General Problem Solver).

Questions for Practice:

1. What is *intelligence*? How do we measure it? Are these measurements useful?
2. When the temperature falls and the thermostat turns the heater on, does it act because it *believes* the room to be too cold? Does it *feel* cold? What sorts of things can have beliefs or feelings? Is this related to the idea of consciousness?
3. Some people believe that the relationship between your mind (a non-physical thing) and your brain (the physical thing inside your skull) is exactly like the relationship between a computational process (a non-physical thing) and a physical computer. Do you agree?
4. How good are machines at playing chess? If a machine can consistently beat all the best human chess players, does this prove that the machine is *intelligent*?
5. What is AI Technique? Explain Tic-Tac-Toe Problem using AI Technique.

PROBLEMS, PROBLEM SPACES AND SEARCH

To solve the problem of building a system you should take the following steps:

1. Define the problem accurately including detailed specifications and what constitutes a suitable solution.
2. Scrutinize the problem carefully, for some features may have a central affect on the chosen method of solution.
3. Segregate and represent the background knowledge needed in the solution of the problem.
4. Choose the best solving techniques for the problem to solve a solution.

Problem solving is a process of generating solutions from observed data.

- a '*problem*' is characterized by a set of *goals*,
- a set of *objects*, and
- a set of *operations*.

These could be ill-defined and may evolve during problem solving.

- A '**problem space**' is an abstract space.
 - ✓ A problem space encompasses all *valid states* that can be generated by the application of any combination of *operators* on any combination of *objects*.
 - ✓ The problem space may contain one or more *solutions*. A solution is a combination of *operations* and *objects* that achieve the *goals*.
- A '**search**' refers to the search for a solution in a problem space.
 - ✓ Search proceeds with different types of '*search control strategies*'.
 - ✓ The *depth-first search* and *breadth-first search* are the two common *search strategies*.

2.1 AI - General Problem Solving

Problem solving has been the key area of concern for Artificial Intelligence.

Problem solving is a process of generating solutions from observed or given data. It is however not always possible to use direct methods (i.e. go directly from data to solution). Instead, problem solving often needs to use indirect or modelbased methods.

General Problem Solver (GPS) was a computer program created in 1957 by Simon and Newell to build a universal problem solver machine. *GPS* was based on Simon and Newell's theoretical work on logic machines. *GPS* in principle can solve any formalized symbolic problem, such as theorems proof and geometric problems and chess playing. *GPS* solved many simple problems, such as the Towers of Hanoi, that could be sufficiently formalized, but ***GPS could not solve any real-world problems.***

To build a system to solve a particular problem, we need to:

- Define the problem precisely – find input situations as well as final situations for an acceptable solution to the problem

- Analyze the problem – find few important features that may have impact on the appropriateness of various possible techniques for solving the problem
- Isolate and represent task knowledge necessary to solve the problem
- Choose the best problem-solving technique(s) and apply to the particular problem

Problem definitions

A problem is defined by its ‘*elements*’ and their ‘*relations*’. To provide a formal description of a problem, we need to do the following:

- a. Define a *state space* that contains all the possible configurations of the relevant objects, including some impossible ones.
- b. Specify one or more states that describe possible situations, from which the problem-solving process may start. These states are called *initial states*.
- c. Specify one or more states that would be acceptable solution to the problem.

These states are called *goal states*.

Specify a set of *rules* that describe the actions (*operators*) available.

The problem can then be solved by using the *rules*, in combination with an appropriate *control strategy*, to move through the *problem space* until a *path* from an *initial state* to a *goal state* is found. This process is known as ‘*search*’. Thus:

- *Search* is fundamental to the problem-solving process.
- *Search* is a general mechanism that can be used when a more direct method is not known.
- *Search* provides the framework into which more direct methods for solving subparts of a problem can be embedded. A very large number of AI problems are formulated as search problems.
- Problem space

A *problem space* is represented by a directed graph, where *nodes* represent search state and *paths* represent the operators applied to change the *state*.

To simplify search algorithms, it is often convenient to logically and programmatically represent a problem space as a **tree**. A *tree* usually decreases the complexity of a search at a cost. Here, the cost is due to duplicating some nodes on the tree that were linked numerous times in the graph, e.g. node **B** and node **D**.

A *tree* is a *graph* in which any two vertices are connected by exactly one path. Alternatively, any connected *graph* with no cycles is a *tree*.

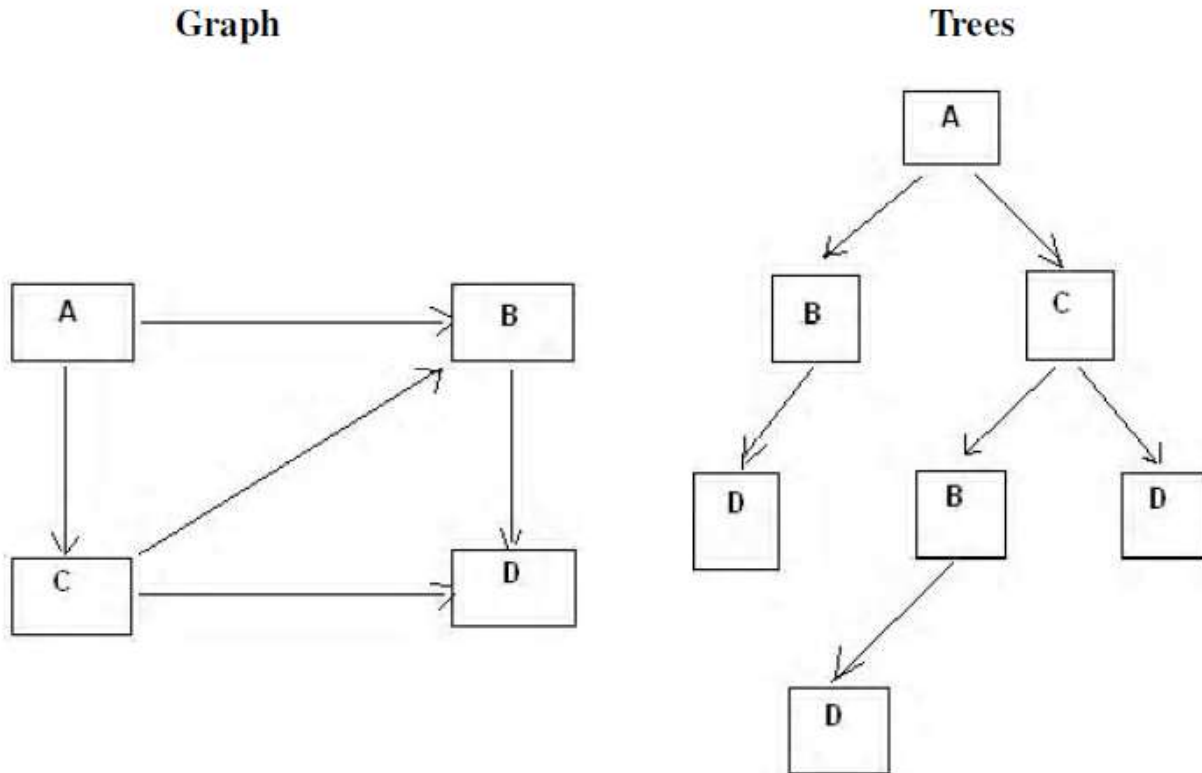


Fig. 2.1 Graph and Tree

- **Problem solving:** The term, Problem Solving relates to analysis in AI. Problem solving may be characterized as a systematic search through a range of possible actions to reach some predefined goal or solution. Problem-solving methods are categorized as *special purpose* and *general purpose*.

- A *special-purpose method* is tailor-made for a particular problem, often exploits very specific features of the situation in which the problem is embedded.

- A *general-purpose method* is applicable to a wide variety of problems. One General-purpose technique used in AI is '*means-end analysis*': a step-bystep, or incremental, reduction of the difference between current state and final goal.

2.3 DEFINING PROBLEM AS A STATE SPACE SEARCH

To solve the problem of playing a game, we require the rules of the game and targets for winning as well as representing positions in the game. The opening position can be defined as the initial state and a winning position as a goal state. Moves from initial state to other states leading to the goal state follow legally. However, the rules are far too abundant in most games— especially in chess, where they exceed the number of particles in the universe. Thus, the rules cannot be supplied accurately and computer programs cannot handle easily. The storage also presents another problem but searching can be achieved by hashing.

The number of rules that are used must be minimized and the set can be created by expressing each rule in a form as possible. The representation of games leads to a state space representation and it is common for well-organized games with some structure. This representation allows for the formal definition of a problem that needs the movement from a set of initial positions to one of a set of target positions. It means that the solution involves using known techniques and a systematic search. This is quite a common method in Artificial Intelligence.

2.3.1 State Space Search

A *state space* represents a problem in terms of *states* and *operators* that change states.

A state space consists of:

- A representation of the *states* the system can be in. For example, in a board game, the board represents the current state of the game.
- A set of *operators* that can change one state into another state. In a board game, the operators are the legal moves from any given state. Often the operators are represented as programs that change a state representation to represent the new state.
- An *initial state*.
- A set of *final states*; some of these may be desirable, others undesirable. This set is often represented implicitly by a program that detects terminal states.

2.3.2 The Water Jug Problem

In this problem, we use two jugs called **four** and **three**; four holds a maximum of four gallons of water and **three** a maximum of three gallons of water. How can we get two gallons of water in the **four** jug?

The state space is a set of prearranged pairs giving the number of gallons of water in the pair of jugs at any time, i.e., (**four**, **three**) where **four** = 0, 1, 2, 3 or 4 and **three** = 0, 1, 2 or 3.

The start state is (0, 0) and the goal state is (2, n) where n may be any but it is limited to **three** holding from 0 to 3 gallons of water or empty. Three and four shows the name and numerical number shows the amount of water in jugs for solving the water jug problem. The major production rules for solving this problem are shown below:

Initial condition

1. (four, three) if four < 4
2. (four, three) if three < 3
3. (four, three) If four > 0
4. (four, three) if three > 0
5. (four, three) if four + three < 4
6. (four, three) if four + three < 3
7. (0, three) If three > 0
8. (four, 0) if four > 0
9. (0, 2)
10. (2, 0)
11. (four, three) if four < 4
12. (three, four) if three < 3

Goal comment

(4, three) fill four from tap
 (four, 3) fill three from tap
 (0, three) empty four into drain
 (four, 0) empty three into drain
 (four + three, 0) empty three into four
 (0, four + three) empty four into three
 (three, 0) empty three into four
 (0, four) empty four into three
 (2, 0) empty three into four
 (0, 2) empty four into three
 (4, three-diff) pour diff, 4-four, into four from three
 (four-diff, 3) pour diff, 3-three, into three from four and a solution is given below four three rule

(Fig. 2.2 Production Rules for the Water Jug Problem)

<u>Gallons in Four Jug</u>	<u>Gallons in Three Jug</u>	<u>Rules Applied</u>
0	0	-
0	3	2
3	0	7
3	3	2
4	2	11
0	2	3
2	0	10

(Fig. 2.3 One Solution to the Water Jug Problem)

The problem solved by using the production rules in combination with an appropriate control strategy, moving through the problem space until a path from an initial state to a goal state is found. In this problem solving process, search is the fundamental concept. For simple problems it is easier to achieve this goal by hand but there will be cases where this is far too difficult.

2.4 PRODUCTION SYSTEMS

Production systems provide appropriate structures for performing and describing search processes. A production system has four basic components as enumerated below.

- A set of rules each consisting of a left side that determines the applicability of the rule and a right side that describes the operation to be performed if the rule is applied.
- A database of current facts established during the process of inference.

- A control strategy that specifies the order in which the rules will be compared with facts in the database and also specifies how to resolve conflicts in selection of several rules or selection of more facts.
- A rule firing module.

The production rules operate on the knowledge database. Each rule has a precondition—that is, either satisfied or not by the knowledge database. If the precondition is satisfied, the rule can be applied. Application of the rule changes the knowledge database. The control system chooses which applicable rule should be applied and ceases computation when a termination condition on the knowledge database is satisfied.

Example: Eight puzzle (8-Puzzle)

The 8-puzzle is a 3×3 array containing eight square pieces, numbered 1 through 8, and one empty space. A piece can be moved horizontally or vertically into the empty space, in effect exchanging the positions of the piece and the empty space. There are four possible moves, UP (move the blank space up), DOWN, LEFT and RIGHT. The aim of the game is to make a sequence of moves that will convert the board from the start state into the goal state:

2	3	4
8	6	2
7		5

Initial State

1	2	3
8		4
7	6	5

Goal State

This example can be solved by the operator sequence UP, RIGHT, UP, LEFT, DOWN.

Example: Missionaries and Cannibals

The Missionaries and Cannibals problem illustrates the use of state space search for planning under constraints:

Three missionaries and three cannibals wish to cross a river using a two person boat. If at any time the cannibals outnumber the missionaries on either side of the river, they will eat the missionaries. How can a sequence of boat trips be performed that will get everyone to the other side of the river without any missionaries being eaten?

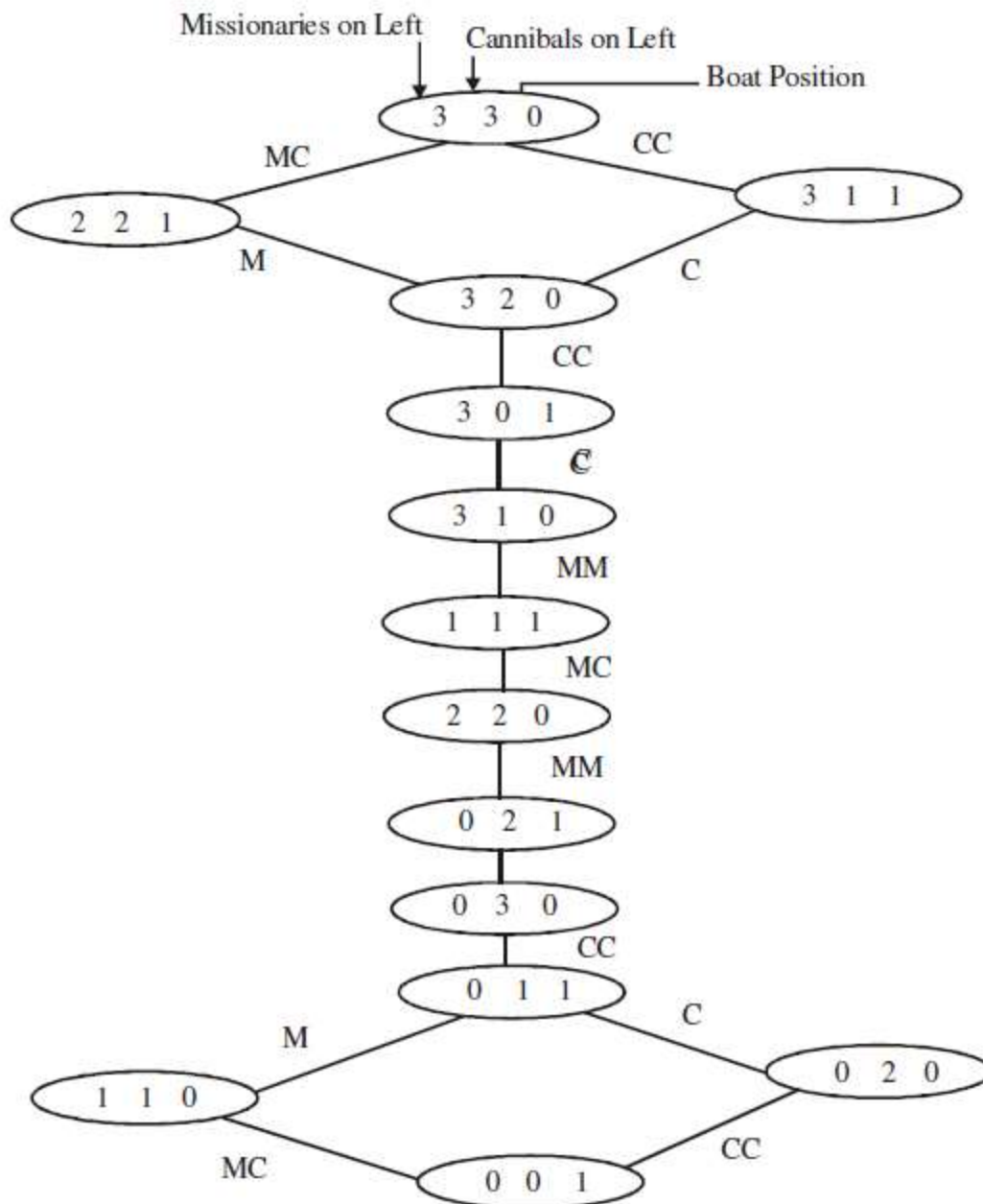
State representation:

1. BOAT position: original (T) or final (NIL) side of the river.
2. Number of Missionaries and Cannibals on the original side of the river.
3. Start is (T 3 3); Goal is (NIL 0 0).

Operators:

(MM 2 0)	Two Missionaries cross the river.
(MC 1 1)	One Missionary and one Cannibal.
(CC 0 2)	Two Cannibals.
(M 1 0)	One Missionary.
(C 0 1)	One Cannibal.

Missionaries/Cannibals Search Graph



2.4.1 Control Strategies

The word '**search**' refers to the search for a solution in a *problem space*.

- Search proceeds with different types of '**search control strategies**'.
- A strategy is defined by picking the order in which the nodes expand.

The Search strategies are evaluated along the following dimensions: Completeness, Time complexity, Space complexity, Optimality (the search- related terms are first explained, and then the search algorithms and control strategies are illustrated next).

Search-related terms

• Algorithm's performance and complexity

Ideally we want a common measure so that we can compare approaches in order to select the most appropriate algorithm for a given situation.

- ✓ *Performance* of an algorithm depends on internal and external factors.

Internal factors/ External factors

- **Time** required to run
 - **Size** of input to the algorithm
 - **Space** (memory) required to run
 - **Speed** of the computer
 - **Quality** of the compiler
- ✓ *Complexity* is a measure of the performance of an algorithm. *Complexity* measures the internal factors, usually in time than space.

• Computational complexity\

It is the measure of resources in terms of **Time** and **Space**.

- ✓ If **A** is an algorithm that solves a decision problem **f**, then run-time of **A** is the number of steps taken on the input of length **n**.
- ✓ *Time Complexity* **T(n)** of a decision problem **f** is the run-time of the 'best' algorithm **A** for **f**.
- ✓ *Space Complexity* **S(n)** of a decision problem **f** is the amount of memory used by the 'best' algorithm **A** for **f**.

• 'Big - O' notation

The **Big-O**, theoretical *measure of the execution of an algorithm*, usually indicates the **time** or the **memory** needed, given the problem size **n**, which is usually the number of items.

• Big-O notation

The **Big-O** notation is used to give an approximation to the *run-time- efficiency of an algorithm*; the letter '**O**' is for order of magnitude of operations or space at run-time.

• The **Big-O** of an Algorithm **A**

- ✓ If an algorithm **A** requires time proportional to **f(n)**, then algorithm **A** is said to be of order **f(n)**, and it is denoted as **O(f(n))**.
- ✓ If algorithm **A** requires time proportional to **n²**, then the order of the algorithm is said to be **O(n²)**.
- ✓ If algorithm **A** requires time proportional to **n**, then the order of the algorithm is said to be **O(n)**.

The function $f(n)$ is called the algorithm's *growth-rate function*. In other words, if an algorithm has performance complexity $O(n)$, this means that the run-time t should be directly proportional to n , ie $t \propto n$ or $t = k n$ where k is constant of proportionality.

Similarly, for algorithms having performance complexity $O(\log^2(n))$, $O(\log N)$, $O(N \log N)$, $O(2N)$ and so on.

Example 1:

Determine the **Big-O** of an algorithm:

Calculate the sum of the n elements in an integer array $a[0..n-1]$.

Line no.	Instructions	No of execution steps
line 1	sum	1
line 2	for (i = 0; i < n; i++)	$n + 1$
line 3	sum += a[i]	n
line 4	print sum	1
	Total	$2n + 3$

Thus, the polynomial $(2n + 3)$ is dominated by the 1st term as n while the number of elements in the array becomes very large.

- In determining the **Big-O**, ignore constants such as 2 and 3. So the algorithm is of order n .
- So the **Big-O** of the algorithm is $O(n)$.
- In other words the run-time of this algorithm increases roughly as the size of the input data n , e.g., an array of size n .

Tree structure

Tree is a way of organizing objects, related in a hierarchical fashion.

- Tree is a type of data structure in which each *element* is attached to one or more elements directly beneath it.
- The connections between elements are called *branches*.
- Tree is often called *inverted trees* because it is drawn with the *root* at the top.
- The elements that have no elements below them are called *leaves*.
- A *binary tree* is a special type: each element has only two branches below it.

Properties

- Tree is a special case of a *graph*.
- The topmost node in a tree is called the *root node*.
- At root node all operations on the tree begin.
- A node has at most one parent.
- The topmost node (root node) has no parents.
- Each node has zero or more *child nodes*, which are below it .
- The nodes at the bottommost level of the tree are called *leaf nodes*. Since *leaf nodes* are at the bottom most level, they do not have children.
- A node that has a child is called the child's *parent node*.
- The *depth of a node n* is the length of the path from the root to the node.
- The root node is at depth zero.

• Stacks and Queues

The *Stacks* and *Queues* are data structures that maintain the order of *last-in, first-out* and *first-in, first-out* respectively. Both *stacks* and *queues* are often implemented as linked lists, but that is not the only possible implementation.

Stack - Last In First Out (LIFO) lists

- An ordered list; a sequence of items, piled one on top of the other.
- The **insertions** and **deletions** are made at one end only, called **Top**.
- If Stack $S = (a[1], a[2], \dots, a[n])$ then $a[1]$ is bottom most element
- Any intermediate element ($a[i]$) is on top of element $a[i-1]$, $1 < i \leq n$.
- In Stack all operation take place on **Top**.

The **Pop** operation removes item from top of the stack.

The **Push** operation adds an item on top of the stack.

Queue - First In First Out (FIFO) lists

- An ordered list; a sequence of items; there are restrictions about how items can be added to and removed from the list. A queue has two ends.
- All **insertions** (enqueue) take place at one end, called **Rear** or **Back**
- All **deletions** (dequeue) take place at other end, called **Front**.
- If Queue has $a[n]$ as rear element then $a[i+1]$ is behind $a[i]$, $1 < i \leq n$.
- All operation takes place at one end of queue or the other.

The **Dequeue** operation removes the item at **Front** of the queue.

The **Enqueue** operation adds an item to the **Rear** of the queue.

Search

Search is the systematic examination of *states* to find path from the *start / root state* to the *goal state*.

- Search usually results from a lack of knowledge.
- Search explores knowledge alternatives to arrive at the best answer.
- Search algorithm output is a solution, that is, a path from the initial state to a state that satisfies the goal test.

For general-purpose problem-solving – ‘**Search**’ is an *approach*.

- Search deals with finding *nodes* having certain properties in a *graph* that represents search space.
- Search methods explore the search space ‘intelligently’, evaluating possibilities without investigating every single possibility.

Examples:

- For a Robot this might consist of PICKUP, PUTDOWN, MOVEFORWARD, MOVEBACK, MOVELEFT, and MOVERIGHT—until the goal is reached.
- Puzzles and Games have explicit rules: e.g., the ‘**Tower of Hanoi**’ puzzle

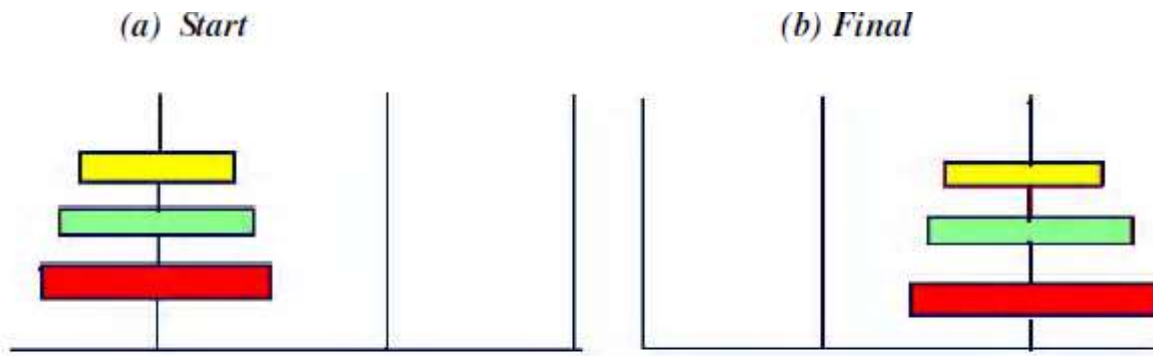


Fig. 2.4 Tower of Hanoi Puzzle

This puzzle involves a set of rings of different sizes that can be placed on three different pegs.

- The puzzle starts with the rings arranged as shown in Figure 2.4(a)
- The goal of this puzzle is to move them all as to Figure 2.4(b)
- Condition: Only the top ring on a peg can be moved, and it may only be placed on a smaller ring, or on an empty peg.

In this *Tower of Hanoi* puzzle:

- Situations encountered while solving the problem are described as *states*.
- Set of all possible configurations of rings on the pegs is called '*problem space*'.

• States

A *State* is a representation of elements in a given moment.

A problem is defined by its *elements* and their *relations*.

At each instant of a problem, the elements have specific descriptors and relations; the *descriptors* indicate how to select elements?

Among all possible states, there are two special states called:

- ✓ *Initial state* – the start point
- ✓ *Final state* – the goal state

• State Change: Successor Function

A '*successor function*' is needed for state change. The Successor Function moves one state to another state.

Successor Function:

- ✓ It is a description of possible actions; a set of operators.
- ✓ It is a transformation function on a state representation, which converts that state into another state.
- ✓ It defines a relation of accessibility among states.
- ✓ It represents the conditions of applicability of a state and corresponding transformation function.

• State space

A *state space* is the set of all *states* reachable from the *initial state*.

- ✓ A *state space* forms a *graph* (or map) in which the *nodes* are states and the *arcs* between nodes are actions.
- ✓ In a *state space*, a *path* is a sequence of states connected by a sequence of actions.
- ✓ The *solution* of a problem is part of the map formed by the *state space*.

- **Structure of a state space**

The *structures* of a *state space* are *trees* and *graphs*.

- ✓ A *tree* is a hierarchical structure in a graphical form.
- ✓ A *graph* is a non-hierarchical structure.
- A *tree* has only one path to a given node;
i.e., a *tree* has one and only one path from any point to any other point.
- A *graph* consists of a set of nodes (vertices) and a set of edges (arcs). Arcs establish relationships (connections) between the nodes; i.e., a graph has several paths to a given node.
- The *Operators* are directed *arcs* between nodes.

A *search* process explores the *state space*. In the worst case, the search explores all possible *paths* between the *initial state* and the *goal state*.

- **Problem solution**

In the *state space*, a *solution* is a path from the *initial state* to a *goal state* or, sometimes, just a *goal state*.

- ✓ A *solution cost function* assigns a numeric cost to each *path*; it also gives the cost of applying the *operators* to the *states*.
- ✓ A *solution quality* is measured by the *path cost function*; and an optimal solution has the lowest path cost among all solutions.
- ✓ The *solutions* can be *any or optimal or all*.
- ✓ The importance of cost depends on the problem and the type of solution asked

- **Problem description**

A problem consists of the description of:

- ✓ The current state of the world,
- ✓ The actions that can transform one state of the world into another,
- ✓ The desired state of the world.

The following action one taken to describe the problem:

- ✓ *State space* is defined explicitly or implicitly

A *state space* should describe everything that is needed to solve a problem and nothing that is not needed to solve the problem.

- ✓ *Initial state* is start state
- ✓ *Goal state* is the conditions it has to fulfill.

The description by a desired state may be complete or partial.

- ✓ *Operators* are to change state
- ✓ Operators do actions that can transform one state into another;
- ✓ Operators consist of: Preconditions and Instructions;

Preconditions provide partial description of the state of the world that must be true in order to perform the action, and

Instructions tell the user how to create the next state.

- Operators should be as general as possible, so as to reduce their number.
- *Elements of the domain* has relevance to the problem
 - ✓ Knowledge of the starting point.
- *Problem solving* is finding a solution
 - ✓ Find an ordered sequence of operators that transform the current (start) state into a goal state.

- *Restrictions* are solution quality any, optimal, or all
 - ✓ Finding the shortest sequence, or
 - ✓ finding the least expensive sequence defining cost, or
 - ✓ finding any sequence as quickly as possible.

This can also be explained with the help of algebraic function as given below.

PROBLEM CHARACTERISTICS

Heuristics cannot be generalized, as they are domain specific. Production systems provide ideal techniques for representing such heuristics in the form of IF-THEN rules. Most problems requiring simulation of intelligence use heuristic search extensively. Some heuristics are used to define the control structure that guides the search process, as seen in the example described above. But heuristics can also be encoded in the rules to represent the domain knowledge. Since most AI problems make use of knowledge and guided search through the knowledge, AI can be described as *the study of techniques for solving exponentially hard problems in polynomial time by exploiting knowledge about problem domain*.

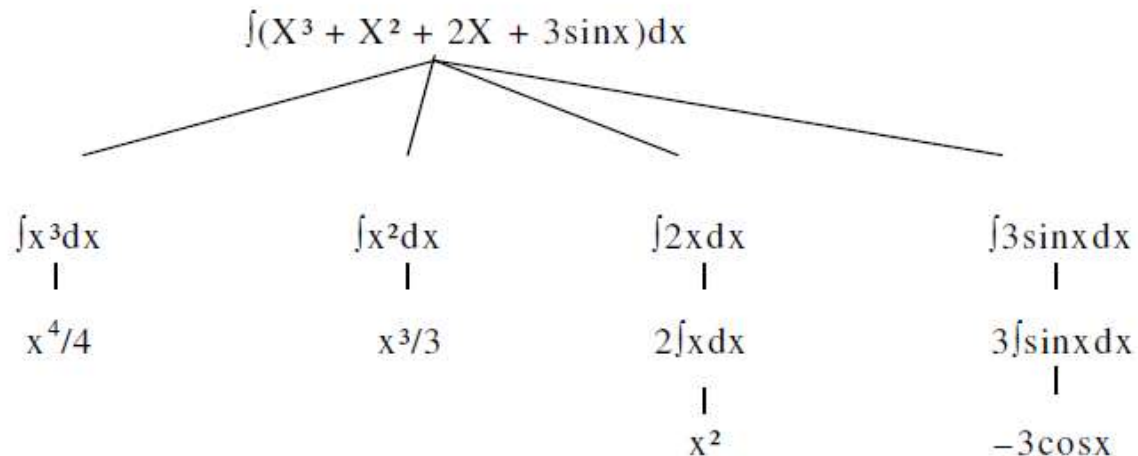
To use the heuristic search for problem solving, we suggest analysis of the problem for the following considerations:

- Decomposability of the problem into a set of independent smaller subproblems
- Possibility of undoing solution steps, if they are found to be unwise
- Predictability of the problem universe
- Possibility of obtaining an obvious solution to a problem without comparison of all other possible solutions
- Type of the solution: whether it is a state or a path to the goal state
- Role of knowledge in problem solving
- Nature of solution process: with or without interacting with the user

The general classes of engineering problems such as planning, classification, diagnosis, monitoring and design are generally knowledge intensive and use a large amount of heuristics. Depending on the type of problem, the knowledge representation schemes and control strategies for search are to be adopted. Combining heuristics with the two basic search strategies have been discussed above. There are a number of other general-purpose search techniques which are essentially heuristics based. Their efficiency primarily depends on how they exploit the domain-specific knowledge to abolish undesirable paths. Such search methods are called 'weak methods', since the progress of the search depends heavily on the way the domain knowledge is exploited. A few of such search techniques which form the centre of many AI systems are briefly presented in the following sections.

Problem Decomposition

Suppose to solve the expression is: $\int (X^3 + X^2 + 2X + 3\sin x) dx$



This problem can be solved by breaking it into smaller problems, each of which we can solve by using a small collection of specific rules. Using this technique of problem decomposition, we can solve very large problems very easily. This can be considered as an intelligent behaviour.

Can Solution Steps be Ignored?

Suppose we are trying to prove a mathematical theorem: first we proceed considering that proving a lemma will be useful. Later we realize that it is not at all useful. We start with another one to prove the theorem. Here we simply ignore the first method.

Consider the 8-puzzle problem to solve: we make a wrong move and realize that mistake. But here, the control strategy must keep track of all the moves, so that we can backtrack to the initial state and start with some new move.

Consider the problem of playing chess. Here, once we make a move we never recover from that step. These problems are illustrated in the three important classes of problems mentioned below:

1. Ignorable, in which solution steps can be ignored. Eg: Theorem Proving
2. Recoverable, in which solution steps can be undone. Eg: 8-Puzzle
3. Irrecoverable, in which solution steps cannot be undone. Eg: Chess

Is the Problem Universe Predictable?

Consider the 8-Puzzle problem. Every time we make a move, we know exactly what will happen. This means that it is possible to plan an entire sequence of moves and be confident what the resulting state will be. We can backtrack to earlier moves if they prove unwise.

Suppose we want to play Bridge. We need to plan before the first play, but we cannot play with certainty. So, the outcome of this game is very uncertain. In case of 8-Puzzle, the outcome is very certain. To solve uncertain outcome problems, we follow the process of plan revision as the plan is carried out and the necessary feedback is provided. The disadvantage is that the planning in this case is often very expensive.

Is Good Solution Absolute or Relative?

Consider the problem of answering questions based on a database of simple facts such as the following:

1. Siva was a man.
2. Siva was a worker in a company.
3. Siva was born in 1905.
4. All men are mortal.
5. All workers in a factory died when there was an accident in 1952.
6. No mortal lives longer than 100 years.

Suppose we ask a question: 'Is Siva alive?'

By representing these facts in a formal language, such as predicate logic, and then using formal inference methods we can derive an answer to this question easily.

There are two ways to answer the question shown below:

Method I:

1. Siva was a man.
2. Siva was born in 1905.
3. All men are mortal.
4. Now it is 2008, so Siva's age is 103 years.
5. No mortal lives longer than 100 years.

Method II:

1. Siva is a worker in the company.
2. All workers in the company died in 1952.

Answer: So Siva is not alive. It is the answer from the above methods.

We are interested to answer the question; it does not matter which path we follow. If we follow one path successfully to the correct answer, then there is no reason to go back and check another path to lead the solution.

CHARACTERISTICS OF PRODUCTION SYSTEMS

Production systems provide us with good ways of describing the operations that can be performed in a search for a solution to a problem.

At this time, two questions may arise:

1. Can production systems be described by a set of characteristics? And how can they be easily implemented?
2. What relationships are there between the problem types and the types of production systems well suited for solving the problems?

To answer these questions, first consider the following definitions of classes of production systems:

1. A monotonic production system is a production system in which the application of a rule never prevents the later application of another rule that could also have been applied at the time the first rule was selected.
2. A non-monotonic production system is one in which this is not true.
3. A partially communicative production system is a production system with the property that if the application of a particular sequence of rules transforms state P into state Q, then any combination of those rules that is allowable also transforms state P into state Q.
4. A commutative production system is a production system that is both monotonic and partially commutative.

Table 2.1 Four Categories of Production Systems

Production System	Monotonic	Non-monotonic
Partially Commutative	Theorem Proving	Robot Navigation
Non-partially Commutative	Chemical Synthesis	Bridge

Is there any relationship between classes of production systems and classes of problems? For any solvable problems, there exist an infinite number of production systems that show how to find solutions. Any problem that can be solved by any production system can be solved by a commutative one, but the commutative one is practically useless. It may use individual states to represent entire sequences of applications of rules of a simpler, non-commutative system. In the formal sense, there is no relationship between kinds of problems and kinds of production systems. Since all problems can be solved by all kinds of systems. But in the practical sense, there is definitely such a relationship between the kinds of problems and the kinds of systems that lend themselves to describing those problems.

Partially commutative, monotonic productions systems are useful for solving ignorable problems. These are important from an implementation point of view without the ability to backtrack to previous states when it is discovered that an incorrect path has been followed. Both types of partially commutative production systems are significant from an implementation point; they tend to lead to many duplications of individual states during the search process. Production systems that are not partially commutative are useful for many problems in which permanent changes occur.

Issues in the Design of Search Programs

Each search process can be considered to be a tree traversal. The object of the search is to find a path from the initial state to a goal state using a tree. The number of nodes generated might be huge; and in practice many of the nodes would not be needed. The secret of a good search routine is to generate only those nodes that are likely to be useful, rather than having a precise tree. The rules are used to represent the tree implicitly and only to create nodes explicitly if they are actually to be of use.

The following issues arise when searching:

- The tree can be searched forward from the initial node to the goal state or backwards from the goal state to the initial state.
- To select applicable rules, it is critical to have an efficient procedure for matching rules against states.
- How to represent each node of the search process? This is the knowledge representation problem or the frame problem. In games, an array suffices; in other problems, more complex data structures are needed.

Finally in terms of data structures, considering the water jug as a typical problem do we use a graph or tree? The breadth-first structure does take note of all nodes generated but the depth-first one can be modified.

Check duplicate nodes

1. Observe all nodes that are already generated, if a new node is present.
2. If it exists add it to the graph.
3. If it already exists, then
 - a. Set the node that is being expanded to the point to the already existing node corresponding to its successor rather than to the new one. The new one can be thrown away.
 - b. If the best or shortest path is being determined, check to see if this path is better or worse than the old one. If worse, do nothing.

Better save the new path and work the change in length through the chain of successor nodes if necessary.

Example: Tic-Tac-Toe

State spaces are good representations for board games such as Tic-Tac-Toe. The position of a game can be explained by the contents of the board and the player whose turn is next. The board can be represented as an array of 9 cells, each of which may contain an X or O or be empty.

• State:

- ✓ Player to move next: X or O.
- ✓ Board configuration:

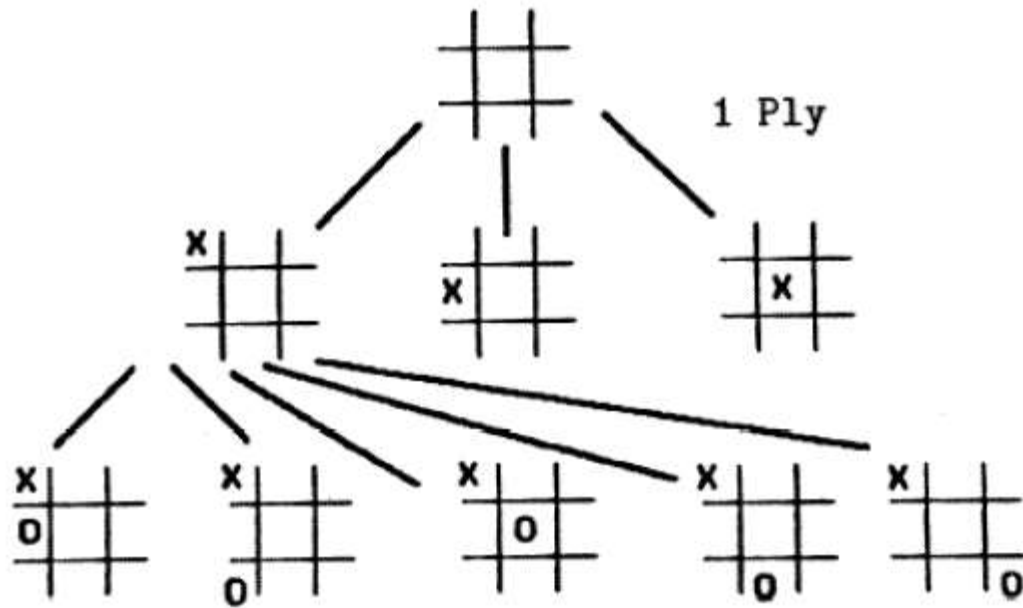
X		O
	O	
X		X

- **Operators:** Change an empty cell to X or O.
- **Start State:** Board empty; X's turn.
- **Terminal States:** Three X's in a row; Three O's in a row; All cells full.

Search Tree

The sequence of states formed by possible moves is called a *search tree*. Each level of the tree is called a *ply*.

Since the same state may be reachable by different sequences of moves, the state space may in general be a graph. It may be treated as a tree for simplicity, at the cost of duplicating states.



Solving problems using search

- Given an informal description of the problem, construct a formal description as a state space:
 - ✓ Define a data structure to represent the *state*.
 - ✓ Make a representation for the *initial state* from the given data.
 - ✓ Write programs to represent *operators* that change a given state representation to a new state representation.
 - ✓ Write a program to detect *terminal states*.
- Choose an appropriate search technique:
 - ✓ How large is the search space?
 - ✓ How well structured is the domain?
 - ✓ What knowledge about the domain can be used to guide the search?

HEURISTIC SEARCH TECHNIQUES:

Search Algorithms

Many traditional search algorithms are used in AI applications. For complex problems, the traditional algorithms are unable to find the solutions within some practical time and space limits. Consequently, many special techniques are developed, using **heuristic functions**. The algorithms that use *heuristic functions* are called **heuristic algorithms**.

- Heuristic algorithms are not really intelligent; they appear to be intelligent because they achieve better performance.
- Heuristic algorithms are more efficient because they take advantage of feedback from the data to direct the search path.
- **Uninformed search algorithms** or *Brute-force algorithms*, search through the search space all possible candidates for the solution checking whether each candidate satisfies the problem's statement.
- **Informed search algorithms** use heuristic functions that are specific to the problem, apply them to guide the search through the search space to try to reduce the amount of time spent in searching.

A good heuristic will make an informed search dramatically outperform any uninformed search: for example, the Traveling Salesman Problem (TSP), where the goal is to find a good solution instead of finding the best solution.

In such problems, the search proceeds using current information about the problem to predict which path is closer to the goal and follow it, although it does not always guarantee to find the best possible solution. Such techniques help in finding a solution within reasonable time and space (memory). Some prominent intelligent search algorithms are stated below:

1. **Generate and Test Search**
2. **Best-first Search**
3. **Greedy Search**
4. **A* Search**
5. **Constraint Search**
6. **Means-ends analysis**

There are some more algorithms. They are either improvements or combinations of these.

- **Hierarchical Representation of Search Algorithms:** A Hierarchical representation of most search algorithms is illustrated below. The representation begins with two types of search:
- **Uninformed Search:** Also called blind, exhaustive or brute-force search, it uses no information about the problem to guide the search and therefore may not be very efficient.
- **Informed Search:** Also called heuristic or intelligent search, this uses information about the problem to guide the search—usually guesses the distance to a goal state and is therefore efficient, but the search may not be always possible.

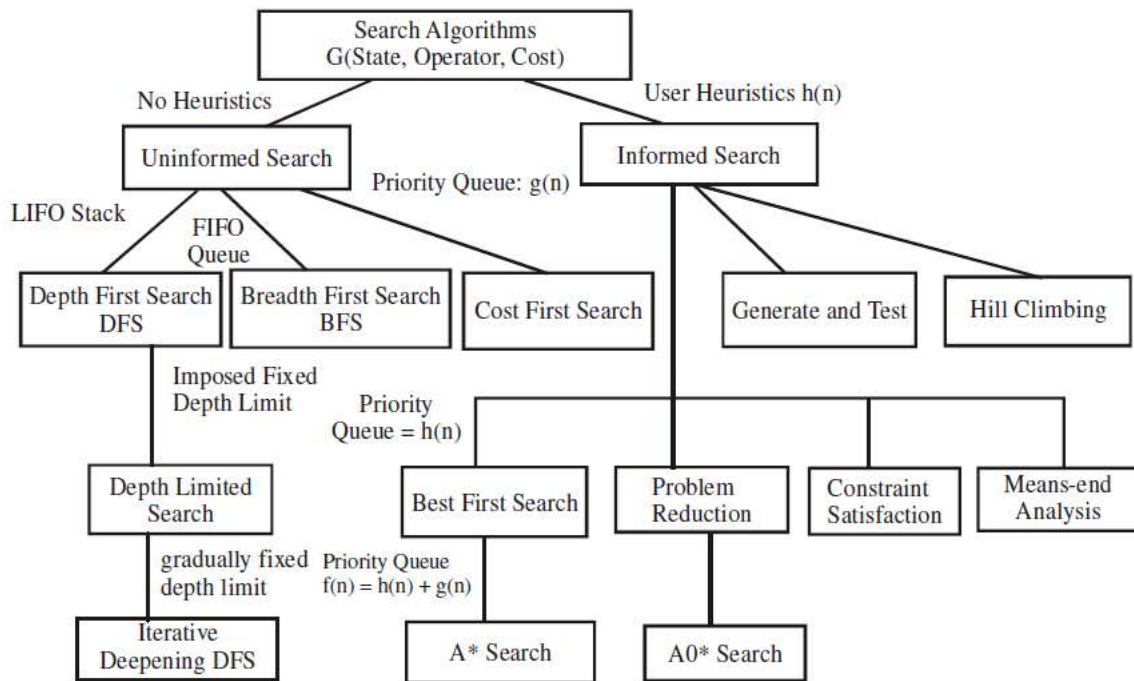
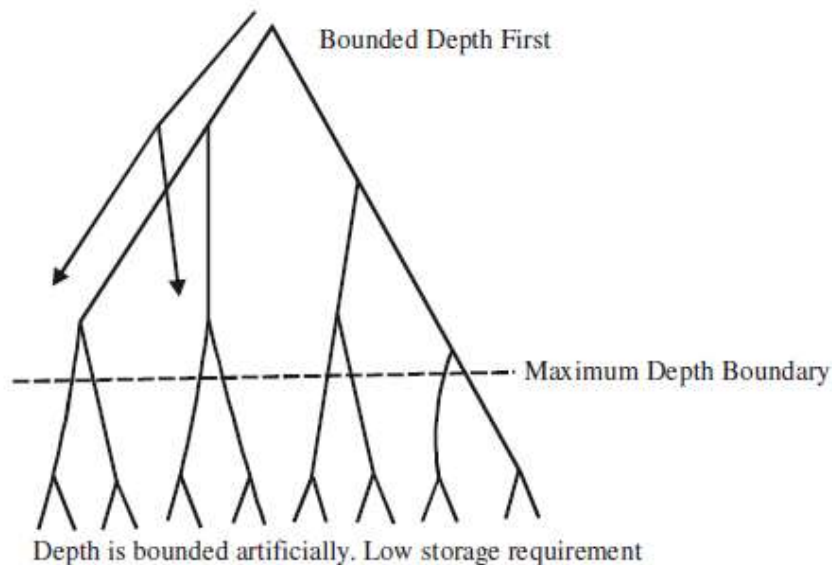
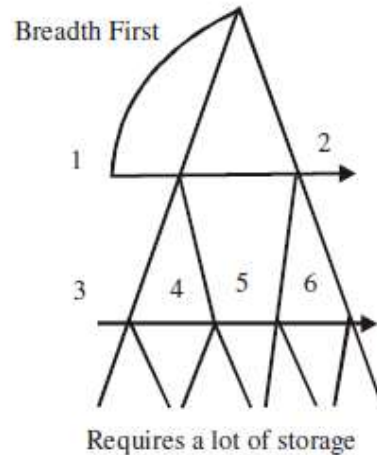
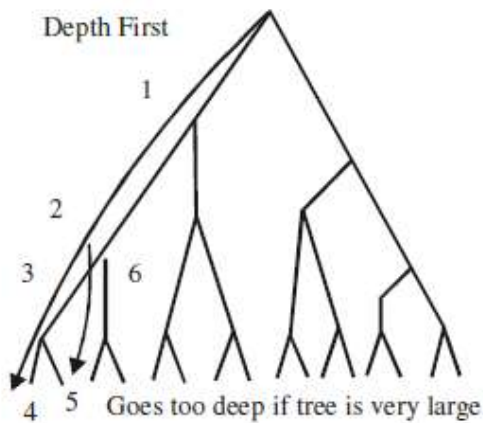


Fig. Different Search Algorithms

The first requirement is that it causes motion, in a game playing program, it moves on the board and in the water jug problem, filling water is used to fill jugs. It means the control strategies without the motion will never lead to the solution.

The second requirement is that it is systematic, that is, it corresponds to the need for global motion as well as for local motion. This is a clear condition that neither would it be rational to fill a jug and empty it repeatedly, nor it would be worthwhile to move a piece round and round on the board in a cyclic way in a game. We shall initially consider two systematic approaches for searching. Searches can be classified by the order in which operators are tried: depth-first, breadth-first, bounded depth-first.



Breadth-first search

A Search strategy, in which the highest layer of a decision tree is searched completely before proceeding to the next layer is called *Breadth-first search (BFS)*.

- In this strategy, no viable solutions are omitted and therefore it is guaranteed that an optimal solution is found.
- This strategy is often not feasible when the search space is large.

Algorithm

1. Create a variable called LIST and set it to be the starting state.
2. Loop until a goal state is found or LIST is empty, Do
 - a. Remove the first element from the LIST and call it E. If the LIST is empty, quit.
 - b. For every path each rule can match the state E, Do
 - (i) Apply the rule to generate a new state.
 - (ii) If the new state is a goal state, quit and return this state.
 - (iii) Otherwise, add the new state to the end of LIST.

Advantages

1. Guaranteed to find an optimal solution (in terms of shortest number of steps to reach the goal).
2. Can always find a goal node if one exists (complete).

Disadvantages

1. High storage requirement: *exponential* with tree depth.

Depth-first search

A search strategy that extends the current path as far as possible before backtracking to the last choice point and trying the next alternative path is called *Depth-first search (DFS)*.

- This strategy does not guarantee that the optimal solution has been found.
- In this strategy, search reaches a satisfactory solution more rapidly than breadth first, an advantage when the search space is large.

Algorithm

Depth-first search applies operators to each newly generated state, trying to drive directly toward the goal.

1. If the starting state is a goal state, quit and return success.
2. Otherwise, do the following until success or failure is signalled:
 - a. Generate a successor E to the starting state. If there are no more successors, then signal failure.
 - b. Call Depth-first Search with E as the starting state.
 - c. If success is returned signal success; otherwise, continue in the loop.

Advantages

1. Low storage requirement: *linear* with tree depth.
2. Easily programmed: function call stack does most of the work of maintaining state of the search.

Disadvantages

1. May find a sub-optimal solution (one that is deeper or more costly than the best solution).
2. Incomplete: without a depth bound, may not find a solution even if one exists.

2.4.2.3 Bounded depth-first search

Depth-first search can spend much time (perhaps infinite time) exploring a very deep path that does not contain a solution, when a shallow solution exists. An easy way to solve this problem is to put a maximum depth bound on the search. Beyond the depth bound, a failure is generated automatically without exploring any deeper.

Problems:

1. It's hard to guess how deep the solution lies.
2. If the estimated depth is too deep (even by 1) the computer time used is dramatically increased, by a factor of *bextra*.
3. If the estimated depth is too shallow, the search fails to find a solution; all that computer time is wasted.

Heuristics

A heuristic is a method that improves the efficiency of the search process. These are like tour guides. There are good to the level that they may neglect the points in general interesting directions; they are bad to the level that they may neglect points of interest to particular individuals. Some heuristics help in the search process without sacrificing any claims to entirety that the process might previously had. Others may occasionally cause an excellent path to be overlooked. By sacrificing entirety it increases efficiency. Heuristics may not find the best

solution every time but guarantee that they find a good solution in a reasonable time. These are particularly useful in solving tough and complex problems, solutions of which would require infinite time, i.e. far longer than a lifetime for the problems which are not solved in any other way.

Heuristic search

To find a solution in proper time rather than a complete solution in unlimited time we use heuristics. 'A heuristic function is a function that maps from problem state descriptions to measures of desirability, usually represented as numbers'. Heuristic search methods use knowledge about the problem domain and choose promising operators first. These heuristic search methods use heuristic functions to evaluate the next state towards the goal state. For finding a solution, by using the heuristic technique, one should carry out the following steps:

1. Add domain—specific information to select what is the best path to continue searching along.
2. Define a heuristic function $h(n)$ that estimates the 'goodness' of a node n . Specifically, $h(n)$ = estimated cost(or distance) of minimal cost path from n to a goal state.
3. The term, heuristic means 'serving to aid discovery' and is an estimate, based on domain specific information that is computable from the current state description of how close we are to a goal.

Finding a route from one city to another city is an example of a search problem in which different search orders and the use of heuristic knowledge are easily understood.

1. State: The current city in which the traveller is located.
2. Operators: Roads linking the current city to other cities.
3. Cost Metric: The cost of taking a given road between cities.
4. Heuristic information: The search could be guided by the direction of the goal city from the current city, or we could use airline distance as an estimate of the distance to the goal.

Heuristic search techniques

For complex problems, the traditional algorithms, presented above, are unable to find the solution within some practical time and space limits. Consequently, many special techniques are developed, using **heuristic functions**.

- Blind search is not always possible, because it requires too much time or Space (memory).

Heuristics are **rules of thumb**; they do not guarantee a solution to a problem.

- Heuristic Search is a weak technique but can be effective if applied correctly; it requires domain specific information.

Characteristics of heuristic search

- Heuristics are knowledge about domain, which help search and reasoning in its domain.
- Heuristic search incorporates domain knowledge to improve efficiency over blind search.
- Heuristic is a function that, when applied to a state, returns value as estimated merit of state, with respect to goal.
 - ✓ Heuristics might (for reasons) *underestimate* or *overestimate* the merit of a state with respect to goal.
 - ✓ Heuristics that underestimate are desirable and called admissible.
- Heuristic evaluation function estimates likelihood of given state leading to goal state.
- Heuristic search function estimates cost from current state to goal, presuming function is efficient.

Heuristic search compared with other search

The Heuristic search is compared with Brute force or Blind search techniques below:

Comparison of Algorithms

Brute force / Blind search

Can only search what it has knowledge about already

No knowledge about how far a node node from goal state

Heuristic search

Estimates 'distance' to goal state through explored nodes

Guides search process toward goal

Prefers states (nodes) that lead close to and not away from goal state

Example: Travelling salesman

A salesman has to visit a list of cities and he must visit each city only once. There are different routes between the cities. The problem is to find the shortest route between the cities so that the salesman visits all the cities at once.

Suppose there are N cities, then a solution would be to take $N!$ possible combinations to find the shortest distance to decide the required route. This is not efficient as with $N=10$ there are 36,28,800 possible routes. This is an example of *combinatorial explosion*.

There are better methods for the solution of such problems: one is called *branch and bound*. First, generate all the complete paths and find the distance of the first complete path. If the next path is shorter, then save it and proceed this way avoiding the path when its length exceeds the saved shortest path length, although it is better than the previous method.

Generate and Test Strategy

Generate-And-Test Algorithm

Generate-and-test search algorithm is a very simple algorithm that guarantees to find a solution if done systematically and there exists a solution.

Algorithm: Generate-And-Test

1. Generate a possible solution.
2. Test to see if this is the expected solution.
3. If the solution has been found quit else go to step 1.

Potential solutions that need to be generated vary depending on the kinds of problems. For some problems the possible solutions may be particular points in the problem space and for some problems, paths from the start state.

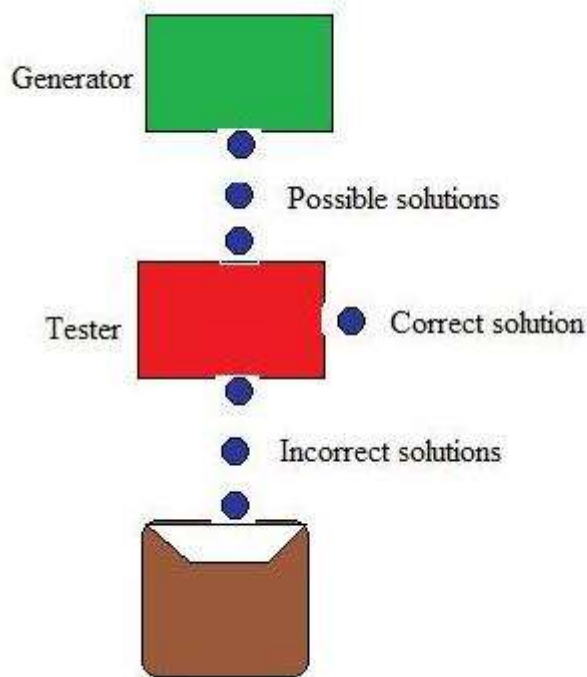


Figure: Generate And Test

Generate-and-test, like depth-first search, requires that complete solutions be generated for testing. In its most systematic form, it is only an exhaustive search of the problem space. Solutions can also be generated randomly but solution is not guaranteed. This approach is what is known as British Museum algorithm: finding an object in the British Museum by wandering randomly.

Systematic Generate-And-Test

While generating complete solutions and generating random solutions are the two extremes there exists another approach that lies in between. The approach is that the search process proceeds systematically but some paths that unlikely to lead the solution are not considered. This evaluation is performed by a heuristic function.

Depth-first search tree with backtracking can be used to implement systematic generate-and-test procedure. As per this procedure, if some intermediate states are likely to appear often in the tree, it would be better to modify that procedure to traverse a graph rather than a tree.

Generate-And-Test And Planning

Exhaustive generate-and-test is very useful for simple problems. But for complex problems even heuristic generate-and-test is not very effective technique. But this may be made effective by combining with other techniques in such a way that the space in which to search is restricted. An AI program DENDRAL, for example, uses plan-Generate-and-test technique. First, the planning process uses constraint-satisfaction techniques and creates lists of recommended and contraindicated substructures. Then the generate-and-test procedure uses the lists generated and required to explore only a limited set of structures. Constrained in this way, generate-and-test proved highly effective. A major weakness of planning is that it often produces inaccurate solutions as there is no feedback from the world. But if it is used to produce only pieces of solutions then lack of detailed accuracy becomes unimportant.

Hill Climbing

Hill Climbing is heuristic search used for mathematical optimization problems in the field of Artificial Intelligence .

Given a large set of inputs and a good heuristic function, it tries to find a sufficiently good solution to the problem. This solution may not be the global optimal maximum.

- In the above definition, mathematical optimization problems implies that hill climbing solves the problems where we need to maximize or minimize a given real function by choosing values from the given inputs. Example-[Travelling salesman problem](#) where we need to minimize the distance traveled by salesman.
- 'Heuristic search' means that this search algorithm may not find the optimal solution to the problem. However, it will give a good solution in reasonable time.
- A heuristic function is a function that will rank all the possible alternatives at any branching step in search algorithm based on the available information. It helps the algorithm to select the best route out of possible routes.

Features of Hill Climbing

1. Variant of generate and test algorithm : It is a variant of generate and test algorithm. The generate and test algorithm is as follows :

1. *Generate a possible solutions.*
2. *Test to see if this is the expected solution.*
3. *If the solution has been found quit else go to step 1.*

Hence we call Hill climbing as a variant of generate and test algorithm as it takes the feedback from test procedure. Then this feedback is utilized by the generator in deciding the next move in search space.

2. Uses the Greedy approach : At any point in state space, the search moves in that direction only which optimizes the cost of function with the hope of finding the optimal solution at the end.

Types of Hill Climbing

1. Simple Hill climbing : It examines the neighboring nodes one by one and selects the first neighboring node which optimizes the current cost as next node.

Algorithm for Simple Hill climbing :

Step 1 : Evaluate the initial state. If it is a goal state then stop and return success. Otherwise, make initial state as current state.

Step 2 : Loop until the solution state is found or there are no new operators present which can be applied to current state.

a) Select a state that has not been yet applied to the current state and apply it to produce a new state.

b) Perform these to evaluate new state

- i. If the current state is a goal state, then stop and return success.*
- ii. If it is better than the current state, then make it current state and proceed further.*
- iii. If it is not better than the current state, then continue in the loop until a solution is found.*

Step 3 : Exit.

2. Steepest-Ascent Hill climbing : It first examines all the neighboring nodes and then selects the node closest to the solution state as next node.

Step 1 : Evaluate the initial state. If it is goal state then exit else make the current state as initial state

Step 2 : Repeat these steps until a solution is found or current state does not change

i. Let 'target' be a state such that any successor of the current state will be better than it;

ii. for each operator that applies to the current state

a. apply the new operator and create a new state

b. evaluate the new state

c. if this state is goal state then quit else compare with 'target'

d. if this state is better than 'target', set this state as 'target'

e. if target is better than current state set current state to Target

Step 3 : Exit

3. Stochastic hill climbing : It does not examine all the neighboring nodes before deciding which node to select .It just selects a neighboring node at random, and decides (based on the amount of improvement in that neighbor) whether to move to that neighbor or to examine another.

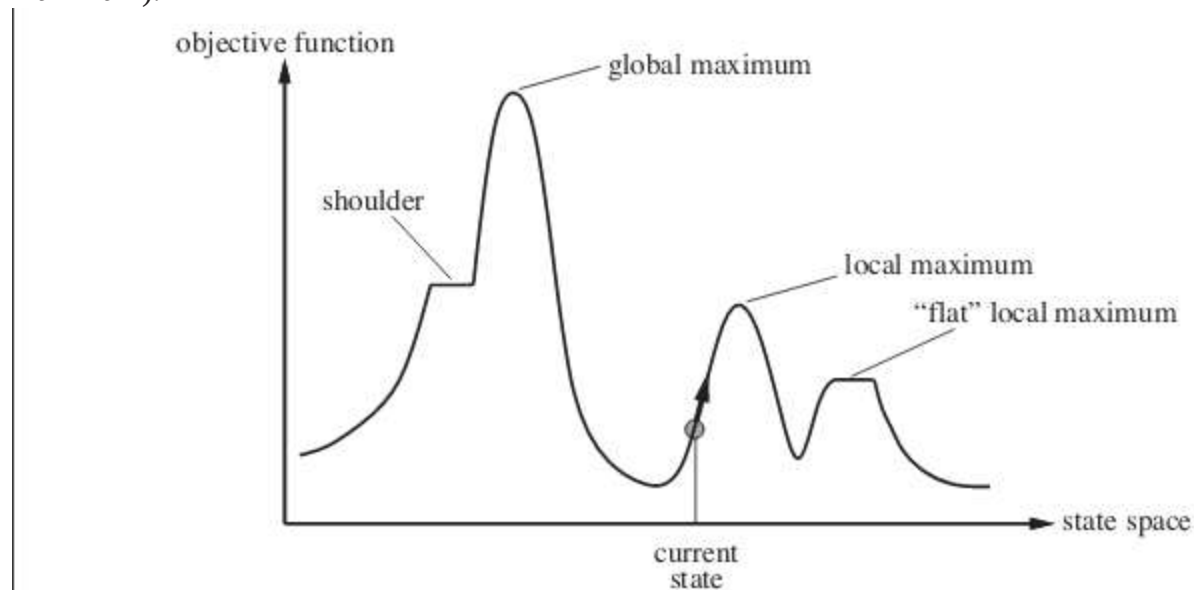
State Space diagram for Hill Climbing

State space diagram is a graphical representation of the set of states our search algorithm can reach vs the value of our objective function(the function which we wish to maximize).

X-axis : denotes the state space ie states or configuration our algorithm may reach.

Y-axis : denotes the values of objective function corresponding to a particular state.

The best solution will be that state space where objective function has maximum value(global maximum).



Different regions in the State Space Diagram

1. Local maximum : It is a state which is better than its neighboring state however there exists a state which is better than it(global maximum). This state is better because here value of objective function is higher than its neighbors.

2. Global maximum : It is the best possible state in the state space diagram. This because at this state, objective function has highest value.
3. Plateau/flat local maximum : It is a flat region of state space where neighboring states have the same value.
4. Ridge : It is region which is higher than its neighbours but itself has a slope. It is a special kind of local maximum.
5. Current state : The region of state space diagram where we are currently present during the search.
6. Shoulder : It is a plateau that has an uphill edge.

Problems in different regions in Hill climbing

Hill climbing cannot reach the optimal/best state(global maximum) if it enters any of the following regions :

1. Local maximum : At a local maximum all neighboring states have a values which is worse than than the current state. Since hill climbing uses greedy approach, it will not move to the worse state and terminate itself. The process will end even though a better solution may exist.
To overcome local maximum problem : Utilize backtracking technique. Maintain a list of visited states. If the search reaches an undesirable state, it can backtrack to the previous configuration and explore a new path.
2. Plateau : On plateau all neighbors have same value . Hence, it is not possible to select the best direction.

To overcome plateaus : Make a big jump. Randomly select a state far away from current state. Chances are that we will land at a non-plateau region

3. Ridge : Any point on a ridge can look like peak because movement in all possible directions is downward. Hence the algorithm stops when it reaches this state.
To overcome Ridge : In this kind of obstacle, use two or more rules before testing. It implies moving in several directions at once.

Best First Search (Informed Search)

In BFS and DFS, when we are at a node, we can consider any of the adjacent as next node. So both BFS and DFS blindly explore paths without considering any cost function. The idea of Best First Search is to use an evaluation function to decide which adjacent is most promising and then explore. Best First Search falls under the category of Heuristic Search or Informed Search.

We use a priority queue to store costs of nodes. So the implementation is a variation of BFS, we just need to change Queue to PriorityQueue.

Algorithm:

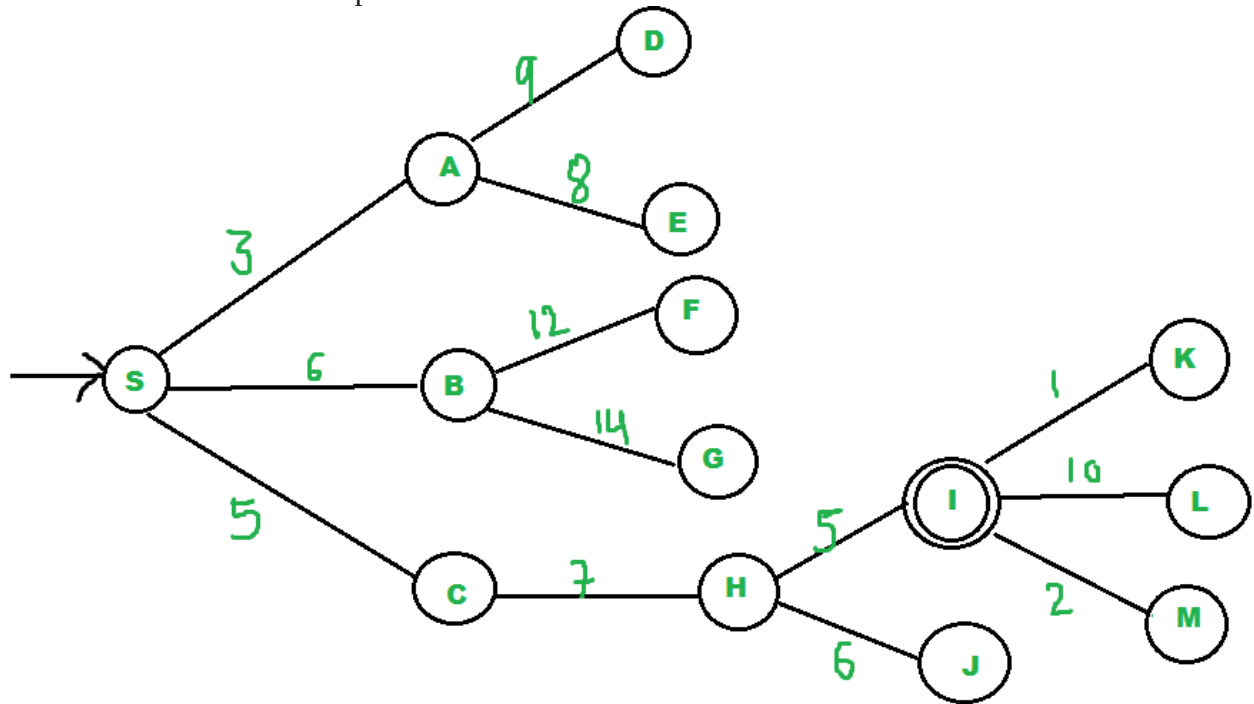
Best-First-Search(Grah g, Node start)

- 1) Create an empty PriorityQueue
PriorityQueue pq;
- 2) Insert "start" in pq.
pq.insert(start)
- 3) Until PriorityQueue is empty
u = PriorityQueue.DeleteMin

```

    If u is the goal
      Exit
    Else
      Foreach neighbor v of u
        If v "Unvisited"
          Mark v "Visited"
          pq.insert(v)
          Mark v "Examined"
      End procedure
  
```

Let us consider below example.



We start from source "S" and search for goal "I" using given costs and Best First search.

pq initially contains S
 We remove s from and process unvisited neighbors of S to pq.
 pq now contains {A, C, B} (C is put before B because C has lesser cost)

We remove A from pq and process unvisited neighbors of A to pq.
 pq now contains {C, B, E, D}

We remove C from pq and process unvisited neighbors of C to pq.
pq now contains {B, H, E, D}

We remove B from pq and process unvisited neighbors of B to pq.
pq now contains {H, E, D, F, G}

We remove H from pq. Since our goal "I" is a neighbor of H, we return.

Analysis :

- The worst case time complexity for Best First Search is $O(n * \log n)$ where n is number of nodes. In worst case, we may have to visit all nodes before we reach goal. Note that priority queue is implemented using Min(or Max) Heap, and insert and remove operations take $O(\log n)$ time.
- Performance of the algorithm depends on how well the cost or evaluation function is designed.

A* Search Algorithm

A* is a type of search algorithm. Some problems can be solved by representing the world in the initial state, and then for each action we can perform on the world we generate states for what the world would be like if we did so. If you do this until the world is in the state that we specified as a solution, then the route from the start to this goal state is the solution to your problem.

In this tutorial I will look at the use of state space search to find the shortest path between two points (pathfinding), and also to solve a simple sliding tile puzzle (the 8-puzzle). Let's look at some of the terms used in Artificial Intelligence when describing this state space search.

Some terminology

A *node* is a state that the problem's world can be in. In pathfinding a node would be just a 2d coordinate of where we are at the present time. In the 8-puzzle it is the positions of all the tiles. Next all the nodes are arranged in a *graph* where links between nodes represent valid steps in solving the problem. These links are known as *edges*. In the 8-puzzle diagram the edges are shown as blue lines. See figure 1 below.

State space search, then, is solving a problem by beginning with the start state, and then for each node we expand all the nodes beneath it in the graph by applying all the possible moves that can be made at each point.

Heuristics and Algorithms

At this point we introduce an important concept, the *heuristic*. This is like an algorithm, but with a key difference. An algorithm is a set of steps which you can follow to solve a problem, which always works for valid input. For example you could probably write an algorithm yourself for

multiplying two numbers together on paper. A heuristic is not guaranteed to work but is useful in that it may solve a problem for which there is no algorithm.

We need a heuristic to help us cut down on this huge search problem. What we need is to use our heuristic at each node to make an estimate of how far we are from the goal. In pathfinding we know exactly how far we are, because we know how far we can move each step, and we can calculate the exact distance to the goal.

But the 8-puzzle is more difficult. There is no known algorithm for calculating from a given position how many moves it will take to get to the goal state. So various heuristics have been devised. The best one that I know of is known as the Nilsson score which leads fairly directly to the goal most of the time, as we shall see.

Cost

When looking at each node in the graph, we now have an idea of a heuristic, which can estimate how close the state is to the goal. Another important consideration is the cost of getting to where we are. In the case of pathfinding we often assign a movement cost to each square. The cost is the same then the cost of each square is one. If we wanted to differentiate between terrain types we may give higher costs to grass and mud than to newly made road. When looking at a node we want to add up the cost of what it took to get here, and this is simply the sum of the cost of this node and all those that are above it in the graph.

8 Puzzle

Let's look at the 8 puzzle in more detail. This is a simple sliding tile puzzle on a 3*3 grid where one tile is missing and you can move the other tiles into the gap until you get the puzzle into the goal position. See figure 1.

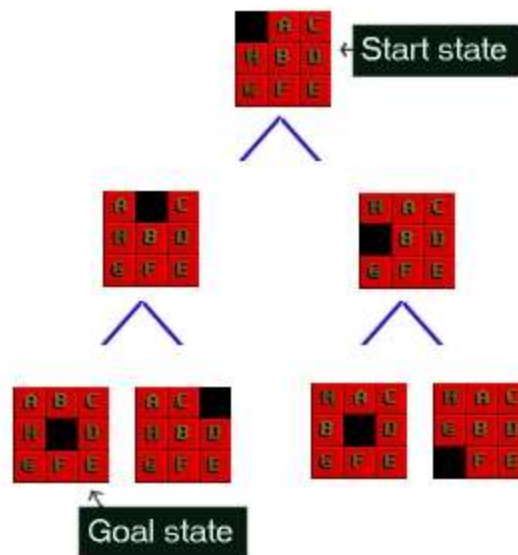


Figure 1 : The 8-Puzzle state space for a very simple example

There are 362,880 different states that the puzzle can be in, and to find a solution the search has to find a route through them. From most positions of the search the number of edges (that's the

blue lines) is two. That means that the number of nodes you have in each level of the search is 2^d where d is the depth. If the number of steps to solve a particular state is 18, then that's 262,144 nodes just at that level.

The 8 puzzle game state is as simple as representing a list of the 9 squares and what's in them. Here are two states for example; the last one is the GOAL state, at which point we've found the solution. The first is a jumbled up example that you may start from.

Start state SPACE, A, C, H, B, D, G, F, E

Goal state A, B, C, H, SPACE, D, G, F, E

The rules that you can apply to the puzzle are also simple. If there is a blank tile above, below, to the left or to the right of a given tile, then you can move that tile into the space. To solve the puzzle you need to find the path from the start state, through the graph down to the goal state.

There is example code to to solve the 8-puzzle on the [github](#) site.

Pathfinding

In a video game, or some other pathfinding scenario, you want to search a state space and find out how to get from somewhere you are to somewhere you want to be, without bumping into walls or going too far. For reasons we will see later, the A* algorithm will not only find a path, if there is one, but it will find the shortest path. A state in pathfinding is simply a position in the world. In the example of a maze game like Pacman you can represent where everything is using a simple 2d grid. The start state for a ghost say, would be the 2d coordinate of where the ghost is at the start of the search. The goal state would be where pacman is so we can go and eat him.

There is also example code to do pathfinding on the [github](#) site.

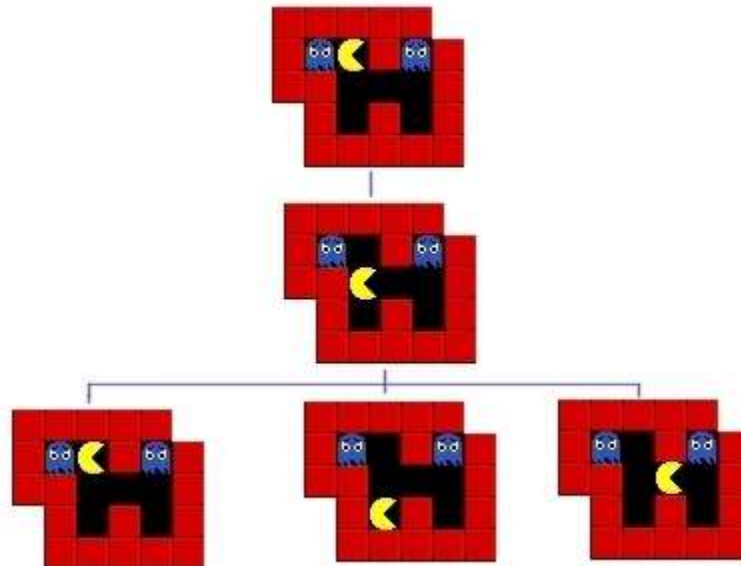


Figure 2 : The first three steps of a pathfinding state space

Implementing A*

We are now ready to look at the operation of the A* algorithm. What we need to do is start with the goal state and then generate the graph downwards from there. Let's take the 8-puzzle in figure 1. We ask how many moves can we make from the start state? The answer is 2, there are two directions we can move the blank tile, and so our graph expands.

If we were just to continue blindly generating successors to each node, we could potentially fill the computer's memory before we found the goal node. Obviously we need to remember the best nodes and search those first. We also need to remember the nodes that we have expanded already, so that we don't expand the same state repeatedly.

Let's start with the OPEN list. This is where we will remember which nodes we haven't yet expanded. When the algorithm begins the start state is placed on the open list, it is the only state we know about and we have not expanded it. So we will expand the nodes from the start and put those on the OPEN list too. Now we are done with the start node and we will put that on the CLOSED list. The CLOSED list is a list of nodes that we have expanded.

$$f = g + h$$

Using the OPEN and CLOSED list lets us be more selective about what we look at next in the search. We want to look at the best nodes first. We will give each node a score on how good we think it is. This score should be thought of as the cost of getting from the node to the goal plus the cost of getting to where we are. Traditionally this has been represented by the letters f, g and h. 'g' is the sum of all the costs it took to get here, 'h' is our heuristic function, the estimate of what it will take to get to the goal. 'f' is the sum of these two. We will store each of these in our nodes.

Using the f, g and h values the A* algorithm will be directed, subject to conditions we will look at further on, towards the goal and will find it in the shortest route possible.

So far we have looked at the components of the A*, let's see how they all fit together to make the algorithm :

Pseudocode

Hopefully the ideas we looked at in the preceding paragraphs will now click into place as we look at the A* algorithm pseudocode. You may find it helpful to print this out or leave the window open while we discuss it.

To help make the operation of the algorithm clear we will look again at the 8-puzzle problem in figure 1 above. Figure 3 below shows the f,g and h scores for each of the tiles.

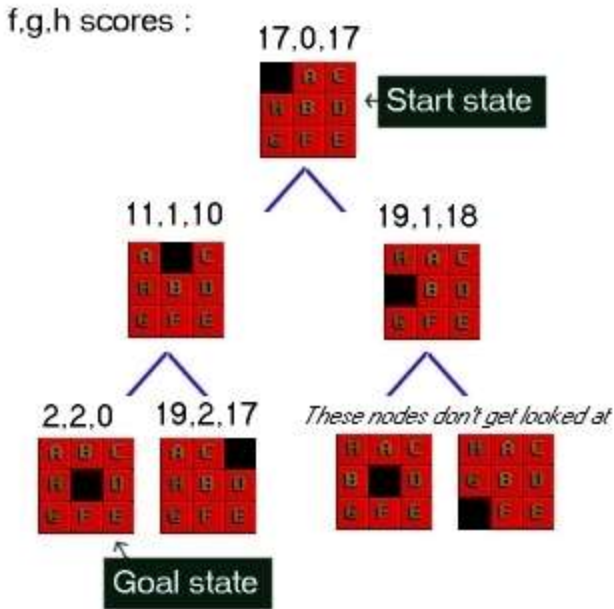


Figure 3 : 8-Puzzle state space showing f,g,h scores

First of all look at the g score for each node. This is the cost of what it took to get from the start to that node. So in the picture the center number is g . As you can see it increases by one at each level. In some problems the cost may vary for different state changes. For example in pathfinding there is sometimes a type of terrain that costs more than other types. Next look at the last number in each triple. This is h , the heuristic score. As I mentioned above I am using a heuristic known as Nilsson's Sequence, which converges quickly to a correct solution in many cases. Here is how you calculate this score for a given 8-puzzle state :

Advantages:

It is complete and optimal.

It is the best one from other techniques. It is used to solve very complex problems.

It is optimally efficient, i.e. there is no other optimal algorithm guaranteed to expand fewer nodes than A^* .

Disadvantages:

This algorithm is complete if the branching factor is finite and every action has fixed cost.

The speed execution of A^* search is highly dependant on the accuracy of the heuristic algorithm that is used to compute $h(n)$.

AO* Search: (And-Or) Graph

The Depth first search and Breadth first search given earlier for OR trees or graphs can be easily adopted by AND-OR graph. The main difference lies in the way termination conditions are determined, since all goals following an AND nodes must be realized; where as a single goal node following an OR node will do. So for this purpose we are using AO* algorithm.

Like A* algorithm here we will use two arrays and one heuristic function.

OPEN:

It contains the nodes that has been traversed but yet not been marked solvable or unsolvable.

CLOSE:

It contains the nodes that have already been processed.

6 7:The distance from current node to goal node.

Algorithm:

Step 1: Place the starting node into OPEN.

Step 2: Compute the most promising solution tree say T0.

Step 3: Select a node n that is both on OPEN and a member of T0. Remove it from OPEN and place it in

CLOSE

Step 4: If n is the terminal goal node then leveled n as solved and leveled all the ancestors of n as solved. If the starting node is marked as solved then success and exit.

Step 5: If n is not a solvable node, then mark n as unsolvable. If starting node is marked as unsolvable, then return failure and exit.

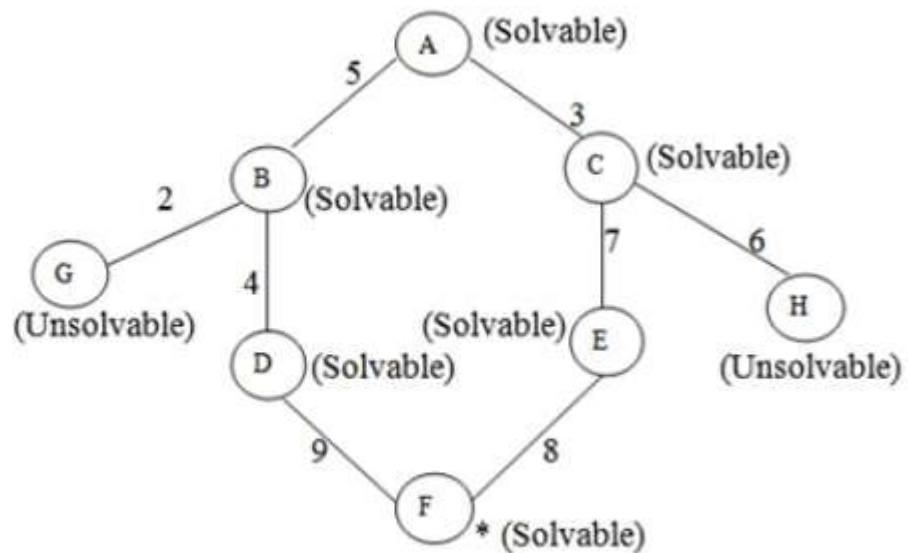
Step 6: Expand n. Find all its successors and find their h (n) value, push them into OPEN.

Step 7: Return to Step 2.

Step 8: Exit.

Implementation:

Let us take the following example to implement the AO* algorithm.



Figure

Step 1:

In the above graph, the solvable nodes are A, B, C, D, E, F and the unsolvable nodes are G, H. Take A as the starting node. So place A into OPEN.

i.e. OPEN = A CLOSE = (NULL) ϕ A

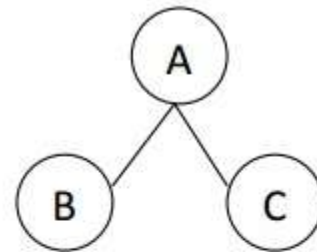
Step 2:

The children of A are B and C which are solvable. So place them into OPEN and place A into the CLOSE.

i.e. OPEN =



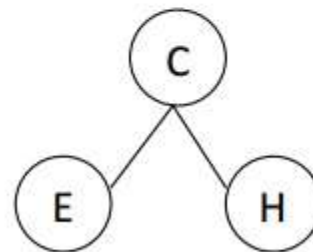
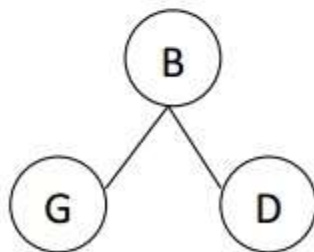
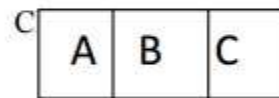
CLOSE =



Step 3:

Now process the nodes B and C. The children of B and C are to be placed into OPEN. Also remove B and C from OPEN and place them into CLOSE.

So OPEN =



(O)

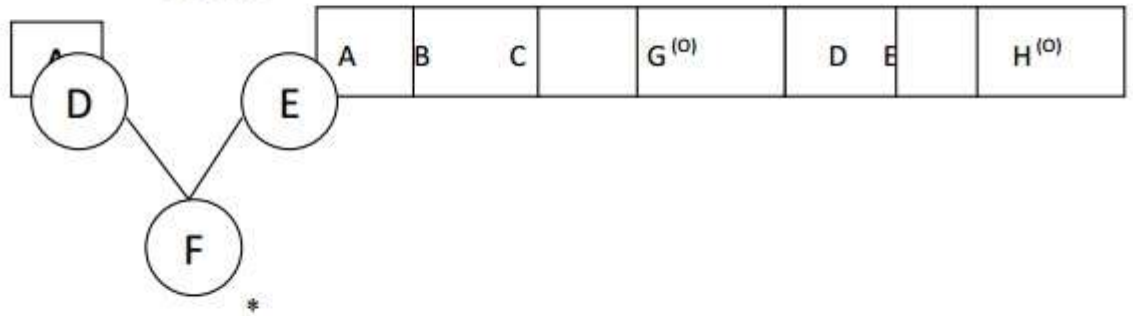
'O' indicated that the nodes G and H are unsolvable.

Step 4:

As the nodes G and H are unsolvable, so place them into CLOSE directly and process the nodes D and E.

i.e. OPEN =

CLOSE =



Step 5:

Now we have been reached at our goal state. So place F into CLOSE.

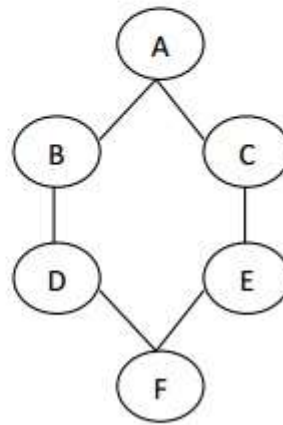
A	B	C		G ^(O)	D	E		H ^(O)	F
---	---	---	--	------------------	---	---	--	------------------	---

i.e. CLOSE =

Step 6:

Success and Exit

AO* Graph:



Figure

Advantages:

It is an optimal algorithm.

If traverse according to the ordering of nodes. It can be used for both OR and AND graph.

Disadvantages:

Sometimes for unsolvable nodes, it can't find the optimal path. Its complexity is than other algorithms.

PROBLEM REDUCTION

Problem Reduction with AO* Algorithm.

When a problem can be divided into a set of sub problems, where each sub problem can be solved separately and a combination of these will be a solution, AND-OR graphs or AND - OR trees are used for representing the solution. The decomposition of the problem or problem reduction generates AND arcs. One AND are may point to any number of successor nodes. All

these must be solved so that the arc will rise to many arcs, indicating several possible solutions. Hence the graph is known as AND - OR instead of AND. Figure shows an AND - OR graph.

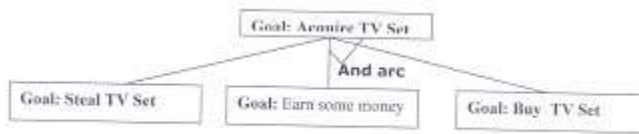


Figure shows AND - Or graph - an example.

An algorithm to find a solution in an AND - OR graph must handle AND area appropriately. A* algorithm can not search AND - OR graphs efficiently. This can be understood from the given figure.

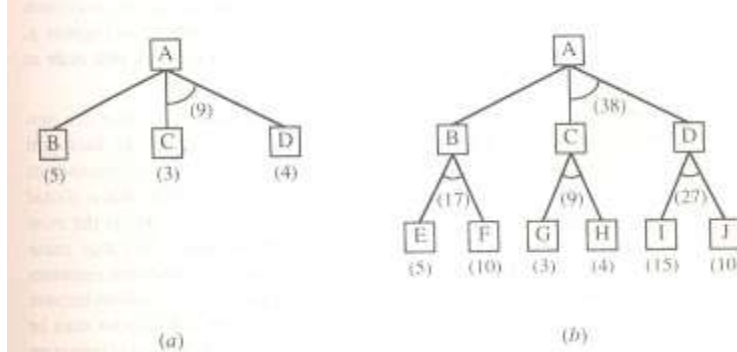


Figure 3.7: AND-OR Graphs

FIGURE : AND - OR graph

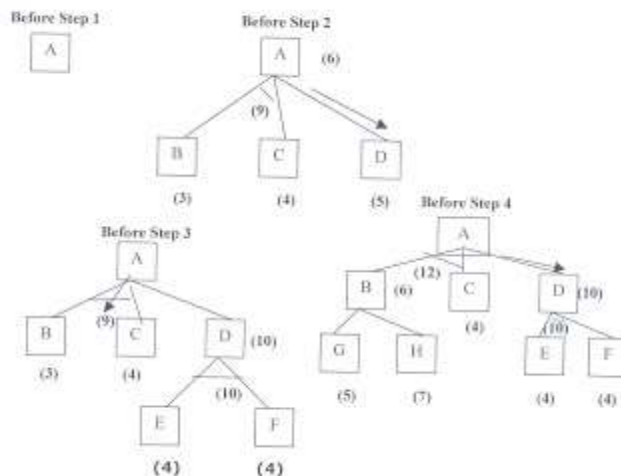
In figure (a) the top node A has been expanded producing two areas one leading to B and leading to C-D. The numbers at each node represent the value of f' at that node (cost of getting to the goal state from current state). For simplicity, it is assumed that every operation (i.e. applying a rule) has unit cost, i.e., each arc with single successor will have a cost of 1 and each of its components. With the available information till now, it appears that C is the most promising node to expand since its $f' = 3$, the lowest but going through B would be better since to use C we must also use D and the cost would be $9(3+4+1+1)$. Through B it would be $6(5+1)$.

Thus the choice of the next node to expand depends not only on a value but also on whether that node is part of the current best path from the initial node. Figure (b) makes this clearer. In figure the node G appears to be the most promising node, with the least f' value. But G is not on the current best path, since to use G we must use GH with a cost of 9 and again this demands that arcs be used (with a cost of 27). The path from A through B, E-F is better with a total cost of $(17+1)=18$. Thus we can see that to search an AND-OR graph, the following three things must be done.

1. traverse the graph starting at the initial node and following the current best path, and accumulate the set of nodes that are on the path and have not yet been expanded.
2. Pick one of these unexpanded nodes and expand it. Add its successors to the graph and compute f' (cost of the remaining distance) for each of them.

3. Change the f' estimate of the newly expanded node to reflect the new information produced by its successors. Propagate this change backward through the graph. Decide which of the current best path.

The propagation of revised cost estimation backward in the tree is not necessary in A* algorithm. This is because in AO* algorithm expanded nodes are re-examined so that the current best path can be selected. The working of AO* algorithm is illustrated in figure as follows:



Referring the figure, The initial node is expanded and D is Marked initially as promising node. D is expanded producing an AND arc E-F. f' value of D is updated to 10. Going backwards we can see that the AND arc B-C is better. It is now marked as current best path. B and C have to be expanded next. This process continues until a solution is found or all paths have led to dead ends, indicating that there is no solution. An A* algorithm the path from one node to the other is always that of the lowest cost and it is independent of the paths through other nodes.

The algorithm for performing a heuristic search of an AND - OR graph is given below. Unlike A* algorithm which used two lists OPEN and CLOSED, the AO* algorithm uses a single structure G. G represents the part of the search graph generated so far. Each node in G points down to its immediate successors and up to its immediate predecessors, and also has with it the value of h' cost of a path from itself to a set of solution nodes. The cost of getting from the start nodes to the current node "g" is not stored as in the A* algorithm. This is because it is not possible to compute a single such value since there may be many paths to the same state. In AO* algorithm serves as the estimate of goodness of a node. Also a threshold value called FUTILITY is used. The estimated cost of a solution is greater than FUTILITY then the search is abandoned as too expensive to be practical.

For representing above graphs AO* algorithm is as follows

AO* ALGORITHM:

1. Let G consists only to the node representing the initial state call this node INTT. Compute h' (INIT).
2. Until INIT is labeled SOLVED or h_i (INIT) becomes greater than FUTILITY, repeat the following procedure.

- (I) Trace the marked arcs from INIT and select an unbounded node NODE.
- (II) Generate the successors of NODE . if there are no successors then assign FUTILITY as h' (NODE). This means that NODE is not solvable. If there are successors then for each one called SUCCESSOR, that is not also an ancestor of NODE do the following
- (a) add SUCCESSOR to graph G
 - (b) if successor is not a terminal node, mark it solved and assign zero to its h' value.
 - (c) If successor is not a terminal node, compute its h' value.
- (III) propagate the newly discovered information up the graph by doing the following . let S be a set of nodes that have been marked SOLVED. Initialize S to NODE. Until S is empty repeat the following procedure;
- (a) select a node from S call it CURRENT and remove it from S.
 - (b) compute h' of each of the arcs emerging from CURRENT , Assign minimum h' to CURRENT.
 - (c) Mark the minimum cost path as the best out of CURRENT.
 - (d) Mark CURRENT SOLVED if all of the nodes connected to it through the new marked are have been labeled SOLVED.
 - (e) If CURRENT has been marked SOLVED or its h' has just changed, its new status must be propagate backwards up the graph . hence all the ancestors of CURRENT are added to S.

(Referred From Artificial Intelligence TMH)

AO* Search Procedure.

1. Place the start node on open.
2. Using the search tree, compute the most promising solution tree TP .
3. Select node n that is both on open and a part of tp, remove n from open and place it on closed.
4. If n is a goal node, label n as solved. If the start node is solved, exit with success where tp is the solution tree, remove all nodes from open with a solved ancestor.

5. If n is not solvable node, label n as unsolvable. If the start node is labeled as unsolvable, exit with failure. Remove all nodes from open, with unsolvable ancestors.

6. Otherwise, expand node n generating all of its successor compute the cost of for each newly generated node and place all such nodes on open.

7. Go back to step(2)

Note: AO* will always find minimum cost solution.

CONSTRAINT SATISFACTION:-

Many problems in AI can be considered as problems of constraint satisfaction, in which the goal state satisfies a given set of constraint. constraint satisfaction problems can be solved by using any of the search strategies. The general form of the constraint satisfaction procedure is as follows:

Until a complete solution is found or until all paths have led to lead ends, do

1. select an unexpanded node of the search graph.
2. Apply the constraint inference rules to the selected node to generate all possible new constraints.
3. If the set of constraints contains a contradiction, then report that this path is a dead end.
4. If the set of constraints describes a complete solution then report success.
5. If neither a constraint nor a complete solution has been found then apply the rules to generate new partial solutions. Insert these partial solutions into the search graph.

Example: consider the crypt arithmetic problems.

```
  SEND
+ MORE
-----
 MONEY
-----
```

Assign decimal digit to each of the letters in such a way that the answer to the problem is correct to the same letter occurs more than once, it must be assigned the same digit each time. no two different letters may be assigned the same digit. Consider the crypt arithmetic problem.


```

  SEND
+ MORE
-----
 MONEY
-----

```

CONSTRAINTS:-

1. no two digit can be assigned to same letter.
2. only single digit number can be assign to a letter.
1. no two letters can be assigned same digit.
2. Assumption can be made at various levels such that they do not contradict each other.
3. The problem can be decomposed into secured constraints. A constraint satisfaction approach may be used.
4. Any of search techniques may be used.
5. Backtracking may be performed as applicable us applied search techniques.
6. Rule of arithmetic may be followed.

Initial state of problem.

D=?

E=?

Y=?

N=?

R=?

O=?

S=?

M=?

C1=?

C2=?

C1 ,C 2, C3 stands for the carry variables respectively.

Goal State: the digits to the letters must be assigned in such a manner so that the sum is satisfied.

Solution Process:

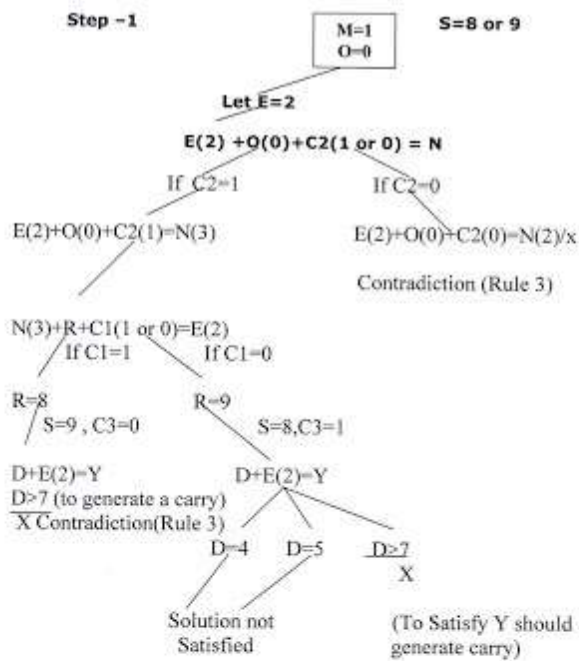
We are following the depth-first method to solve the problem.

1. initial guess $m=1$ because the sum of two single digits can generate at most a carry '1'.

2. When $n=1$ $o=0$ or 1 because the largest single digit number added to $m=1$ can generate the sum of either 0 or 1 depend on the carry received from the carry sum. By this we conclude that $o=0$ because m is already 1 hence we cannot assign same digit another letter(rule no.)

3. We have $m=1$ and $o=0$ to get $o=0$ we have $s=8$ or 9 , again depending on the carry received from the earlier sum.

The same process can be repeated further. The problem has to be composed into various constraints. And each constraints is to be satisfied by guessing the possible digits that the letters can be assumed that the initial guess has been already made . rest of the process is being shown in the form of a tree, using depth-first search for the clear understandability of the solution process.

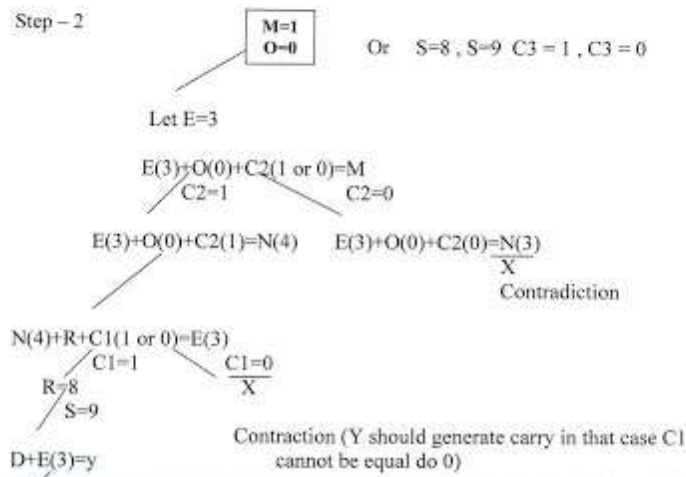


Contradiction for value of 0 Comes

X

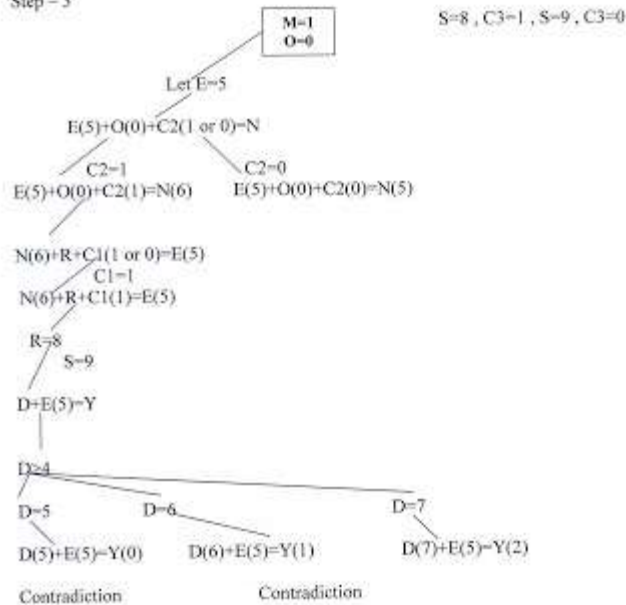
After Step 1 we derive are more conclusion that Y contradiction should generate a Carry. That is $D+2>9$

Step - 2



$D>6$ (Controuduction)

After Step 2, we found that C1 cannot be Zero, Since Y has to generate a carry to satisfy goal state. From this step onwards, no need to branch for C1=0.
Step - 3



At Step (4) we have assigned a single digit to every letter in accordance with the constraints & production rules.
Now by backtracking, we find the different digits assigned to different letters and hence reach the solution state.

Solution State:-

$$Y = 2$$

$$D = 7$$

$$S = 9$$

$$R = 8$$

$$N = 6$$

$$E = 5$$

$$O = 0$$

$$M = 1$$

$$C1 = 1$$

$$C2 = 0$$

$$C3 = 0$$

	C3(0)	C2(1)	C1(1)	
	S(9)	E(5)	N(6)	D(7)
+	M(1)	O(0)	R(8)	E(5)
	M(1)	O(0)	N(6)	E(5)
				Y(2)

MEANS - ENDS ANALYSIS:-

Most of the search strategies either reason forward or backward however, often a mixture of the two directions is appropriate. Such a mixed strategy would make it possible to solve the major parts of the problem first and solve the smaller problems that arise when combining them together. Such a technique is called "Means - Ends Analysis".

The means -ends analysis process centers around finding the difference between the current state and the goal state. The problem space of means - ends analysis has an initial state and one or more goal states, a set of operators with a set of preconditions, their application and difference functions that compute the difference between two states $a(i)$ and $s(j)$. A problem is solved using means - ends analysis by

1. Computing the current state s_1 to a goal state s_2 and computing their difference D_{12} .
2. Satisfy the preconditions for some recommended operator op is selected, then to reduce the difference D_{12} .
3. The operator OP is applied if possible. If not the current state is solved a goal is created and means- ends analysis is applied recursively to reduce the sub goal.
4. If the sub goal is solved state is restored and work resumed on the original problem.

(the first AI program to use means - ends analysis was the GPS General problem solver)

means- ends analysis is useful for many human planning activities. Consider the example of planning for an office worker. Suppose we have a different table of three rules:

1. If in our current state we are hungry , and in our goal state we are not hungry , then either the "visit hotel" or "visit Canteen " operator is recommended.
2. If in our current state we do not have money , and if in our goal state we have money, then the "Visit our bank" operator or the "Visit secretary" operator is recommended.
3. If in our current state we do not know where something is , need in our goal state we do know, then either the "visit office enquiry" , "visit secretary" or "visit co worker " operator is recommended.

KNOWLEDGE REPRESENTATION

KNOWLEDGE REPRESENTATION:-

For the purpose of solving complex problems encountered in AI, we need both a large amount of knowledge and some mechanism for manipulating that knowledge to create solutions to new problems. A variety of ways of representing knowledge (facts) have been exploited in AI programs. In all variety of knowledge representations, we deal with two kinds of entities.

A. Facts: Truths in some relevant world. These are the things we want to represent.

B. Representations of facts in some chosen formalism. These are things we will actually be able to manipulate.

One way to think of structuring these entities is at two levels: (a) the knowledge level, at which facts are described, and (b) the symbol level, at which representations of objects at the knowledge level are defined in terms of symbols that can be manipulated by programs.

The facts and representations are linked with two-way mappings. This link is called representation mappings. The forward representation mapping maps from facts to representations. The backward representation mapping goes the other way, from representations to facts.

One common representation is natural language (particularly English) sentences. Regardless of the representation for facts we use in a program, we may also need to be concerned with an English representation of those facts in order to facilitate getting information into and out of the system. We need mapping functions from English sentences to the representation we actually use and from it back to sentences.

Representations and Mappings

- In order to solve complex problems encountered in artificial intelligence, one needs both a large amount of knowledge and some mechanism for manipulating that knowledge to create solutions.
- Knowledge and Representation are two distinct entities. They play central but distinguishable roles in the intelligent system.
- Knowledge is a description of the world. It determines a system's competence by what it knows.
- Moreover, Representation is the way knowledge is encoded. It defines a system's performance in doing something.
- Different types of knowledge require different kinds of representation.

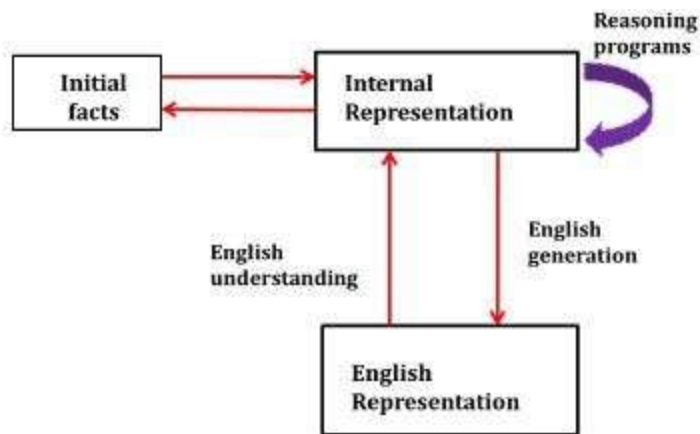


Fig: Mapping between Facts and Representations

The Knowledge Representation models/mechanisms are often based on:

- Logic
- Rules
- Frames
- Semantic Net

Knowledge is categorized into two major types:

1. Tacit corresponds to “informal” or “implicit“
 - Exists within a human being;
 - It is embodied.
 - Difficult to articulate formally.
 - Difficult to communicate or share.
 - Moreover, Hard to steal or copy.
 - Drawn from experience, action, subjective insight
2. Explicit formal type of knowledge, Explicit
 - Explicit knowledge
 - Exists outside a human being;
 - It is embedded.
 - Can be articulated formally.
 - Also, Can be shared, copied, processed and stored.
 - So, Easy to steal or copy
 - Drawn from the artifact of some type as a principle, procedure, process, concepts.

A variety of ways of representing knowledge have been exploited in AI programs.

There are two different kinds of entities, we are dealing with.

1. Facts: Truth in some relevant world. Things we want to represent.
2. Also, Representation of facts in some chosen formalism. Things we will actually be able to manipulate.

These entities structured at two levels:

1. The knowledge level, at which facts described.
2. Moreover, The symbol level, at which representation of objects defined in terms of symbols that can manipulate by programs

Framework of Knowledge Representation

- The computer requires a well-defined problem description to process and provide a well-defined acceptable solution.

- Moreover, To collect fragments of knowledge we need first to formulate a description in our spoken language and then represent it in formal language so that computer can understand.
- Also, The computer can then use an algorithm to compute an answer.

So, This process illustrated as,

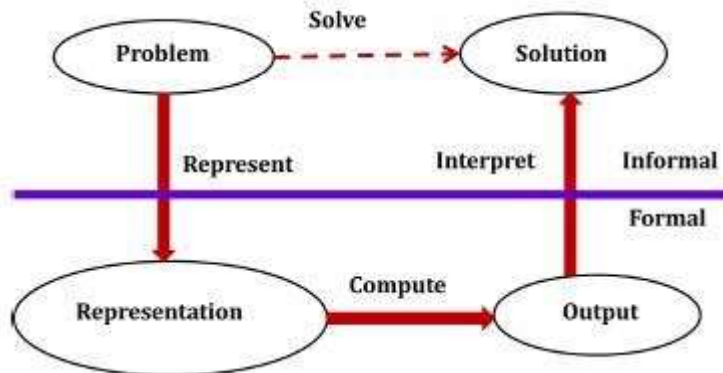


Fig: Knowledge Representation Framework

The steps are:

- The informal formalism of the problem takes place first.
- It then represented formally and the computer produces an output.
- This output can then represented in an informally described solution that user understands or checks for consistency.

The Problem solving requires,

- Formal knowledge representation, and
- Moreover, Conversion of informal knowledge to a formal knowledge that is the conversion of implicit knowledge to explicit knowledge.

Mapping between Facts and Representation

- Knowledge is a collection of facts from some domain.
- Also, We need a representation of “facts“ that can manipulate by a program.
- Moreover, Normal English is insufficient, too hard currently for a computer program to draw inferences in natural languages.
- Thus some symbolic representation is necessary.

A good knowledge representation enables fast and accurate access to knowledge and understanding of the content.

A knowledge representation system should have following properties.

1. Representational Adequacy
 - The ability to represent all kinds of knowledge that are needed in that domain.
2. Inferential Adequacy
 - Also, The ability to manipulate the representational structures to derive new structures corresponding to new knowledge inferred from old.
3. Inferential Efficiency
 - The ability to incorporate additional information into the knowledge structure that can be used to focus the attention of the inference mechanisms in the most promising direction.
4. Acquisitional Efficiency
 - Moreover, The ability to acquire new knowledge using automatic methods wherever possible rather than reliance on human intervention.

Knowledge Representation Schemes

Relational Knowledge

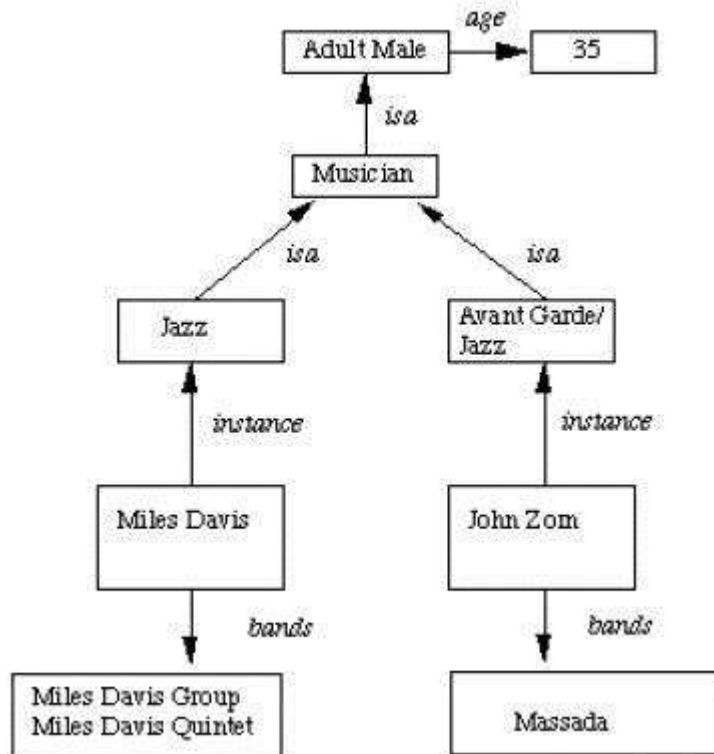
- The simplest way to represent declarative facts is a set of relations of the same sort used in the database system.
- Provides a framework to compare two objects based on equivalent attributes. o Any instance in which two different objects are compared is a relational type of knowledge.
- The table below shows a simple way to store facts.
 - Also, The facts about a set of objects are put systematically in columns.
 - This representation provides little opportunity for inference.

Player	Height	Weight	Bats - Throws
Aaron	6-0	180	Right - Right
Mays	5-10	170	Right - Right
Ruth	6-2	215	Left - Left
Williams	6-3	205	Left - Right

- Given the facts, it is not possible to answer a simple question such as: “Who is the heaviest player?”
- Also, But if a procedure for finding the heaviest player is provided, then these facts will enable that procedure to compute an answer.
- Moreover, We can ask things like who “bats – left” and “throws – right”.

Inheritable Knowledge

- Here the knowledge elements inherit attributes from their parents.
- The knowledge embodied in the design hierarchies found in the functional, physical and process domains.
- Within the hierarchy, elements inherit attributes from their parents, but in many cases, not all attributes of the parent elements prescribed to the child elements.
- Also, The inheritance is a powerful form of inference, but not adequate.
- Moreover, The basic KR (Knowledge Representation) needs to augment with inference mechanism.
- Property inheritance: The objects or elements of specific classes inherit attributes and values from more general classes.
- So, The classes organized in a generalized hierarchy.



- Boxed nodes — objects and values of attributes of objects.
- Arrows — the point from object to its value.
- This structure is known as a slot and filler structure, semantic network or a collection of frames.

The steps to retrieve a value for an attribute of an instance object:

1. Find the object in the knowledge base
2. If there is a value for the attribute report it
3. Otherwise look for a value of an instance, if none fail
4. Also, Go to that node and find a value for the attribute and then report it
5. Otherwise, search through using is until a value is found for the attribute.

Inferential Knowledge

- This knowledge generates new information from the given information.
- This new information does not require further data gathering from source but does require analysis of the given information to generate new knowledge.
- Example: given a set of relations and values, one may infer other values or relations. A predicate logic (a mathematical deduction) used to infer from a set of attributes. Moreover, Inference through predicate logic uses a set of logical operations to relate individual data.
- Represent knowledge as formal logic: All dogs have tails $\forall x: dog(x) \rightarrow hastail(x)$
- Advantages:
 - A set of strict rules.
 - Can use to derive more facts.
 - Also, Truths of new statements can be verified.
 - Guaranteed correctness.
- So, Many inference procedures available to implement standard rules of logic popular in AI systems. e.g Automated theorem proving.

Procedural Knowledge

- A representation in which the control information, to use the knowledge, embedded in the knowledge itself. For example, computer programs, directions, and recipes; these indicate specific use or implementation;
- Moreover, Knowledge encoded in some procedures, small programs that know how to do specific things, how to proceed.
- Advantages:
 - Heuristic or domain-specific knowledge can represent.
 - Moreover, Extended logical inferences, such as default reasoning facilitated.
 - Also, Side effects of actions may model. Some rules may become false in time. Keeping track of this in large systems may be tricky.
- Disadvantages:
 - Completeness — not all cases may represent.
 - Consistency — not all deductions may be correct. e.g If we know that Fred is a bird we might deduce that Fred can fly. Later we might discover that Fred is an emu.
 - Modularity sacrificed. Changes in knowledge base might have far-reaching effects.
 - Cumbersome control information.

USING PREDICATE LOGIC

Representation of Simple Facts in Logic

Propositional logic is useful because it is simple to deal with and a decision procedure for it exists.

Also, In order to draw conclusions, facts are represented in a more convenient way as,

1. Marcus is a man.
 - `man(Marcus)`
2. Plato is a man.
 - `man(Plato)`
3. All men are mortal.
 - `mortal(men)`

But propositional logic fails to capture the relationship between an individual being a man and that individual being a mortal.

- How can these sentences be represented so that we can infer the third sentence from the first two?
- Also, Propositional logic commits only to the existence of facts that may or may not be the case in the world being represented.
- Moreover, It has a simple syntax and simple semantics. It suffices to illustrate the process of inference.
- Propositional logic quickly becomes impractical, even for very small worlds.

Predicate logic

First-order Predicate logic (FOPL) models the world in terms of

- Objects, which are things with individual identities
- Properties of objects that distinguish them from other objects
- Relations that hold among sets of objects

- Functions, which are a subset of relations where there is only one “value” for any given “input”

First-order Predicate logic (FOPL) provides

- Constants: a, b, dog33. Name a specific object.
- Variables: X, Y. Refer to an object without naming it.
- Functions: Mapping from objects to objects.
- Terms: Refer to objects
- Atomic Sentences: in(dad-of(X), food6) Can be true or false, Correspond to propositional symbols P, Q.

A well-formed formula (wff) is a sentence containing no “free” variables. So, That is, all variables are “bound” by universal or existential quantifiers.

$(\forall x)P(x, y)$ has x bound as a universally quantified variable, but y is free.

Quantifiers

Universal quantification

- $(\forall x)P(x)$ means that P holds for all values of x in the domain associated with that variable
- E.g., $(\forall x) \text{dolphin}(x) \rightarrow \text{mammal}(x)$

Existential quantification

- $(\exists x)P(x)$ means that P holds for some value of x in the domain associated with that variable
- E.g., $(\exists x) \text{mammal}(x) \wedge \text{lays-eggs}(x)$

Also, Consider the following example that shows the use of predicate logic as a way of representing knowledge.

1. Marcus was a man.
2. Marcus was a Pompeian.
3. All Pompeians were Romans.
4. Caesar was a ruler.
5. Also, All Pompeians were either loyal to Caesar or hated him.
6. Everyone is loyal to someone.
7. People only try to assassinate rulers they are not loyal to.
8. Marcus tried to assassinate Caesar.

The facts described by these sentences can be represented as a set of well-formed formulas (wffs) as follows:

1. Marcus was a man.
 - $\text{man}(\text{Marcus})$
2. Marcus was a Pompeian.
 - $\text{Pompeian}(\text{Marcus})$
3. All Pompeians were Romans.
 - $\forall x: \text{Pompeian}(x) \rightarrow \text{Roman}(x)$
4. Caesar was a ruler.
 - $\text{ruler}(\text{Caesar})$
5. All Pompeians were either loyal to Caesar or hated him.
 - inclusive-or
 - $\forall x: \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar})$
 - exclusive-or
 - $\forall x: \text{Roman}(x) \rightarrow (\text{loyalto}(x, \text{Caesar}) \wedge \neg \text{hate}(x, \text{Caesar})) \vee$
 - $(\neg \text{loyalto}(x, \text{Caesar}) \wedge \text{hate}(x, \text{Caesar}))$

6. Everyone is loyal to someone.
 - $\forall x: \exists y: \text{loyalto}(x, y)$
7. People only try to assassinate rulers they are not loyal to.
 - $\forall x: \forall y: \text{person}(x) \wedge \text{ruler}(y) \wedge \text{tryassassinate}(x, y)$
 - $\rightarrow \neg \text{loyalto}(x, y)$
8. Marcus tried to assassinate Caesar.
 - $\text{tryassassinate}(\text{Marcus}, \text{Caesar})$

Now suppose if we want to use these statements to answer the question: **Was Marcus loyal to Caesar?**

Also, Now let's try to produce a formal proof, reasoning backward from the desired goal: $\neg \text{loyalto}(\text{Marcus}, \text{Caesar})$

In order to prove the goal, we need to use the rules of inference to transform it into another goal (or possibly a set of goals) that can, in turn, transformed, and so on, until there are no unsatisfied goals remaining.

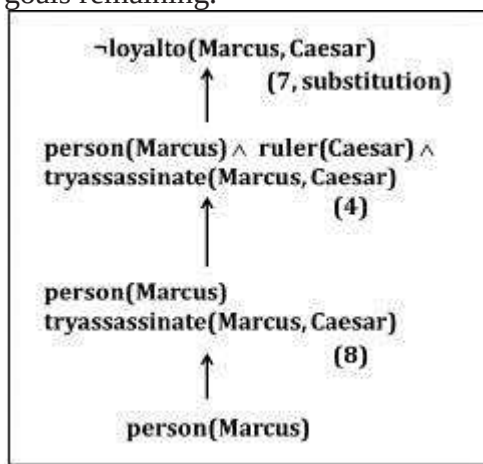


Figure: An attempt to prove $\neg \text{loyalto}(\text{Marcus}, \text{Caesar})$.

- The problem is that, although we know that Marcus was a man, we do not have any way to conclude from that that Marcus was a person. Also, We need to add the representation of another fact to our system, namely: $\forall \text{man}(x) \rightarrow \text{person}(x)$
- Now we can satisfy the last goal and produce a proof that Marcus was not loyal to Caesar.
- Moreover, From this simple example, we see that three important issues must be addressed in the process of converting English sentences into logical statements and then using those statements to deduce new ones:
 1. Many English sentences are ambiguous (for example, 5, 6, and 7 above). Choosing the correct interpretation may be difficult.
 2. Also, There is often a choice of how to represent the knowledge. Simple representations are desirable, but they may exclude certain kinds of reasoning.
 3. Similarly, Even in very simple situations, a set of sentences is unlikely to contain all the information necessary to reason about the topic at hand. In order to be able to use a set of statements effectively. Moreover, It is usually necessary to have access to another set of statements that represent facts that people consider too obvious to mention.

Representing Instance and ISA Relationships

- Specific attributes **instance** and **isa** play an important role particularly in a useful form of reasoning called property inheritance.
- The predicates instance and isa explicitly captured the relationships they used to express, namely class membership and class inclusion.
- 4.2 shows the first five sentences of the last section represented in logic in three different ways.
- The first part of the figure contains the representations we have already discussed. In these representations, class membership represented with unary predicates (such as Roman), each of which corresponds to a class.
- Asserting that $P(x)$ is true is equivalent to asserting that x is an instance (or element) of P .
- The second part of the figure contains representations that use the **instance** predicate explicitly.

1. Man(Marcus). 2. Pompeian(Marcus). 3. $\forall x: \text{Pompeian}(x) \rightarrow \text{Roman}(x).$ 4. ruler(Caesar). 5. $\forall x: \text{Roman}(x) \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar}).$
1. instance(Marcus, man). 2. instance(Marcus, Pompeian). 3. $\forall x: \text{instance}(x, \text{Pompeian}) \rightarrow \text{instance}(x, \text{Roman}).$ 4. instance(Caesar, ruler). 5. $\forall x: \text{instance}(x, \text{Roman}). \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar}).$
1. instance(Marcus, man). 2. instance(Marcus, Pompeian). 3. isa(Pompeian, Roman) 4. instance(Caesar, ruler). 5. $\forall x: \text{instance}(x, \text{Roman}). \rightarrow \text{loyalto}(x, \text{Caesar}) \vee \text{hate}(x, \text{Caesar}).$ 6. $\forall x: \forall y: \forall z: \text{instance}(x, y) \wedge \text{isa}(y, z) \rightarrow \text{instance}(x, z).$

Figure: Three ways of representing class membership: ISA Relationships

- The predicate **instance** is a binary one, whose first argument is an object and whose second argument is a class to which the object belongs.
- But these representations do not use an explicit **isa** predicate.
- Instead, subclass relationships, such as that between Pompeians and Romans, described as shown in sentence 3.
- The implication rule states that if an object is an instance of the subclass Pompeian then it is an instance of the superclass Roman.
- Note that this rule is equivalent to the standard set-theoretic definition of the subclass-superclass relationship.
- The third part contains representations that use both the **instance** and **isa** predicates explicitly.
- The use of the **isa** predicate simplifies the representation of sentence 3, but it requires that one additional axiom (shown here as number 6) be provided.

Computable Functions and Predicates

- To express simple facts, such as the following greater-than and less-than relationships:
 $gt(1,0)$ $lt(0,1)$ $gt(2,1)$ $lt(1,2)$ $gt(3,2)$ $lt(2,3)$
- It is often also useful to have computable functions as well as computable predicates.
Thus we might want to be able to evaluate the truth of $gt(2 + 3,1)$
- To do so requires that we first compute the value of the plus function given the arguments 2 and 3, and then send the arguments 5 and 1 to gt .

Consider the following set of facts, again involving Marcus:

- 1) Marcus was a man.
 $man(Marcus)$
- 2) Marcus was a Pompeian.
 $Pompeian(Marcus)$
- 3) Marcus was born in 40 A.D.
 $born(Marcus, 40)$
- 4) All men are mortal.
 $x: man(x) \rightarrow mortal(x)$
- 5) All Pompeians died when the volcano erupted in 79 A.D.
 $erupted(volcano, 79) \wedge \forall x : [Pompeian(x) \rightarrow died(x, 79)]$
- 6) No mortal lives longer than 150 years.
 $x: t1: At2: mortal(x) \wedge born(x, t1) \wedge gt(t2 - t1, 150) \rightarrow died(x, t2)$
- 7) It is now 1991.
 $now = 1991$

So, Above example shows how these ideas of computable functions and predicates can be useful. It also makes use of the notion of equality and allows equal objects to be substituted for each other whenever it appears helpful to do so during a proof.

- So, Now suppose we want to answer the question “Is Marcus alive?”
- The statements suggested here, there may be two ways of deducing an answer.
- Either we can show that Marcus is dead because he was killed by the volcano or we can show that he must be dead because he would otherwise be more than 150 years old, which we know is not possible.
- Also, As soon as we attempt to follow either of those paths rigorously, however, we discover, just as we did in the last example, that we need some additional knowledge. For example, our statements talk about dying, but they say nothing that relates to being alive, which is what the question is asking.

So we add the following facts:

- 8) Alive means not dead.
 $x: t: [alive(x, t) \rightarrow \neg dead(x, t)] [\neg dead(x, t) \rightarrow alive(x, t)]$
- 9) If someone dies, then he is dead at all later times.
 $x: t1: At2: died(x, t1) \wedge gt(t2, t1) \rightarrow dead(x, t2)$

So, Now let's attempt to answer the question “Is Marcus alive?” by proving: $\neg alive(Marcus, now)$

Resolution

Propositional Resolution

1. Convert all the propositions of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in step 1.
3. Repeat until either a contradiction is found or no progress can be made:
 1. Select two clauses. Call these the parent clauses.
 2. Resolve them together. The resulting clause, called the resolvent, will be the disjunction of all of the literals of both of the parent clauses with the following exception: If there are any pairs of literals L and $\neg L$ such that one of the parent clauses contains L and the other contains $\neg L$, then select one such pair and eliminate both L and $\neg L$ from the resolvent.
 3. If the resolvent is the empty clause, then a contradiction has been found. If it is not, then add it to the set of classes available to the procedure.

The Unification Algorithm

- In propositional logic, it is easy to determine that two literals cannot both be true at the same time.
- Simply look for L and $\neg L$ in predicate logic, this matching process is more complicated since the arguments of the predicates must be considered.
- For example, $\text{man}(\text{John})$ and $\neg \text{man}(\text{John})$ is a contradiction, while the $\text{man}(\text{John})$ and $\neg \text{man}(\text{Spot})$ is not.
- Thus, in order to determine contradictions, we need a matching procedure that compares two literals and discovers whether there exists a set of substitutions that makes them identical.
- There is a straightforward recursive procedure, called the unification algorithm, that does it.

Algorithm: Unify(L_1, L_2)

1. If L_1 or L_2 are both variables or constants, then:
 1. If L_1 and L_2 are identical, then return NIL.
 2. Else if L_1 is a variable, then if L_1 occurs in L_2 then return {FAIL}, else return (L_2/L_1).
 3. Also, Else if L_2 is a variable, then if L_2 occurs in L_1 then return {FAIL}, else return (L_1/L_2).
 - d. Else return {FAIL}.
2. If the initial predicate symbols in L_1 and L_2 are not identical, then return {FAIL}.
3. If L_1 and L_2 have a different number of arguments, then return {FAIL}.
4. Set SUBST to NIL. (At the end of this procedure, SUBST will contain all the substitutions used to unify L_1 and L_2 .)
5. For $I \leftarrow 1$ to the number of arguments in L_1 :
 1. Call Unify with the i^{th} argument of L_1 and the i^{th} argument of L_2 , putting the result in S .
 2. If S contains FAIL then return {FAIL}.
 3. If S is not equal to NIL then:
 2. Apply S to the remainder of both L_1 and L_2 .
 3. SUBST: = APPEND(S , SUBST).
6. Return SUBST.

Resolution in Predicate Logic

We can now state the resolution algorithm for predicate logic as follows, assuming a set of given statements F and a statement to be proved P :

Algorithm: Resolution

1. Convert all the statements of F to clause form.
2. Negate P and convert the result to clause form. Add it to the set of clauses obtained in 1.
3. Repeat until a contradiction found, no progress can make, or a predetermined amount of effort has expanded.
 1. Select two clauses. Call these the parent clauses.
 2. Resolve them together. The resolvent will be the disjunction of all the literals of both parent clauses with appropriate substitutions performed and with the following exception: If there is one pair of literals T_1 and $\neg T_2$ such that one of the parent clauses contains T_2 and the other contains T_1 and if T_1 and T_2 are unifiable, then neither T_1 nor T_2 should appear in the resolvent. We call T_1 and T_2 Complementary literals. Use the substitution produced by the unification to create the resolvent. If there is more than one pair of complementary literals, only one pair should omit from the resolvent.
 3. If the resolvent is an empty clause, then a contradiction has found. Moreover, If it is not, then add it to the set of clauses available to the procedure.

Resolution Procedure

- Resolution is a procedure, which gains its efficiency from the fact that it operates on statements that have been converted to a very convenient standard form.
- Resolution produces proofs by refutation.
- In other words, ***to prove a statement (i.e., to show that it is valid), resolution attempts to show that the negation of the statement produces a contradiction with the known statements (i.e., that it is unsatisfiable).***
- The resolution procedure is a simple iterative process: at each step, two clauses, called the parent clauses, are compared (resolved), resulting in a new clause that has inferred from them. The new clause represents ways that the two parent clauses interact with each other. Suppose that there are two clauses in the system:

winter $\vee$ summer

$\neg$ winter $\vee$ cold

- Now we observe that precisely one of winter and $\neg$ winter will be true at any point.
- If winter is true, then cold must be true to guarantee the truth of the second clause. If $\neg$ winter is true, then summer must be true to guarantee the truth of the first clause.
- Thus we see that from these two clauses we can deduce **summer $\vee$ cold**
- This is the deduction that the resolution procedure will make.
- Resolution operates by taking two clauses that each contains the same literal, in this example, **winter**.
- Moreover, The literal must occur in the positive form in one clause and in negative form in the other. The resolvent obtained by combining all of the literals of the two parent clauses except the ones that cancel.
- If the clause that produced is the empty clause, then a contradiction has found.

For example, the two clauses

winter

$\neg$ winter
will produce the empty clause.

Natural Deduction Using Rules

Testing whether a proposition is a tautology by testing every possible truth assignment is expensive—there are exponentially many. We need a **deductive system**, which will allow us to construct proofs of tautologies in a step-by-step fashion.

The system we will use is known as **natural deduction**. The system consists of a set of **rules of inference** for deriving consequences from premises. One builds a proof tree whose root is the proposition to be proved and whose leaves are the initial assumptions or axioms (for proof trees, we usually draw the root at the bottom and the leaves at the top).

For example, one rule of our system is known as **modus ponens**. Intuitively, this says that if we know P is true, and we know that P implies Q , then we can conclude Q .

$$\frac{P \quad P \Rightarrow Q}{Q} \text{ (modus ponens)}$$

The propositions above the line are called **premises**; the proposition below the line is the **conclusion**. Both the premises and the conclusion may contain metavariables (in this case, P and Q) representing arbitrary propositions. When an inference rule is used as part of a proof, the metavariables are replaced in a consistent way with the appropriate kind of object (in this case, propositions).

Most rules come in one of two flavors: **introduction** or **elimination** rules. Introduction rules introduce the use of a logical operator, and elimination rules eliminate it. Modus ponens is an elimination rule for $\Rightarrow$. On the right-hand side of a rule, we often write the name of the rule. This is helpful when reading proofs. In this case, we have written (modus ponens). We could also have written ($\Rightarrow$ -elim) to indicate that this is the elimination rule for $\Rightarrow$.

Rules for Conjunction

Conjunction ($\wedge$) has an introduction rule and two elimination rules:

$$\frac{P \quad Q}{P \wedge Q} \text{ (\wedge-intro)} \qquad \frac{P \wedge Q}{P} \text{ (\wedge-elim-left)} \qquad \frac{P \wedge Q}{Q} \text{ (\wedge-elim-right)}$$

Rule for T

The simplest introduction rule is the one for T . It is called "unit". Because it has no premises, this rule is an **axiom**: something that can start a proof.

$$\frac{}{T} \text{ (unit)}$$

Rules for Implication

In natural deduction, to prove an implication of the form $P \Rightarrow Q$, we assume P , then reason under that assumption to try to derive Q . If we are successful, then we can conclude that $P \Rightarrow Q$.

In a proof, we are always allowed to introduce a new assumption P , then reason under that assumption. We must give the assumption a name; we have used the name x in the example below. Each distinct assumption must have a different name.

$$\frac{}{[x : P]} \text{ (assum)}$$

Because it has no premises, this rule can also start a proof. It can be used as if the proposition P were proved. The name of the assumption is also indicated here.

However, you do not get to make assumptions for free! To get a complete proof, all assumptions must be eventually *discharged*. This is done in the implication introduction rule. This rule introduces an implication $P \Rightarrow Q$ by discharging a prior assumption $[x : P]$. Intuitively, if Q can be proved under the assumption P , then the implication $P \Rightarrow Q$ holds without any assumptions. We write x in the rule name to show which assumption is discharged. This rule and modus ponens are the introduction and elimination rules for implications.

$$\frac{\begin{array}{c} [x : P] \\ \vdots \\ Q \end{array}}{P \Rightarrow Q} \quad (\Rightarrow\text{-intro}/x) \qquad \frac{P \quad P \Rightarrow Q}{Q} \quad (\Rightarrow\text{-elim, modus ponens})$$

A proof is valid only if every assumption is eventually discharged. This must happen in the proof tree below the assumption. The same assumption can be used more than once.

Rules for Disjunction

$$\frac{P}{P \vee Q} \quad (\vee\text{-intro-left}) \qquad \frac{Q}{P \vee Q} \quad (\vee\text{-intro-right}) \qquad \frac{P \vee Q \quad P \Rightarrow R \quad Q \Rightarrow R}{R} \quad (\vee\text{-elim})$$

Rules for Negation

A negation $\neg P$ can be considered an abbreviation for $P \Rightarrow \perp$:

$$\frac{P \Rightarrow \perp}{\neg P} \quad (\neg\text{-intro}) \qquad \frac{\neg P}{P \Rightarrow \perp} \quad (\neg\text{-elim})$$

Rules for Falsity

$$\frac{\begin{array}{c} [x : \neg P] \\ \vdots \\ \perp \end{array}}{P} \quad (\text{reductio ad absurdum, RAA}/x) \qquad \frac{\perp}{P} \quad (\text{ex falso quodlibet, EFQ})$$

Reductio ad absurdum (RAA) is an interesting rule. It embodies proofs by contradiction. It says that if by assuming that P is false we can derive a contradiction, then P must be true. The assumption x is discharged in the application of this rule. This rule is present in classical logic but not in **intuitionistic** (constructive) logic. In intuitionistic logic, a proposition is not considered true simply because its negation is false.

Excluded Middle

Another classical tautology that is not intuitionistically valid is the **the law of the excluded middle**, $P \vee \neg P$. We will take it as an axiom in our system. The Latin name for this rule is *tertium non datur*, but we will call it *magic*.

$$\frac{}{P \vee \neg P} \quad (\text{magic})$$

Proofs

A proof of proposition P in natural deduction starts from axioms and assumptions and derives P with all assumptions discharged. Every step in the proof is an instance of an inference rule with metavariables substituted consistently with expressions of the appropriate syntactic class.

Example

For example, here is a proof of the proposition $(A \Rightarrow B \Rightarrow C) \Rightarrow (A \wedge B \Rightarrow C)$.

$$\begin{array}{c}
 \frac{\overline{[y : A \wedge B]} \text{ (A)}}{A} \text{ (\wedge E)} \quad \frac{\overline{[x : A \Rightarrow B \Rightarrow C]} \text{ (A)}}{B \Rightarrow C} \text{ (\Rightarrow E)} \quad \frac{\overline{[y : A \wedge B]} \text{ (A)}}{B} \text{ (\wedge E)} \\
 \frac{B \Rightarrow C \quad C}{A \wedge B \Rightarrow C} \text{ (\Rightarrow I, y)} \\
 \frac{A \wedge B \Rightarrow C}{(A \Rightarrow B \Rightarrow C) \Rightarrow (A \wedge B \Rightarrow C)} \text{ (\Rightarrow I, x)}
 \end{array}$$

The final step in the proof is to derive $(A \Rightarrow B \Rightarrow C) \Rightarrow (A \wedge B \Rightarrow C)$ from $(A \wedge B \Rightarrow C)$, which is done using the rule $(\Rightarrow\text{-intro})$, discharging the assumption $[x : A \Rightarrow B \Rightarrow C]$. To see how this rule generates the proof step, substitute for the metavariables P, Q, x in the rule as follows: $P = (A \Rightarrow B \Rightarrow C)$, $Q = (A \wedge B \Rightarrow C)$, and $x = x$. The immediately previous step uses the same rule, but with a different substitution: $P = A \wedge B$, $Q = C$, $x = y$.

The proof tree for this example has the following form, with the proved proposition at the root and axioms and assumptions at the leaves.



A proposition that has a complete proof in a deductive system is called a **theorem** of that system.

Soundness and Completeness

A measure of a deductive system's power is whether it is powerful enough to prove all true statements. A deductive system is said to be **complete** if all true statements are theorems (have proofs in the system). For propositional logic and natural deduction, this means that all tautologies must have natural deduction proofs. Conversely, a deductive system is called **sound** if all theorems are true. The proof rules we have given above are in fact sound and complete for propositional logic: every theorem is a tautology, and every tautology is a theorem. Finding a proof for a given tautology can be difficult. But once the proof is found, checking that it is indeed a proof is completely mechanical, requiring no intelligence or insight whatsoever. It is therefore a very strong argument that the thing proved is in fact true.

We can also make writing proofs less tedious by adding more rules that provide reasoning shortcuts. These rules are sound if there is a way to convert a proof using them into a proof using the original rules. Such added rules are called **admissible**.

Procedural versus Declarative Knowledge

We have discussed various search techniques in previous units. Now we would consider a set of rules that represent,

1. Knowledge about relationships in the world and
2. Knowledge about how to solve the problem using the content of the rules.

Procedural vs Declarative Knowledge

Procedural Knowledge

- A representation in which the control information that is necessary to use the knowledge is embedded in the knowledge itself for e.g. computer programs, directions, and recipes; these indicate specific use or implementation;
- The real difference between declarative and procedural views of knowledge lies in where control information reside.

For example, consider the following

Man (Marcus)

Man (Caesar)

Person (Cleopatra)

$\forall x: \text{Man}(x) \rightarrow \text{Person}(x)$

Now, try to answer the question. *?Person(y)*

The knowledge base justifies any of the following answers.

Y=Marcus

Y=Caesar

Y=Cleopatra

- We get more than one value that satisfies the predicate.
- If only one value needed, then the answer to the question will depend on the order in which the assertions examined during the search for a response.
- If the assertions declarative then they do not themselves say anything about how they will be examined. In case of procedural representation, they say how they will examine.

Declarative Knowledge

- A statement in which knowledge specified, but the use to which that knowledge is to be put is not given.
- For example, laws, people's name; these are the facts which can stand alone, not dependent on other knowledge;
- So to use declarative representation, we must have a program that explains what is to do with the knowledge and how.
- For example, a set of logical assertions can combine with a resolution theorem prover to give a complete program for solving problems but in some cases, the logical assertions can view as a program rather than data to a program.
- Hence the implication statements define the legitimate reasoning paths and automatic assertions provide the starting points of those paths.
- These paths define the execution paths which is similar to the 'if then else' in traditional programming.
- So logical assertions can view as a procedural representation of knowledge.

Logic Programming – Representing Knowledge Using Rules

- Logic programming is a programming paradigm in which logical assertions viewed as programs.
- These are several logic programming systems, PROLOG is one of them.
- ***A PROLOG program consists of several logical assertions where each is a horn clause i.e. a clause with at most one positive literal.***
- Ex : $P, P \vee Q, P \rightarrow Q$
- The facts are represented on Horn Clause for two reasons.
 1. Because of a uniform representation, a simple and efficient interpreter can write.
 2. The logic of Horn Clause decidable.

- Also, The first two differences are the fact that PROLOG programs are actually sets of Horn clause that have been transformed as follows:-
 1. If the Horn Clause contains no negative literal then leave it as it is.
 2. Also, Otherwise rewrite the Horn clauses as an implication, combining all of the negative literals into the antecedent of the implications and the single positive literal into the consequent.
- Moreover, This procedure causes a clause which originally consisted of a disjunction of literals (one of them was positive) to be transformed into a single implication whose antecedent is a conjunction universally quantified.
- But when we apply this transformation, any variables that occurred in negative literals and so now occur in the antecedent become existentially quantified, while the variables in the consequent are still universally quantified.

For example the PROLOG clause $P(x) :- Q(x, y)$ is equal to logical expression $\forall x: \exists y: Q(x, y) \rightarrow P(x)$.

- The difference between the logic and PROLOG representation is that the PROLOG interpretation has a fixed control strategy. And so, the assertions in the PROLOG program define a particular search path to answer any question.
- But, the logical assertions define only the set of answers but not about how to choose among those answers if there is more than one.

Consider the following example:

1. Logical representation

$\forall x : pet(x) \sqcap small(x) \rightarrow apartmentpet(x)$
 $\forall x : cat(x) \sqcap dog(x) \rightarrow pet(x)$
 $\forall x : poodle(x) \rightarrow dog(x) \sqcap small(x)$
 $poodle(fluffy)$

2. Prolog representation

$apartmentpet(x) : pet(x), small(x)$
 $pet(x) : cat(x)$
 $pet(x) : dog(x)$
 $dog(x) : poodle(x)$
 $small(x) : poodle(x)$
 $poodle(fluffy)$

Forward versus Backward Reasoning

Forward versus Backward Reasoning

A search procedure must find a path between initial and goal states.

There are two directions in which a search process could proceed.

The two types of search are:

1. Forward search which starts from the start state
2. Backward search that starts from the goal state

The production system views the forward and backward as symmetric processes.

Consider a game of playing 8 puzzles. The rules defined are

Square 1 empty and square 2 contains tile n. $\rightarrow$

- Also, Square 2 empty and square 1 contains the tile n .

Square 1 empty Square 4 contains tile n . $\rightarrow$

- Also, Square 4 empty and Square 1 contains tile n .

We can solve the problem in 2 ways:

1. Reason forward from the initial state

- Step 1. Begin building a tree of move sequences by starting with the initial configuration at the root of the tree.
- Step 2. Generate the next level of the tree by finding all rules **whose left-hand side matches** against the root node. The right-hand side is used to create new configurations.
- Step 3. Generate the next level by considering the nodes in the previous level and applying it to all rules whose left-hand side match.

2. Reasoning backward from the goal states:

- Step 1. Begin building a tree of move sequences by starting with the goal node configuration at the root of the tree.
- Step 2. Generate the next level of the tree by finding all rules **whose right-hand side matches** against the root node. The left-hand side used to create new configurations.
- Step 3. Generate the next level by considering the nodes in the previous level and applying it to all rules whose right-hand side match.
- So, The same rules can use in both cases.
- Also, In forwarding reasoning, the left-hand sides of the rules matched against the current state and right sides used to generate the new state.
- Moreover, In backward reasoning, the right-hand sides of the rules matched against the current state and left sides are used to generate the new state.

There are four factors influencing the type of reasoning. They are,

1. Are there more possible start or goal state? We move from smaller set of sets to the length.
2. In what direction is the branching factor greater? We proceed in the direction with the lower branching factor.
3. Will the program be asked to justify its reasoning process to a user? If, so then it is selected since it is very close to the way in which the user thinks.
4. What kind of event is going to trigger a problem-solving episode? If it is the arrival of a new factor, the forward reasoning makes sense. If it is a query to which a response is desired, backward reasoning is more natural.

Example 1 of Forward versus Backward Reasoning

- It is easier to drive from an unfamiliar place from home, rather than from home to an unfamiliar place. Also, If you consider a home as starting place an unfamiliar place as a goal then we have to backtrack from unfamiliar place to home.

Example 2 of Forward versus Backward Reasoning

- Consider a problem of symbolic integration. Moreover, The problem space is a set of formulas, which contains integral expressions. Here START is equal to the given formula with some integrals. GOAL is equivalent to the expression of the formula without any integral. Here we start from the formula with some integrals and proceed to an integral free expression rather than starting from an integral free expression.

Example 3 of Forward versus Backward Reasoning

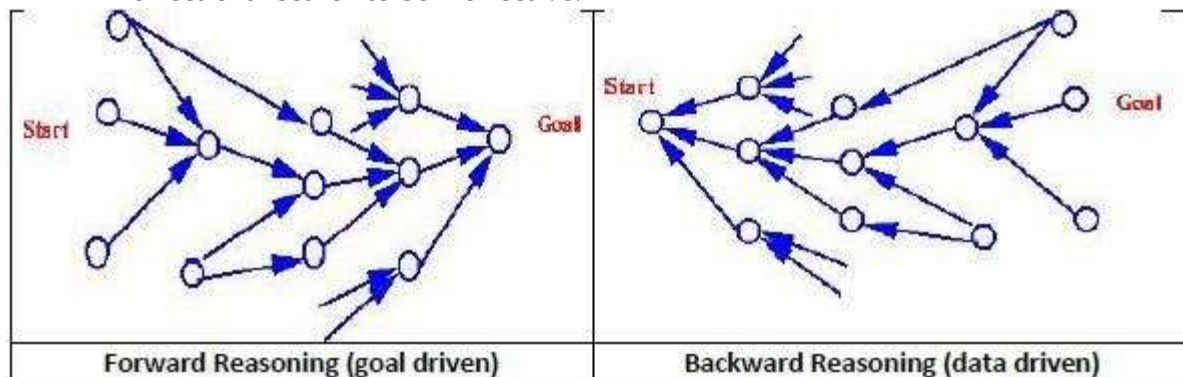
- The third factor is nothing but deciding whether the reasoning process can justify its reasoning. If it justifies then it can apply. For example, doctors are usually unwilling to accept any advice from diagnostics process because it cannot explain its reasoning.

Example 4 of Forward versus Backward Reasoning

- Prolog is an example of backward chaining rule system. In Prolog rules restricted to Horn clauses. This allows for rapid indexing because all the rules for deducing a given fact share the same rule head. Rules matched with unification procedure. Unification tries to find a set of bindings for variables to equate a sub-goal with the head of some rule. Rules in the Prolog program matched in the order in which they appear.

Combining Forward and Backward Reasoning

- Instead of searching either forward or backward, you can search both simultaneously.
- Also, That is, start forward from a starting state and backward from a goal state simultaneously until the paths meet.
- This strategy called Bi-directional search. The following figure shows the reason for a Bidirectional search to be ineffective.



Forward versus Backward Reasoning

- Also, The two searches may pass each other resulting in more work.
- Based on the form of the rules one can decide whether the same rules can apply to both forward and backward reasoning.
- Moreover, If left-hand side and right of the rule contain pure assertions then the rule can reverse.
- And so the same rule can apply to both types of reasoning.
- If the right side of the rule contains an arbitrary procedure then the rule cannot reverse.
- So, In this case, while writing the rule the commitment to a direction of reasoning must make.

Symbolic Reasoning Under Uncertainty

Symbolic Reasoning

- The reasoning is the act of deriving a conclusion from certain properties using a given methodology.
- The reasoning is a process of thinking; reasoning is logically arguing; reasoning is drawing the inference.
- *When a system is required to do something, that it has not been explicitly told how to do, it must reason. It must figure out what it needs to know from what it already knows.*

- Many types of Reasoning have been identified and recognized, but many questions regarding their logical and computational properties still remain controversial.
- The popular methods of Reasoning include abduction, induction, model-based, explanation and confirmation. All of them are intimately related to problems of belief revision and theory development, knowledge absorption, discovery, and learning.

Logical Reasoning

- Logic is a language for reasoning. It is a collection of rules called Logic arguments, we use when doing logical reasoning.
- The logic reasoning is the process of drawing conclusions from premises using rules of inference.
- The study of logic divided into formal and informal logic. The formal logic is sometimes called symbolic logic.
- Symbolic logic is the study of symbolic abstractions (construct) that capture the formal features of logical inference by a formal system.
- The formal system consists of two components, a formal language plus a set of inference rules.
- The formal system has axioms. Axiom is a sentence that is always true within the system.
- Sentences derived using the system's axioms and rules of derivation called theorems.
- The Logical Reasoning is of our concern in AI.

Approaches to Reasoning

- There are three different approaches to reasoning under uncertainties.
 1. Symbolic reasoning
 2. Statistical reasoning
 3. Fuzzy logic reasoning

Symbolic Reasoning

- The basis for intelligent mathematical software is the integration of the “power of symbolic mathematical tools” with the suitable “proof technology”.
- Mathematical reasoning enjoys a property called monotonicity, that says, “If a conclusion follows from given premises A, B, C... then it also follows from any larger set of premises, as long as the original premises A, B, C.. included.”
- Moreover, Human reasoning is not monotonic.
- People arrive at conclusions only tentatively; based on partial or incomplete information, reserve the right to retract those conclusions while they learn new facts. Such reasoning non-monotonic, precisely because the set of accepted conclusions have become smaller when the set of premises expanded.

Formal Logic

Moreover, The Formal logic is the study of inference with purely formal content, i.e. where content made explicit.

Examples – Propositional logic and Predicate logic.

- Here the logical arguments are a set of rules for manipulating symbols. The rules are of two types,
 1. Syntax rules: say how to build meaningful expressions.
 2. Inference rules: say how to obtain true formulas from other true formulas.
- Moreover, Logic also needs semantics, which says how to assign meaning to expressions.

Uncertainty in Reasoning

- The world is an uncertain place; often the Knowledge is imperfect which causes uncertainty.
- So, Therefore reasoning must be able to operate under uncertainty.
- Also, AI systems must have the ability to reason under conditions of uncertainty.

Monotonic Reasoning

- A reasoning process that moves in one direction only.
- Moreover, The number of facts in the knowledge base is always increasing.
- The conclusions derived are valid deductions and they remain so.

A monotonic logic cannot handle

1. Reasoning by default: because consequences may derive only because of lack of evidence to the contrary.
2. Abductive reasoning: because consequences only deduced as most likely explanations.
3. Belief revision: because new knowledge may contradict old beliefs.

Introduction to Nonmonotonic Reasoning

Non-monotonic Reasoning

The definite clause logic is **monotonic** in the sense that anything that could be concluded before a clause is added can still be concluded after it is added; adding knowledge does not reduce the set of propositions that can be derived.

A logic is **non-monotonic** if some conclusions can be invalidated by adding more knowledge.

The logic of definite clauses with negation as failure is non-monotonic. Non-monotonic reasoning is useful for representing defaults. A **default** is a rule that can be used unless it overridden by an exception.

For example, to say that b is normally true if c is true, a knowledge base designer can write a rule of the form

$$b \leftarrow c \wedge \sim ab_a.$$

where ab_a is an atom that means abnormal with respect to some aspect a . Given c , the agent can infer b unless it is told ab_a . Adding ab_a to the knowledge base can prevent the conclusion of b .

Rules that imply ab_a can be used to prevent the default under the conditions of the body of the rule.

Example 5.27: Suppose the purchasing agent is investigating purchasing holidays. A resort may be adjacent to a beach or away from a beach. This is not symmetric; if the resort was adjacent to a beach, the knowledge provider would specify this. Thus, it is reasonable to have the clause $away_from_beach \leftarrow \sim on_beach$.

This clause enables an agent to infer that a resort is away from the beach if the agent is not told it is adjacent to a beach.

A **cooperative system** tries to not mislead. If we are told the resort is on the beach, we would expect that resort users would have access to the beach. If they have access to a beach, we would expect them to be able to swim at the beach. Thus, we would expect the following defaults:

$$beach_access \leftarrow on_beach \wedge \sim ab_{beach_access}.$$

$$swim_at_beach \leftarrow beach_access \wedge \sim ab_{swim_at_beach}.$$

A cooperative system would tell us if a resort on the beach has no beach access or if there is no swimming. We could also specify that, if there is an enclosed bay and a big city, then there is no swimming, by default:

$$ab_{swim_at_beach} \leftarrow enclosed_bay \wedge big_city \wedge \sim ab_{no_swimming_near_city}.$$

We could say that British Columbia is abnormal with respect to swimming near cities:

$ab_{no_swimming_near_city} \leftarrow in_BC \wedge \sim ab_{BC_beaches}$.

Given only the preceding rules, an agent infers *away_from_beach*. If it is then told *on_beach*, it can no longer infer *away_from_beach*, but it can now infer *beach_access* and *swim_at_beach*. If it is also told *enclosed_bay* and *big_city*, it can no longer infer *swim_at_beach*. However, if it is then told *in_BC*, it can then infer *swim_at_beach*.

By having defaults of what is normal, a user can interact with the system by telling it what is abnormal, which allows for economy in communication. The user does not have to state the obvious.

One way to think about non-monotonic reasoning is in terms of **arguments**. The rules can be used as components of arguments, in which the negated abnormality gives a way to undermine arguments. Note that, in the language presented, only positive arguments exist that can be undermined. In more general theories, there can be positive and negative arguments that attack each other.

Implementation Issues

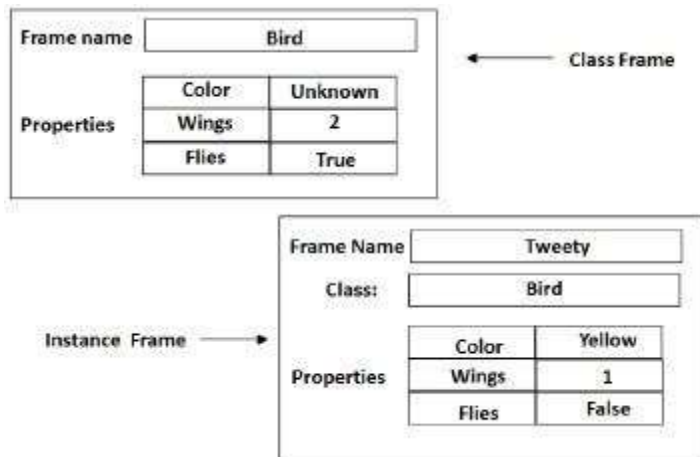
Weak Slot and Filler Structures

Evolution Frames

- As seen in the previous example, there are certain problems which are difficult to solve with Semantic Nets.
- Although there is no clear distinction between a semantic net and frame system, more structured the system is, more likely it is to be termed as a frame system.
- A frame is a collection of attributes (called slots) and associated values that describe some entities in the world. Sometimes a frame describes an entity in some absolute sense;
- Sometimes it represents the entity from a particular point of view only.
- A single frame taken alone is rarely useful; we build frame systems out of collections of frames that connected to each other by virtue of the fact that the value of an attribute of one frame may be another frame.

Frames as Sets and Instances

- The set theory is a good basis for understanding frame systems.
- Each frame represents either a class (a set) or an instance (an element of class)
- Both **isa** and **instance** relations have inverse attributes, which we call subclasses & all instances.
- As a class represents a set, there are 2 kinds of attributes that can be associated with it.
 1. Its own attributes &
 2. Attributes that are to be inherited by each element of the set.



Frames as Sets and Instances

- Sometimes, the difference between a set and an individual instance may not be clear.
- Example: Team India is an instance of the class of Cricket Teams and can also think of as the set of players.
- Now the problem is if we present Team India as a subclass of Cricket teams, then Indian players automatically become part of all the teams, which is not true.
- So, we can make Team India a subclass of class called Cricket Players.
- To do this we need to differentiate between regular classes and meta-classes.
- Regular Classes are those whose elements are individual entities whereas Meta-classes are those special classes whose elements are themselves, classes.
- The most basic meta-class is the class *CLASS*.
- It represents the set of all classes.
- All classes are instances of it, either directly or through one of its subclasses.
- The class *CLASS* introduces the attribute cardinality, which is to be inherited by all instances of *CLASS*. Cardinality stands for the number.

Other ways of Relating Classes to Each Other

- We have discussed that a class1 can be a subset of class2.
- If Class2 is a meta-class then Class1 can be an instance of Class2.
- Another way is the **mutually-disjoint-with** relationship, which relates a class to one or more other classes that guaranteed to have no elements in common with it.
- Another one is, **is-covered-by** which relates a class to a set of subclasses, the union of which is equal to it.
- If a class is-covered-by a set S of mutually disjoint classes, then S called a partition of the class.

Slots as Full-Fledged Objects (Frames)

Till now we have used attributes as slots, but now we will represent attributes explicitly and describe their properties.

Some of the properties we would like to be able to represent and use in reasoning include,

- The class to which the attribute can attach.
- Constraints on either the type or the value of the attribute.
- A default value for the attribute. Rules for inheriting values for the attribute.
- To be able to represent these attributes of attributes, we need to describe attributes (slots) as frames.

- These frames will organize into an *isa* hierarchy, just as any other frames, and that hierarchy can then be used to support inheritance of values for attributes of slots.
- Now let us formalize what is a slot. A slot here is a relation.
- It maps from elements of its domain (the classes for which it makes sense) to elements of its range (its possible values).
- A relation is a set of ordered pairs.
- Thus it makes sense to say that relation R1 is a subset of another relation R2.
- In that case, R1 is a specialization of R2. Since a slot is a set, the set of all slots, which we will call SLOT, is a meta-class.
- Its instances are slots, which may have sub-slots.

Frame Example

In this example, the frames Person, Adult-Male, ML-Baseball-Player (corresponding to major league baseball players), Pitcher, and ML-Baseball-Team (for major league baseball team) are all classes.

```

Person
  isa : Mammal
  cardinality : 6,000,000,000
  * handed : Right
Adult-Male
  isa : Person
  cardinality : 2,000,000,000
  * height : 5-10
ML-Baseball-Player
  isa : Adult-Male
  cardinality : 624
  * height : 6-1
  * bats : equal to handed
  * batting-average : .252
  * team :
  * uniform-color :
Fielder
  isa : ML-Baseball-Player
  cardinality : 376
  * batting-average : .262
Pee-Wee-Reese
  instance : Fielder
  height : 5-10
  bats : Right
  batting-average : .309
  team : Brooklyn-Dodgers
  uniform-color : Blue
ML-Baseball-Team
  isa : Team
  cardinality : 26
  * team-size : 24
  * manager :
Brooklyn-Dodgers
  instance : ML-Baseball-Team
  team-size : 24
  manager : Leo-Durocher
  players : {Pee-Wee-Reese,...}

```

- The frames Pee-Wee-Reese and Brooklyn-Dodgers are instances.
- The *isa* relation that we have been using without a precise definition is, in fact, the subset relation. The set of adult males is a subset of the set of people.
- The set of major league baseball players subset of the set of adult males, and so forth.
- Our instance relation corresponds to the relation element-of. Pee Wee Reese is an element of the set of fielders.
- Thus he is also an element of all of the supersets of fielders, including major league baseball players and people. The transitivity of *isa* follows directly from the transitivity of the subset relation.

- Both the isa and instance relations have inverse attributes, which we call subclasses and all instances.
- Because a class represents a set, there are two kinds of attributes that can associate with it.
- Some attributes are about the set itself, and some attributes are to inherited by each element of the set.
- We indicate the difference between these two by prefixing the latter with an asterisk (*).
- For example, consider the class ML-Baseball-Player, we have shown only two properties of it as a set: It is a subset of the set of adult males. And it has cardinality 624.
- We have listed five properties that all major league baseball players have (height, bats, batting average, team, and uniform-color), and we have specified default values for the first three of them.
- By providing both kinds of slots, we allow both classes to define a set of objects and to describe a prototypical object of the set.
- Frames are useful for representing objects that are typical of stereotypical situations.
- The situation like the structure of complex physical objects, visual scenes, etc.
- A commonsense knowledge can represent using default values if no other value exists. Commonsense is generally used in the absence of specific knowledge.

Semantic Nets

- Inheritance property can represent using **isa** and **instance**
- Monotonic Inheritance can perform substantially more efficiently with such structures than with pure logic, and non-monotonic inheritance is also easily supported.
- The reason that makes Inheritance easy is that the knowledge in slot and filler systems is structured as a set of entities and their attributes.

These structures turn out to be useful as,

- It indexes assertions by the entities they describe. As a result, retrieving the value for an attribute of an entity is fast.
- Moreover, It makes easy to describe properties of relations. To do this in a purely logical system requires higher-order mechanisms.
- It is a form of object-oriented programming and has the advantages that such systems normally include modularity and ease of viewing by people.

Here we would describe two views of this kind of structure – Semantic Nets & Frames.

Semantic Nets

- There are different approaches to knowledge representation include semantic net, frames, and script.
- The semantic net describes both objects and events.
- In a semantic net, information represented as a set of nodes connected to each other by a set of labeled arcs, which represents relationships among the nodes.
- It is a directed graph consisting of vertices which represent concepts and edges which represent semantic relations between the concepts.
- It is also known as associative net due to the association of one node with other.
- The main idea is that the meaning of the concept comes from the ways in which it connected to other concepts.
- We can use inheritance to derive additional relations.

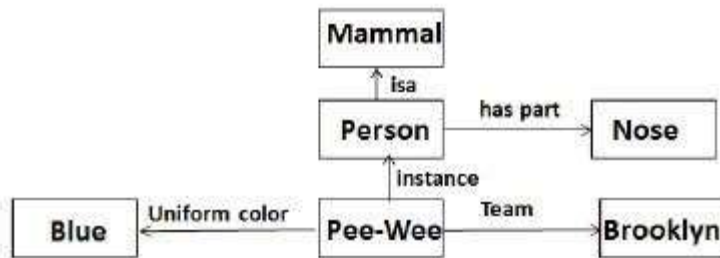


Figure: A Semantic Network

Intersection Search Semantic Nets

- We try to find relationships among objects by spreading activation out from each of two nodes. And seeing where the activation meets.
- Using this we can answer the questions like, what is the relation between India and Blue.
- It takes advantage of the entity-based organization of knowledge that slot and filler representation provides.

Representing Non-binary Predicates Semantic Nets

- Simple binary predicates like *isa(Person, Mammal)* can represent easily by semantic nets but other non-binary predicates can also represent by using general-purpose predicates such as *isa* and *instance*.
- Three or even more place predicates can also convert to a binary form by creating one new object representing the entire predicate statement and then introducing binary predicates to describe a relationship to this new object.

Conceptual Dependency

Introduction to Strong Slot and Filler Structures

- The main problem with semantic networks and frames is that they lack formality; there is no specific guideline on how to use the representations.
- In frame when things change, we need to modify all frames that are relevant – this can be time-consuming.
- Strong slot and filler structures typically represent links between objects according to more rigid rules, specific notions of what types of object and relations between them are provided and represent knowledge about common situations.
- Moreover, We have types of strong slot and filler structures:
 1. Conceptual Dependency (CD)
 2. Scripts
 3. Cyc

Conceptual Dependency (CD)

Conceptual Dependency originally developed to represent knowledge acquired from natural language input.

The goals of this theory are:

- To help in the drawing of the inference from sentences.
- To be independent of the words used in the original input.
- That is to say: For any 2 (or more) sentences that are identical in meaning there should be only one representation of that meaning.

Moreover, It has used by many programs that portend to understand English (MARGIE, SAM, PAM).

Conceptual Dependency (CD) provides:

- A structure into which nodes representing information can be placed.
- Also, A specific set of primitives.
- A given level of granularity.

Sentences are represented as a series of diagrams depicting actions using both abstract and real physical situations.

- The agent and the objects represented.
- Moreover, The actions are built up from a set of primitive acts which can modify by tense.

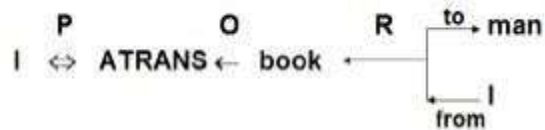
CD is based on events and actions. Every event (if applicable) has:

- an ACTOR or an ACTION performed by the Actor
- Also, an OBJECT that the action performs on
- A DIRECTION in which that action is oriented

These are represented as slots and fillers. In English sentences, many of these attributes left out.

A Simple Conceptual Dependency Representation

For the sentence, “I have a book to the man” CD representation is as follows:



Where the symbols have the following meaning.

- Arrows indicate directions of dependency.
- Moreover, The double arrow indicates the two-way link between actor and action.
- O — for the object case relation
- R – for the recipient case relation
- P – for past tense
- D – destination

Primitive Acts of Conceptual Dependency Theory

ATRANS

- Transfer of an abstract relationship (i.e. give)

PTRANS

- Transfer of the physical location of an object (e.g., go)

PROPEL

- Also, Application of physical force to an object (e.g. push)

MOVE

- Moreover, Movement of a body part by its owner (e.g. kick)

GRASP

- Grasping of an object by an action (e.g. throw)

INGEST

- Ingesting of an object by an animal (e.g. eat)

EXPEL

- Expulsion of something from the body of an animal (e.g. cry)

MTRANS

- Transfer of mental information (e.g. tell)

MBUILD

- Building new information out of old (e.g. decide)

SPEAK

- Producing of sounds (e.g. say)

ATTEND

- Focusing of a sense organ toward a stimulus (e.g. listen)

There are four conceptual categories. These are,

ACT

- Actions {one of the CD primitives}

PP

- Also, Objects {picture producers}

AA

- Modifiers of actions {action aiders}

PA

- Modifiers of PP's {picture aiders}

Advantages of Conceptual Dependency

- Using these primitives involves fewer inference rules.
- So, Many inference rules already represented in CD structure.
- Moreover, The holes in the initial structure help to focus on the points still to established.

Disadvantages of Conceptual Dependency

- Knowledge must decompose into fairly low-level primitives.
- Impossible or difficult to find the correct set of primitives.
- Also, A lot of inference may still require.
- Representations can be complex even for relatively simple actions.
- Consider: Dave bet Frank five pounds that Wales would win the Rugby World Cup.
- Moreover, Complex representations require a lot of storage.

Scripts

Scripts Strong Slot

- A script is a structure that prescribes a set of circumstances which could be expected to follow on from one another.
- It is similar to a thought sequence or a chain of situations which could be anticipated.
- It could be considered to consist of a number of slots or frames but with more specialized roles.

Scripts are beneficial because:

- Events tend to occur in known runs or patterns.
- Causal relationships between events exist.
- Entry conditions exist which allow an event to take place
- Prerequisites exist for events taking place. E.g. when a student progresses through a degree scheme or when a purchaser buys a house.

Script Components

Each script contains the following main components.

- Entry Conditions: Must be satisfied before events in the script can occur.
- Results: Conditions that will be true after events in script occur.
- Props: Slots representing objects involved in the events.
- Roles: Persons involved in the events.
- Track: the Specific variation on the more general pattern in the script. Different tracks may share many components of the same script but not all.

- Scenes: The sequence of events that occur. Events represented in conceptual dependency form.

Advantages and Disadvantages of Script

Advantages

- Capable of predicting implicit events
- Single coherent interpretation may be build up from a collection of observations.

Disadvantage

- More specific (inflexible) and less general than frames.
- Not suitable to represent all kinds of knowledge.

To deal with inflexibility, smaller modules called memory organization packets (MOP) can combine in a way that appropriates for the situation.

Script Example

Script : Play in theater	Various Scenes
Track: Play in Theater Props: • Tickets • Seat • Play Roles: • Person (who wants to see a play) – P • Ticket distributor – TD • Ticket checker – TC Entry Conditions: • P wants to see a play • P has a money Results: • P saw a play • P has less money • P is happy (optional if he liked the play)	Scene 1: Going to theater • P PTRANS P into theater • P ATTEND eyes to ticket counter
	Scene 2: Buying ticket • P PTRANS P to ticket counter • P MTRANS (need a ticket) to TD • TD ATRANS ticket to P
	Scene 3: Going inside hall of theater and sitting on a seat • P PTRANS P into Hall of theater • TC ATTEND eyes on ticket POSS_by P • TC MTRANS (showed seat) to P • P PTRANS P to seat • P MOVES P to sitting position
	Scene 4: Watching a play • P ATTEND eyes on play • P MBUILD (good moments) from play
	Scene 5: Exiting • P PTRANS P out of Hall and theater

- It must activate based on its significance.
- If the topic important, then the script should open.
- If a topic just mentioned, then a pointer to that script could hold.
- For example, given “John enjoyed the play in theater”, a script “Play in Theater” suggested above invoke.
- All implicit questions can answer correctly.

Here the significance of this script is high.

- Did John go to the theater?
- Also, Did he buy the ticket?
- Did he have money?

If we have a sentence like “John went to the theater to pick his daughter”, then invoking this script will lead to many wrong answers.

- Here significance of the script theater is less.

Getting significance from the story is not straightforward. However, some heuristics can apply to get the value.

CYC

What is CYC?

- An ambitious attempt to form a very large knowledge base aimed at capturing commonsense reasoning.
- Initial goals to capture knowledge from a hundred randomly selected articles in the Encyclopedia Britannica.
- Also, Both Implicit and Explicit knowledge encoded.
- Moreover, Emphasis on study of underlying information (assumed by the authors but not needed to tell to the readers).

Example: Suppose we read that Wellington learned of Napoleon's death • Then we (humans) can conclude Napoleon never new that Wellington had died.

How do we do this?

So, We require special implicit knowledge or commonsense such as:

- We only die once.
- You stay dead.
- Moreover, You cannot learn anything when dead.
- Time cannot go backward.

Why build large knowledge bases:

1. Brittleness
 - Specialised knowledge bases are brittle. Hard to encode new situations and non-graceful degradation in performance. Commonsense based knowledge bases should have a firmer foundation.
2. Form and Content
 - Moreover, Knowledge representation may not be suitable for AI. Commonsense strategies could point out where difficulties in content may affect the form.
3. Shared Knowledge
 - Also, Should allow greater communication among systems with common bases and assumptions.

How is CYC coded?

- By hand.
- Special CYCL language:
- LISP-like.
- Frame-based
- Multiple inheritances
- Slots are fully fledged objects.
- Generalized inheritance — any link not just *isa* and *instance*.

Module 2

Game Playing:

Game Playing

- Charles Babbage, the nineteenth-century computer architect thought about programming his analytical engine to play chess and later of building a machine to play tic-tac-toe.
- There are two reasons that games appeared to be a good domain.
 1. They provide a structured task in which it is very easy to measure success or failure.
 2. They are easily solvable by straightforward search from the starting state to a winning position.
- The first is true for all games but the second is not true for all, except simplest games.
- For example, consider chess.
- The average branching factor is around 35. In an average game, each player might make 50.
- So in order to examine the complete game tree, we would have to examine 35^{100}
- Thus it is clear that a simple search is not able to select even its first move during the lifetime of its opponent.
- It is clear that to improve the effectiveness of a search based problem-solving program two things can do.
 1. Improve the generate procedure so that only good moves generated.
 2. Improve the test procedure so that the best move will recognize and explored first.
- If we use legal-move generator then the test procedure will have to look at each of them because the test procedure must look at so many possibilities, it must be fast.
- Instead of the legal-move generator, we can use plausible-move generator in which only some small numbers of promising moves generated.
- As the number of lawyers available moves increases, it becomes increasingly important in applying heuristics to select only those moves that seem more promising.
- The performance of the overall system can improve by adding heuristic knowledge into both the generator and the tester.
- In game playing, a goal state is one in which we win but the game like chess. It is not possible. Even we have good plausible move generator.
- The depth of the resulting tree or graph and its branching factor is too great.
- It is possible to search tree only ten or twenty moves deep then in order to choose the best move. The resulting board positions must compare to discover which is most advantageous.
- This is done using static evaluation function, which uses whatever information it has to evaluate individual board position by estimating how likely they are to lead eventually to a win.
- Its function is similar to that of the heuristic function h' in the A* algorithm: in the absence of complete information, choose the most promising position.

MINIMAX Search Procedure

- The minimax search is a depth-first and depth limited procedure.
- The idea is to start at the current position and use the plausible-move generator to generate the set of possible successor positions.

- Now we can apply the static evolution function to those positions and simply choose the best one.
- After doing so, we can back that value up to the starting position to represent our evolution of it.
- Here we assume that static evolution function returns larger values to indicate good situations for us.
- So our goal is to maximize the value of the static evaluation function of the next board position.
- The opponents' goal is to minimize the value of the static evaluation function.
- **The alternation of maximizing and minimizing at alternate ply when evaluations are to be pushed back up corresponds to the opposing strategies of the two players is called MINIMAX.**
- It is the recursive procedure that depends on two procedures
 - MOVEGEN(position, player)— The plausible-move generator, which returns a list of nodes representing the moves that can make by Player in Position.
 - STATIC(position, player)— static evaluation function, which returns a number representing the goodness of Position from the standpoint of Player.
- With any recursive program, we need to decide when recursive procedure should stop.
- There are the variety of factors that may influence the decision they are,
 - Has one side won?
 - How many plies have we already explored? Or how much time is left?
 - How stable is the configuration?
- We use DEEP-ENOUGH which assumed to evaluate all of these factors and to return TRUE if the search should be stopped at the current level and FALSE otherwise.
- It takes two parameters, position, and depth, it will ignore its position parameter and simply return TRUE if its depth parameter exceeds a constant cut off value.
- One problem that arises in defining MINIMAX as a recursive procedure is that it needs to return not one but two results.
 - The backed-up value of the path it chooses.
 - The path itself. We return the entire path even though probably only the first element, representing the best move from the current position, actually needed.
- We assume that MINIMAX returns a structure containing both results and we have two functions, VALUE and PATH that extract the separate components.
- Initially, It takes three parameters, a board position, the current depth of the search, and the player to move,
 - MINIMAX(current,0,player-one) If player –one is to move
 - MINIMAX(current,0,player-two) If player –two is to move

Adding alpha-beta cutoffs

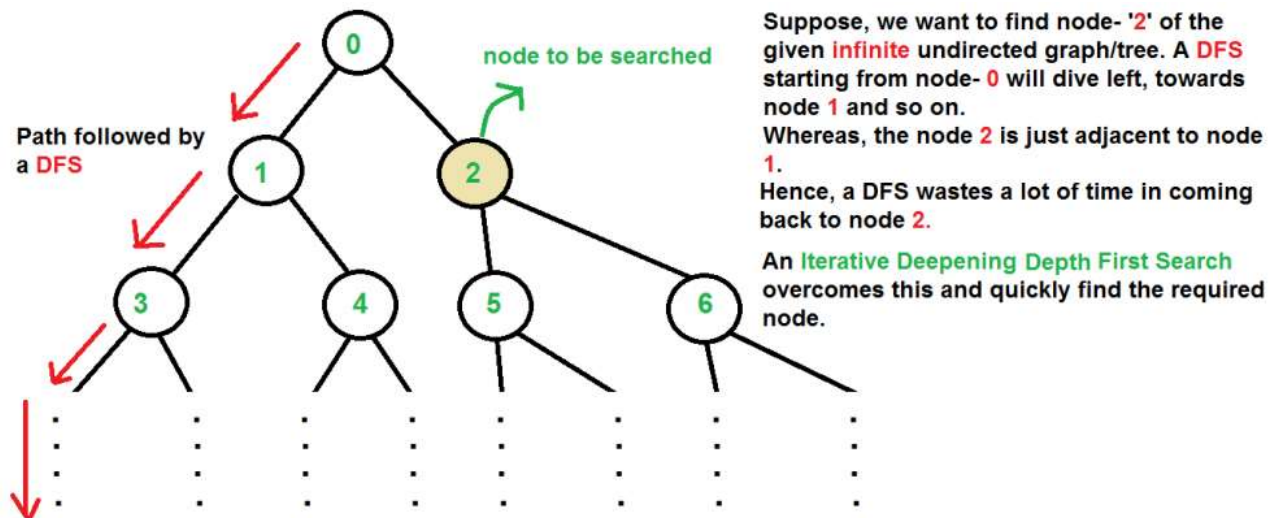
- Minimax procedure is a depth-first process. One path is explored as far as time allows, the static evolution function is applied to the game positions at the last step of the path.
- The efficiency of the depth-first search can improve by branch and bound technique in which partial solutions that clearly worse than known solutions can abandon early.
- It is necessary to modify the branch and bound strategy to include two bounds, one for each of the players.
- This modified strategy called alpha-beta pruning.

- It requires maintaining of two threshold values, one representing a lower bound on that a maximizing node may ultimately assign (we call this alpha).
- And another representing an upper bound on the value that a minimizing node may assign (this we call beta).
- Each level must receive both the values, one to use and one to pass down to the next level to use.
- The MINIMAX procedure as it stands does not need to treat maximizing and minimizing levels differently. Since it simply negates evaluation each time it changes levels.
- Instead of referring to alpha and beta, MINIMAX uses two values, USE-THRESH and PASSTHRESH.
- USE-THRESH used to compute cutoffs. PASS-THRESH passed to next level as its USETHRESH.
- USE-THRESH must also pass to the next level, but it will pass as PASS-THRESH so that it can be passed to the third level down as USE-THRESH again, and so forth.
- Just as values had to negate each time they passed across levels.
- Still, there is no difference between the code required at maximizing levels and that required at minimizing levels.
- PASS-THRESH should always the maximum of the value it inherits from above and the best move found at its level.
- If PASS-THRESH updated the new value should propagate both down to lower levels. And back up to higher ones so that it always reflects the best move found anywhere in the tree.
- The MINIMAX-A-B requires five arguments, position, depth, player, Use-thresh, and passThresh.
- MINIMAX-A-B(current,0,player-one,maximum value static can compute, minimum value static can compute).

Iterative Deepening Search(IDS) or Iterative Deepening Depth First Search(IDDFS)

There are two common ways to traverse a graph, [BFS](#) and [DFS](#). Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

1. **DFS** first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges).



2. **BFS** goes level by level, but requires more space. The space required by DFS is $O(d)$ where d is depth of tree, but space required by BFS is $O(n)$ where n is number of nodes in tree (Why? Note that the last level of tree can have around $n/2$ nodes and second last level $n/4$ nodes and in BFS we need to have every level one by one in queue).

IDDFS combines depth-first search's space-efficiency and breadth-first search's fast search (for nodes closer to root).

How does IDDFS work?

IDDFS calls DFS for different depths starting from an initial value. In every call, DFS is restricted from going beyond given depth. So basically we do DFS in a BFS fashion.

Algorithm:

```
// Returns true if target is reachable from
// src within max_depth
```

```
bool IDDFS(src, target, max_depth)
    for limit from 0 to max_depth
        if DLS(src, target, limit) == true
            return true
    return false
```

```
bool DLS(src, target, limit)
    if (src == target)
        return true;
```

```
// If reached the maximum depth,
// stop recursing.
if (limit <= 0)
    return false;
```

```
foreach adjacent i of src
```



```
if DLS(i, target, limit?1)
    return true
```

```
return false
```

An important thing to note is, we visit top level nodes multiple times. The last (or max depth) level is visited once, second last level is visited twice, and so on. It may seem expensive, but it turns out to be not so costly, since in a tree most of the nodes are in the bottom level. So it does not matter much if the upper levels are visited multiple times.

Planning

Blocks World Problem

In order to compare the variety of methods of planning, we should find it useful to look at all of them in a single domain that is complex enough that the need for each of the mechanisms is apparent yet simple enough that easy-to-follow examples can be found.

- There is a flat surface on which blocks can be placed.
- There are a number of square blocks, all the same size.
- They can be stacked one upon the other.
- There is robot arm that can manipulate the blocks.

Actions of the robot arm

1. UNSTACK(A, B): Pick up block A from its current position on block B.
2. STACK(A, B): Place block A on block B.
3. PICKUP(A): Pick up block A from the table and hold it.
4. PUTDOWN(A): Put block A down on the table.

Notice that the robot arm can hold only one block at a time.

Predicates

- In order to specify both the conditions under which an operation may be performed and the results of performing it, we need the following predicates:
 1. ON(A, B): Block A is on Block B.
 2. ONTABLES(A): Block A is on the table.
 3. CLEAR(A): There is nothing on the top of Block A.
 4. HOLDING(A): The arm is holding Block A.
 5. ARMEMPTY: The arm is holding nothing.

Robot problem-solving systems (STRIPS)

- List of new predicates that the operator causes to become true is ADD List
- Moreover, List of old predicates that the operator causes to become false is DELETE List
- PRECONDITIONS list contains those predicates that must be true for the operator to be applied.

STRIPS style operators for BLOCKs World

STACK(x, y)

P: CLEAR(y)^HOLDING(x)

D: CLEAR(y)^HOLDING(x)

A: ARMEMPTY^ON(x, y)

UNSTACK(x, y)

PICKUP(x)

P: CLEAR(x) ^ ONTABLE(x) ^ ARMEMPTY

D: ONTABLE(x) ^ ARMEMPTY

A: HOLDING(x)

PUTDOWN(x)

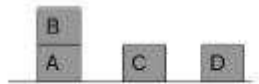
Goal Stack Planning

To start with goal stack is simply:

- $ON(C,A) \wedge ON(B,D) \wedge ONTABLE(A) \wedge ONTABLE(D)$

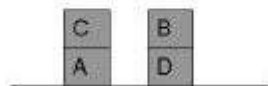
This problem is separate into four sub-problems, one for each component of the goal.

Two of the sub-problems $ONTABLE(A)$ and $ONTABLE(D)$ are already true in the initial state.



Start:

$ON(B,A) \wedge ONTABLE(A) \wedge ONTABLE(C) \wedge ONTABLE(D) \wedge ARMEMPTY$



Goal: $ON(C,A) \wedge ON(B,D) \wedge ONTABLE(A) \wedge ONTABLE(D)$

Alternative 1: Goal Stack:

- $ON(C,A)$
- $ON(B,D)$
- $ON(C,A) \wedge ON(B,D) \wedge OTAD$

Alternative 2: Goal stack:

- $ON(B,D)$
- $ON(C,A)$
- $ON(C,A) \wedge ON(B,D) \wedge OTAD$

Exploring Operators

- Pursuing alternative 1, we check for operators that could cause $ON(C, A)$
- Out of the 4 operators, there is only one STACK. So it yields:
 - $STACK(C,A)$
 - $ON(B,D)$
 - $ON(C,A) \wedge ON(B,D) \wedge OTAD$
- Preconditions for $STACK(C, A)$ should be satisfied, we must establish them as sub-goals:
 - $CLEAR(A)$
 - $HOLDING(C)$
 - $CLEAR(A) \wedge HOLDING(C)$
 - $STACK(C,A) \circ ON(B,D)$
 - $ON(C,A) \wedge ON(B,D) \wedge OTAD$
- Here we exploit the Heuristic that if $HOLDING$ is one of the several goals to be achieved at once, it should be tackled last.

Goal stack Planning

- Next, we see if $CLEAR(A)$ is true. It is not. The only operator that could make it true is $UNSTACK(B, A)$. Also, This produces the goal stack:
 - $ON(B, A)$
 - $CLEAR(B)$
 - $ON(B,A) \wedge CLEAR(B) \wedge ARMEMPTY$
 - $UNSTACK(B, A)$
 - $HOLDING(C)$

- $CLEAR(A) \wedge HOLDING(C)$
- $STACK(C,A)$
- $ON(B,D)$
- $ON(C,A) \wedge ON(B,D) \wedge OTAD$
- We see that we can pop predicates on the stack till we reach $HOLDING(C)$ for which we need to find a suitable operator.
- Moreover, The operators that might make $HOLDING(C)$ true: $PICKUP(C)$ and $UNSTACK(C, x)$. Without looking ahead, since we cannot tell which of these operators is appropriate. Also, we create two branches of the search tree corresponding to the following goal stacks:

ALT1:	ALT2:
ONTABLE(C)	$ON(C, x)$
CLEAR(C)	CLEAR(C)
ARMEMPTY	ARMEMPTY
ONTABLE(C)	$ON(C,x) \wedge CLEAR(C) \wedge ARMEMPTY$
$\wedge CLEAR(C) \wedge ARMEMPTY$	
PICKUP(C)	UNSTACK(C,x)
$CLEAR(A) \wedge HOLDING(C)$	$CLEAR(A) \wedge HOLDING(C)$
STACK(C,A)	STACK(C,A)
$ON(B,D)$	$ON(B,D)$
$ON(C,A) \wedge ON(B,D) \wedge OTAD$	$ON(C,A) \wedge ON(B,D) \wedge OTAD$

Complete plan

1. UNSTACK(C, A)
2. PUTDOWN(C)
3. PICKUP(A)
4. STACK(A, B)
5. UNSTACK(A, B)
6. PUTDOWN(A)
7. PICKUP(B)
8. STACK(B, C)
9. PICKUP(A)
10. STACK(A,B)

Planning Components

- Methods which focus on ways of decomposing the original problem into appropriate subparts and on ways of recording and handling interactions among the subparts as they are detected during the problem-solving process are often called as planning.
- Planning refers to the process of computing several steps of a problem-solving procedure before executing any of them.

Components of a planning system

Choose the best rule to apply next, based on the best available heuristic information.

- The most widely used technique for selecting appropriate rules to apply is first to isolate a set of differences between desired goal state and then to identify those rules that are relevant to reduce those differences.
- If there are several rules, a variety of other heuristic information can be exploited to choose among them.

Apply the chosen rule to compute the new problem state that arises from its application.

- In simple systems, applying rules is easy. Each rule simply specifies the problem state that would result from its application.
- In complex systems, we must be able to deal with rules that specify only a small part of the complete problem state.
- One way is to describe, for each action, each of the changes it makes to the state description.

Detect when a solution has found.

- A planning system has succeeded in finding a solution to a problem when it has found a sequence of operators that transform the initial problem state into the goal state.
- How will it know when this has done?
- In simple problem-solving systems, this question is easily answered by a straightforward match of the state descriptions.
- One of the representative systems for planning systems is predicate logic. Suppose that as a part of our goal, we have the predicate $P(x)$.
- To see whether $P(x)$ satisfied in some state, we ask whether we can prove $P(x)$ given the assertions that describe that state and the axioms that define the world model.

Detect dead ends so that they can abandon and the system's effort directed in more fruitful directions.

- As a planning system is searching for a sequence of operators to solve a particular problem, it must be able to detect when it is exploring a path that can never lead to a solution.
- The same reasoning mechanisms that can use to detect a solution can often use for detecting a dead end.
- If the search process is reasoning forward from the initial state. It can prune any path that leads to a state from which the goal state cannot reach.
- If search process reasoning backward from the goal state, it can also terminate a path either because it is sure that the initial state cannot reach or because little progress made.

Detect when an almost correct solution has found and employ special techniques to make it totally correct.

- The kinds of techniques discussed are often useful in solving nearly decomposable problems.
- One good way of solving such problems is to assume that they are completely decomposable, proceed to solve the sub-problems separately. And then check that when the sub-solutions combined. They do in fact give a solution to the original problem.

Goal Stack Planning

- Methods which focus on ways of decomposing the original problem into appropriate subparts and on ways of recording. And handling interactions among the subparts as they are detected during the problem-solving process are often called as planning.
- Planning refers to the process of computing several steps of a problem-solving procedure before executing any of them.
-

Goal Stack Planning Method

- In this method, the problem solver makes use of a single stack that contains both goals and operators. That have proposed to satisfy those goals.

- The problem solver also relies on a database that describes the current situation and a set of operators described as PRECONDITION, ADD and DELETE lists.
- The goal stack planning method attacks problems involving conjoined goals by solving the goals one at a time, in order.
- A plan generated by this method contains a sequence of operators for attaining the first goal, followed by a complete sequence for the second goal etc.
- At each succeeding step of the problem-solving process, the top goal on the stack will pursue.
- When a sequence of operators that satisfies it, found, that sequence applied to the state description, yielding new description.
- Next, the goal that then at the top of the stack explored. And an attempt made to satisfy it, starting from the situation that produced as a result of satisfying the first goal.
- This process continues until the goal stack is empty.
- Then as one last check, the original goal compared to the final state derived from the application of the chosen operators.
- If any components of the goal not satisfied in that state. Then those unsolved parts of the goal reinserted onto the stack and the process resumed.

Nonlinear Planning using Constraint Posting

- Difficult problems cause goal interactions.
- The operators used to solve one sub-problem may interfere with the solution to a previous sub-problem.
- Most problems require an intertwined plan in which multiple sub-problems worked on simultaneously.
- Such a plan is called nonlinear plan because it is not composed of a linear sequence of complete sub-plans.

Constraint Posting

- The idea of constraint posting is to build up a plan by incrementally hypothesizing operators, partial orderings between operators, and binding of variables within operators.
- At any given time in the problem-solving process, we may have a set of useful operators but perhaps no clear idea of how those operators should order with respect to each other.
- A solution is a partially ordered, partially instantiated set of operators to generate an actual plan. And we convert the partial order into any number of total orders.

Constraint Posting versus State Space search

State Space Search

- Moves in the space: Modify world state via operator
- Model of time: Depth of node in search space
- Plan stored in Series of state transitions

Constraint Posting Search

- Moves in the space: Add operators, Order Operators, Bind variables Or Otherwise constrain plan
- Model of Time: Partially ordered set of operators
- Plan stored in Single node

Algorithm: Nonlinear Planning (TWEAK)

1. Initialize S to be the set of propositions in the goal state.
2. Remove some unachieved proposition P from S.

3. Moreover, Achieve P by using step addition, promotion, DE clobbering, simple establishment or separation.
4. Review all the steps in the plan, including any new steps introduced by step addition, to see if any of their preconditions unachieved. Add to S the new set of unachieved preconditions.
5. Also, If S is empty, complete the plan by converting the partial order of steps into a total order, instantiate any variables as necessary.
6. Otherwise, go to step 2.

Hierarchical Planning

- In order to solve hard problems, a problem solver may have to generate long plans.
- It is important to be able to eliminate some of the details of the problem until a solution that addresses the main issues is found.
- Then an attempt can make to fill in the appropriate details.
- Early attempts to do this involved the use of macro operators, in which larger operators were built from smaller ones.
- In this approach, no details eliminated from actual descriptions of the operators.

ABSTRIPS

A better approach developed in ABSTRIPS systems which actually planned in a hierarchy of abstraction spaces, in each of which preconditions at a lower level of abstraction ignored.

ABSTRIPS approach is as follows:

- First solve the problem completely, considering only preconditions whose criticality value is the highest possible.
- These values reflect the expected difficulty of satisfying the precondition.
- To do this, do exactly what STRIPS did, but simply ignore the preconditions of lower than peak criticality.
- Once this done, use the constructed plan as the outline of a complete plan and consider preconditions at the next-lowest criticality level.
- Augment the plan with operators that satisfy those preconditions.
- Because this approach explores entire plans at one level of detail before it looks at the lower-level details of any one of them, it has called length-first approach.

The assignment of appropriate criticality value is crucial to the success of this hierarchical planning method.

Those preconditions that no operator can satisfy are clearly the most critical.

Example, solving a problem of moving the robot, for applying an operator, PUSH-THROUGH DOOR, the precondition that there exist a door big enough for the robot to get through is of high criticality since there is nothing we can do about it if it is not true.

Other Planning Techniques

Reactive Systems

- The idea of reactive systems is to avoid planning altogether, and instead, use the observable situation as a clue to which one can simply react.
- A reactive system must have access to a knowledge base of some sort that describes what actions should be taken under what circumstances.
- A reactive system is very different from the other kinds of planning systems we have discussed. Because it chooses actions one at a time.
- It does not anticipate and select an entire action sequence before it does the first thing.
- The example is a Thermostat. The job of the thermostat is to keep the temperature constant inside a room.
- Reactive systems are capable of surprisingly complex behaviors.
- The main advantage reactive systems have over traditional planners is that they operate robustly in domains that are difficult to model completely and accurately.
- Reactive systems dispense with modeling altogether and base their actions directly on their perception of the world.
- Another advantage of reactive systems is that they are extremely responsive since they avoid the combinatorial explosion involved in deliberative planning.
- This makes them attractive for real-time tasks such as driving and walking.

Other Planning Techniques

Triangle tables

- Provides a way of recording the goals that each operator expected to satisfy as well as the goals that must be true for it to execute correctly.

Meta-planning

- A technique for reasoning not just about the problem solved but also about the planning process itself.

Macro-operators

- Allow a planner to build new operators that represent commonly used sequences of operators.

Case-based planning:

- Re-uses old plans to make new ones.

UNDERSTANDING

Understanding is the simplest procedure of all human beings. Understanding means ability to determine some new knowledge from a given knowledge. For each action of a problem, the mapping of some new actions is very necessary. Mapping the knowledge means transferring the knowledge from one representation to another representation. For example, if you will say “I need to go to New Delhi” for which you will book the tickets. The system will have “understood” if it finds the first available plane to New Delhi. But if you will say the same thing to you friends, who knows that your family lives in “New Delhi”, he/she will have “understood” if he/she realizes that there may be a problem or occasion in your family. For people, understanding applies to inputs from all the senses. Computer understanding has so far been applied primarily to images, speech and typed languages. It is important to keep in mind that the success or failure of an “understanding” problem can rarely be measured in an absolute sense but must instead be measured with respect to a particular task to be performed. There are some factors that contribute to the difficulty of an understanding problem.

(a) If the target representation is very complex for which you cannot map from the original representation.

- (b) There are different types of mapping factors may arise like one-to-one, one-to-many and many to many.
- (c) Some noise or disturbing factors are also there.
- (d) The level of interaction of the source components may be complex one.
- (e) The problem solver might be unknown about some more complex problems.
- (f) The intermediary actions may also be unavailable.

Consider an example of an English sentence which is being used for communication with a keyword based data retrieval system. Suppose I want to know all about the temples in India. So I would need to be translated into a representation such as The above sentence is a simple sentence for which the corresponding representation may be easy to implement. But what for the complex queries?

Consider the following query.

“Ram told Sita he would not eat apple with her. He has to go to the office”.

This type of complex queries can be modeled with the conceptual dependency representation which is more complex than that of simple representation. Constructing these queries is very difficult since more information are to be extracted. Extracting more information will require some more knowledge. Also the type of mapping process is not quite easy to the problem solver. Understanding is the process of mapping an input from its original form to a more useful one. The simplest kind of mapping is “one-to-one”.

In one-to-one mapping each different problems would lead to only one solution. But there are very few inputs which are one-to-one. Other mappings are quite difficult to implement. Many-to-one mappings are frequent is that free variation is often allowed, either because of the physical limitations of that produces the inputs or because such variation simply makes the task of generating the inputs.

Many to one mapping require that the understanding system know about all the ways that a target representation can be expressed in the source language. One-to-many mapping requires a great deal of domain knowledge in order to make the correct choice among the available target representation.

The mapping process is simplest if each component can be mapped without concern for the other components of the statement. If the number of interactions increases, then the complexity of the problem will increase. In many understanding situations the input to which meaning should be assigned is not always the input that is presented to the under stander.

Because of the complex environment in which understanding usually occurs, other things often interfere with the basic input before it reaches the under stander. Hence the understanding will be more complex if there will be some sort of noise on the inputs.

Natural Language Processing

Introduction to Natural Language Processing

- Language meant for communicating with the world.
- Also, By studying language, we can come to understand more about the world.
- If we can succeed at building computational mode of language, we will have a powerful tool for communicating with the world.
- Also, We look at how we can exploit knowledge about the world, in combination with linguistic facts, to build computational natural language systems.

Natural Language Processing (NLP) problem can divide into two tasks:

1. Processing written text, using lexical, syntactic and semantic knowledge of the language as well as the required real-world information.
2. Processing spoken language, using all the information needed above plus additional knowledge about phonology as well as enough added information to handle the further ambiguities that arise in speech.

Steps in Natural Language Processing

Morphological Analysis

- Individual words analyzed into their components and non-word tokens such as punctuation separated from the words.

Syntactic Analysis

- Linear sequences of words transformed into structures that show how the words relate to each other.
- Moreover, Some word sequences may reject if they violate the language's rule for how words may combine.

Semantic Analysis

- The structures created by the syntactic analyzer assigned meanings.
- Also, A mapping made between the syntactic structures and objects in the task domain.
- Moreover, Structures for which no such mapping possible may reject.

Discourse integration

- The meaning of an individual sentence may depend on the sentences that precede it. And also, may influence the meanings of the sentences that follow it.

Pragmatic Analysis

- Moreover, The structure representing what said reinterpreted to determine what was actually meant.

Summary

- Results of each of the main processes combine to form a natural language system.
- All of the processes are important in a complete natural language understanding system.
- Not all programs are written with exactly these components.
- Sometimes two or more of them collapsed.
- Doing that usually results in a system that is easier to build for restricted subsets of English but one that is harder to extend to wider coverage.

Steps Natural Language Processing

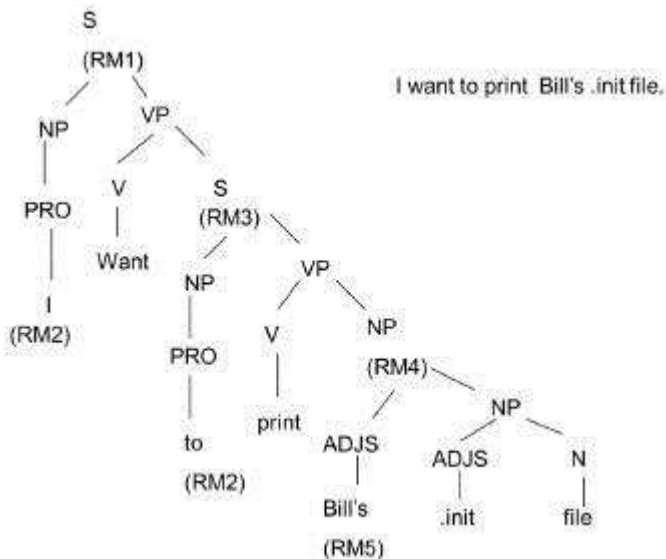
Morphological Analysis

- Suppose we have an English interface to an operating system and the following sentence typed: I want to print Bill's .init file.
- The morphological analysis must do the following things:

- Pull apart the word “Bill’s” into proper noun “Bill” and the possessive suffix “’s”
- Recognize the sequence “.init” as a file extension that is functioning as an adjective in the sentence.
- This process will usually assign syntactic categories to all the words in the sentence.

Syntactic Analysis

- A syntactic analysis must exploit the results of the morphological analysis to build a structural description of the sentence.
- The goal of this process, called parsing, is to convert the flat list of words that form the sentence into a structure that defines the units that represented by that flat list.
- The important thing here is that a flat sentence has been converted into a hierarchical structure. And that the structure corresponds to meaning units when a semantic analysis performed.
- Reference markers (set of entities) shown in the parenthesis in the parse tree.
- Each one corresponds to some entity that has mentioned in the sentence.
- These reference markers are useful later since they provide a place in which to accumulate information about the entities as we get it.



Semantic Analysis

- The semantic analysis must do two important things:
 1. It must map individual words into appropriate objects in the knowledge base or database.
 2. It must create the correct structures to correspond to the way the meanings of the individual words combine with each other.

Discourse Integration

- Specifically, we do not know whom the pronoun “I” or the proper noun “Bill” refers to.
- To pin down these references requires an appeal to a model of the current discourse context, from which we can learn that the current user is USER068 and that the only person named “Bill” about whom we could be talking is USER073.
- Once the correct referent for Bill known, we can also determine exactly which file referred to.

Pragmatic Analysis

- The final step toward effective understanding is to decide what to do as a result.

- One possible thing to do to record what was said as a fact and done with it.
- For some sentences, a whose intended effect is clearly declarative, that is the precisely correct thing to do.
- But for other sentences, including this one, the intended effect is different.
- We can discover this intended effect by applying a set of rules that characterize cooperative dialogues.
- The final step in pragmatic processing to translate, from the knowledge-based representation to a command to be executed by the system.

Syntactic Processing

- Syntactic Processing is the step in which a flat input sentence converted into a hierarchical structure that corresponds to the units of meaning in the sentence. This process called parsing.
- It plays an important role in natural language understanding systems for two reasons:
 1. Semantic processing must operate on sentence constituents. If there is no syntactic parsing step, then the semantics system must decide on its own constituents. If parsing is done, on the other hand, it constrains the number of constituents that semantics can consider.
 2. Syntactic parsing is computationally less expensive than is semantic processing. Thus it can play a significant role in reducing overall system complexity.
- Although it is often possible to extract the meaning of a sentence without using grammatical facts, it is not always possible to do so.
- Almost all the systems that are actually used have two main components:
 1. A declarative representation, called a grammar, of the syntactic facts about the language.
 2. A procedure, called parser that compares the grammar against input sentences to produce parsed structures.

Grammars and Parsers

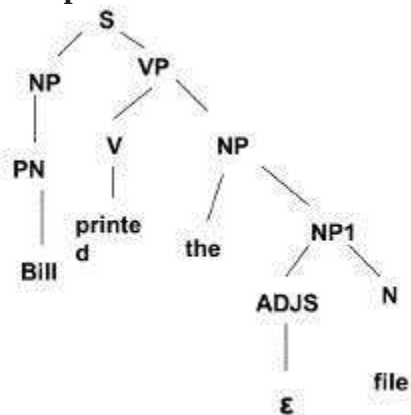
- The most common way to represent grammars is a set of production rules.
- The first rule can read as “A sentence composed of a noun phrase followed by Verb Phrase”; the Vertical bar is OR; ϵ represents the empty string.
- Symbols that further expanded by rules called non-terminal symbols.
- Symbols that correspond directly to strings that must found in an input sentence called terminal symbols.
- Grammar formalism such as this one underlies many linguistic theories, which in turn provide the basis for many natural language understanding systems.
- Pure context-free grammars are not effective for describing natural languages.
- NLPs have less in common with computer language processing systems such as compilers.
- Parsing process takes the rules of the grammar and compares them against the input sentence.
- The simplest structure to build is a Parse Tree, which simply records the rules and how they matched.
- Every node of the parse tree corresponds either to an input word or to a non-terminal in our grammar.
- Each level in the parse tree corresponds to the application of one grammar rule.

Example for Syntactic Processing – Augmented Transition Network

Syntactic Processing is the step in which a flat input sentence is converted into a hierarchical structure that corresponds to the units of meaning in the sentence. This process called parsing. It plays an important role in natural language understanding systems for two reasons:

1. Semantic processing must operate on sentence constituents. If there is no syntactic parsing step, then the semantics system must decide on its own constituents. If parsing is done, on the other hand, it constrains the number of constituents that semantics can consider.
2. Syntactic parsing is computationally less expensive than is semantic processing. Thus it can play a significant role in reducing overall system complexity.

Example: A Parse tree for a sentence: Bill Printed the file



The grammar specifies two things about a language:

1. Its weak generative capacity, by which we mean the set of sentences that contained within the language. This set made up of precisely those sentences that can completely match by a series of rules in the grammar.
2. Its strong generative capacity, by which we mean the structure to assign to each grammatical sentence of the language.

Augmented Transition Network (ATN)

- An augmented transition network is a top-down parsing procedure that allows various kinds of knowledge to be incorporated into the parsing system so it can operate efficiently.
- ATNs build on the idea of using finite state machines (Markov model) to parse sentences.
- Instead of building an automaton for a particular sentence, a collection of transition graphs is built.
- A grammatically correct sentence is parsed by reaching a final state in any state graph.
- Transitions between these graphs simply subroutine calls from one state to any initial state on any graph in the network.
- A sentence is determined to be grammatically correct if a final state is reached by the last word in the sentence.
- The ATN is similar to a finite state machine in which the class of labels that can attach to the arcs that define the transition between states is augmented.

Arcs may be labeled with:

- Specific words such as “in”.
- Word categories such as noun.

- Procedures that build structures that will form part of the final parse.
- Procedures that perform arbitrary tests on current input and sentence components that have identified.

Semantic Analysis

- The structures created by the syntactic analyzer assigned meanings.
- A mapping made between the syntactic structures and objects in the task domain.
- Structures for which no such mapping is possible may be rejected.
- The semantic analysis must do two important things:
 - It must map individual words into appropriate objects in the knowledge base or database.
 - It must create the correct structures to correspond to the way the meanings of the individual words combine with each other. Semantic Analysis AI
- Producing a syntactic parse of a sentence is only the first step toward understanding it.
- We must produce a representation of the meaning of the sentence.
- Because understanding is a mapping process, we must first define the language into which we are trying to map.
- There is no single definitive language in which all sentence meaning can be described.
- The choice of a target language for any particular natural language understanding program must depend on what is to be done with the meanings once they are constructed.
- Choice of the target language in Semantic Analysis AI
 - There are two broad families of target languages that are used in NL systems, depending on the role that the natural language system is playing in a larger system:
 - When natural language is considered as a phenomenon on its own, as for example when one builds a program whose goal is to read the text and then answer questions about it. A target language can be designed specifically to support language processing.
 - When natural language is used as an interface language to another program (such as a db query system or an expert system), then the target language must be legal input to that other program. Thus the design of the target language is driven by the backend program.

Discourse and Pragmatic Processing

To understand a single sentence, it is necessary to consider the discourse and pragmatic context in which the sentence was uttered.

There are a number of important relationships that may hold between phrases and parts of their discourse contexts, including:

1. Identical entities. Consider the text:
 - Bill had a red balloon. o John wanted it.
 - The word “it” should identify as referring to the red balloon. These types of references called anaphora.
2. Parts of entities. Consider the text:
 - Sue opened the book she just bought.
 - The title page was torn.
 - The phrase “title page” should be recognized as part of the book that was just bought.

3. Parts of actions. Consider the text:
 - John went on a business trip to New York.
 - He left on an early morning flight.
 - Taking a flight should recognize as part of going on a trip.
4. Entities involved in actions. Consider the text:
 - My house was broken into last week.
 - Moreover, They took the TV and the stereo.
 - The pronoun “they” should recognize as referring to the burglars who broke into the house.
5. Elements of sets. Consider the text:
 - The decals we have in stock are stars, the moon, item and a flag.
 - I’ll take two moons.
 - Moons mean moon decals.
6. Names of individuals:
 - Dev went to the movies.
7. Causal chains
 - There was a big snow storm yesterday.
 - So, The schools closed today.
8. Planning sequences:
 - Sally wanted a new car
 - She decided to get a job.
9. Implicit presuppositions:
 - Did Joe fail CS101?

The major focus is on using following kinds of knowledge:

- The current focus of the dialogue.
- Also, A model of each participant’s current beliefs.
- Moreover, The goal-driven character of dialogue.
- The rules of conversation shared by all participants.

Statistical Natural Language Processing

Formerly, many language-processing tasks typically involved the direct hand coding of rules, which is not in general robust to natural-language variation. The machine-learning paradigm calls instead for using [statistical inference](#) to automatically learn such rules through the analysis of large **corpora** of typical real-world examples (a *corpus* (plural, “corpora”) is a set of documents, possibly with human or computer annotations).

Many different classes of machine learning algorithms have been applied to natural-language processing tasks. These algorithms take as input a large set of “features” that are generated from the input data. Some of the earliest-used algorithms, such as [decision trees](#), produced systems of hard if-then rules similar to the systems of hand-written rules that were then common.

Increasingly, however, research has focused on [statistical models](#), which make soft, [probabilistic](#) decisions based on attaching [real-valued](#) weights to each input feature. Such models have the advantage that they can express the relative certainty of many different possible answers rather than only one, producing more reliable results when such a model is included as a component of a larger system.

Systems based on machine-learning algorithms have many advantages over hand-produced rules:

- The learning procedures used during machine learning automatically focus on the most common cases, whereas when writing rules by hand it is often not at all obvious where the effort should be directed.
- Automatic learning procedures can make use of statistical inference algorithms to produce models that are robust to unfamiliar input (e.g. containing words or structures that have not been seen before) and to erroneous input (e.g. with misspelled words or words accidentally omitted). Generally, handling such input gracefully with hand-written rules—or more generally, creating systems of hand-written rules that make soft decisions—is extremely difficult, error-prone and time-consuming.
- Systems based on automatically learning the rules can be made more accurate simply by supplying more input data. However, systems based on hand-written rules can only be made more accurate by increasing the complexity of the rules, which is a much more difficult task. In particular, there is a limit to the complexity of systems based on hand-crafted rules, beyond which the systems become more and more unmanageable. However, creating more data to input to machine-learning systems simply requires a corresponding increase in the number of man-hours worked, generally without significant increases in the complexity of the annotation process.

Spell Checking

Spell checking is one of the applications of natural language processing that impacts billions of users daily. A good introduction to spell checking can be found on Peter Norvig's webpage. The article introduces a simple 21-line spell checker implementation in Python combining simple language and error models to predict the word a user intended to type. **The language model estimates how likely a given word c is in the language for which the spell checker is designed, this can be written as $P(C)$. The error model estimates the probability $P(w|c)$ of typing the misspelled version w conditionally to the intention of typing the correctly spelled word c .** The spell checker then returns word c corresponding to the highest value of $P(w|c)P(c)$ among all possible words in the language.

Module 3

LEARNING

Learning is the improvement of performance with experience over time.

Learning element is the portion of a learning AI system that decides how to modify the performance element and implements those modifications.

We all learn new knowledge through different methods, depending on the type of material to be learned, the amount of relevant knowledge we already possess, and the environment in which the learning takes place. There are five methods of learning . They are,

1. Memorization (rote learning)
2. Direct instruction (by being told)
3. Analogy
4. Induction

5. Deduction

Learning by memorizations is the simplest form of learning. It requires the least amount of inference and is accomplished by simply copying the knowledge in the same form that it will be used directly into the knowledge base.

Example:- Memorizing multiplication tables, formulate , etc.

Direct instruction is a complex form of learning. This type of learning requires more inference than rote learning since the knowledge must be transformed into an operational form before learning when a teacher presents a number of facts directly to us in a well organized manner.

Analogical learning is the process of learning a new concept or solution through the use of similar known concepts or solutions. We use this type of learning when solving problems on an exam where previously learned examples serve as a guide or when make frequent use of analogical learning. This form of learning requires still more inferring than either of the previous forms. Since difficult transformations must be made between the known and unknown situations.

Learning by induction is also one that is used frequently by humans . it is a powerful form of learning like analogical learning which also require s more inferring than the first two methods. This learning requires the use of inductive inference, a form of invalid but useful inference. We use inductive learning of instances of examples of the concept. For example we learn the concepts of color or sweet taste after experiencing the sensations associated with several examples of colored objects or sweet foods.

Deductive learning is accomplished through a sequence of deductive inference steps using known facts. From the known facts, new facts or relationships are logically derived. Deductive learning usually requires more inference than the other methods.

Review Questions:-

1. what is perception ?
2. How do we overcome the Perceptual Problems?
3. Explain in detail the constraint satisfaction waltz algorithm?
4. What is learning ?
5. What is Learning element ?
6. List and explain the methods of learning?

Types of learning:- Classification or taxonomy of learning types serves as a guide in studying or comparing a differences among them. One can develop learning taxonomies based on the type of knowledge representation used (predicate calculus , rules, frames), the type of knowledge learned (concepts, game playing, problem solving), or by the area of application (medical diagnosis , scheduling , prediction and so on).

The classification is intuitively more appealing and is one which has become popular among machine learning researchers . it is independent of the knowledge domain and the representation scheme is used. It is based on the type of inference strategy employed or the methods used in the learning process. The five different learning methods under this taxonomy are:

Memorization (rote learning)

Direct instruction (by being told)

Analogy

Induction

Deduction

Learning by memorization is the simplest form of learning. It requires the least amount of inference and is accomplished by simply copying the knowledge in the same form that it will be

used directly into the knowledge base. We use this type of learning when we memorize multiplication tables ,
for example.

A slightly more complex form of learning is by direct instruction. This type of learning requires more understanding and inference than rote learning since the knowledge must be transformed into an operational form before being integrated into the knowledge base. We use this type of learning when a teacher presents a number of facts directly to us in a well organized manner. The third type listed, analogical learning, is the process of learning a new concept or solution through the use of similar known concepts or solutions. We use this type of learning when solving problems on an examination where previously learned examples serve as a guide or when we learn to drive a truck using our knowledge of car driving. We make frequent use of analogical learning. This form of learning requires still more inferring than either of the previous forms, since difficult transformations must be made between the known and unknown situations. This is a kind of application of knowledge in a new situation.

The fourth type of learning is also one that is used frequently by humans. It is a powerful form of learning which, like analogical learning, also requires more inferring than the first two methods. This form of learning requires the use of inductive inference, a form of invalid but useful inference. We use inductive learning when we formulate a general concept after seeing a number of instances or examples of the concept. For example, we learn the concepts of color and sweet taste after experiencing the sensation associated with several examples of colored objects or sweet foods.

The final type of acquisition is deductive learning. It is accomplished through a sequence of deductive inference steps using known facts. From the known facts, new facts or relationships are logically derived. Deductive learning usually requires more inference than the other methods. The inference method used is, of course , a deductive type, which is a valid form of inference. In addition to the above classification, we will sometimes refer to learning methods as weak methods or knowledge-rich methods. Weak methods are general purpose methods in which little or no initial knowledge is available. These methods are more mechanical than the classical AI knowledge – rich methods. They often rely on a form of heuristics search in the learning process.

Rote Learning

Rote learning is the basic learning activity. **Rote learning** is a [memorization](#) technique based on [repetition](#). It is also called memorization because the [knowledge](#), without any modification is, simply copied into the knowledge base. As computed values are stored, this technique can save a significant amount of time.

Rote learning technique can also be used in complex learning systems provided sophisticated techniques are employed to use the stored values faster and there is a generalization to keep the number of stored information down to a manageable level. Checkers-playing program, for ex

The idea is that one will be able to quickly recall the meaning of the material the more one repeats it. Some of the alternatives to rote learning include meaningful learning, associative learning, and active learning. ample, uses this technique to learn the board positions it evaluates in its look-ahead search.

Learning By Taking Advice.

This is a simple form of learning. Suppose a programmer writes a set of instructions to instruct the computer what to do, the programmer is a teacher and the computer is a student. Once learned (i.e. programmed), the system will be in a position to do new things.

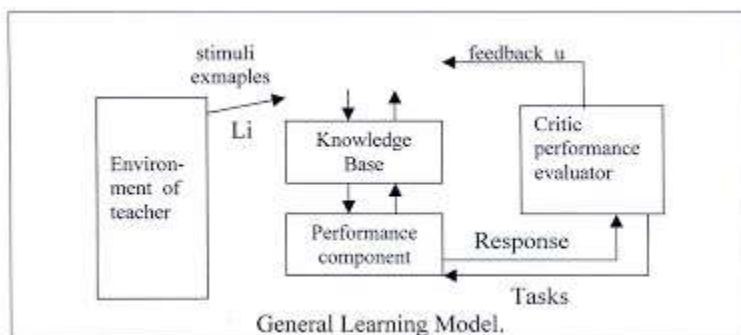
The advice may come from many sources: human experts, internet to name a few. This type of learning requires more inference than rote learning. The knowledge must be transformed into an operational form before stored in the knowledge base. Moreover the reliability of the source of knowledge should be considered.

The system should ensure that the new knowledge is conflicting with the existing knowledge. FOO (First Operational Operationaliser), for example, is a learning system which is used to learn the game of Hearts. It converts the advice which is in the form of principles, problems, and methods into effective executable (LISP) procedures (or knowledge). Now this knowledge is ready to use.

General Learning Model.

General Learning Model: - AS noted earlier, learning can be accomplished using a number of different methods, such as by memorization facts, by being told, or by studying examples like problem solution. Learning requires that new knowledge structures be created from some form of input stimulus. This new knowledge must then be assimilated into a knowledge base and be tested in some way for its utility. Testing means that the knowledge should be used in performance of some task from which meaningful feedback can be obtained, where the feedback provides some measure of the accuracy and usefulness of the newly acquired knowledge.

General Learning Model



general learning model is depicted in figure 4.1 where the environment has been included as a part of the overall learner system. The environment may be regarded as either a form of nature which produces random stimuli or as a more organized training source such as a teacher which provides carefully selected training examples for the learner component. The actual form of environment used will depend on the particular learning paradigm. In any case, some representation language must be assumed for communication between the environment and the learner. The language may be the same representation scheme as that used in the knowledge base (such as a form of predicate calculus). When they are chosen to be the same, we say the single representation trick is being used. This usually results in a simpler implementation since it is not necessary to transform between two or more different representations.

For some systems the environment may be a user working at a keyboard . Other systems will use program modules to simulate a particular environment. In even more realistic cases the system will have real physical sensors which interface with some world environment.

Inputs to the learner component may be physical stimuli of some type or descriptive , symbolic training examples. The information conveyed to the learner component is used to create and modify knowledge structures in the knowledge base. This same knowledge is used by the performance component to carry out some tasks, such as solving a problem playing a game, or classifying instances of some concept.

given a task, the performance component produces a response describing its action in performing the task. The critic module then evaluates this response relative to an optimal response.

Feedback , indicating whether or not the performance was acceptable , is then sent by the critic module to the learner component for its subsequent use in modifying the structures in the knowledge base. If proper learning was accomplished, the system's performance will have improved with the changes made to the knowledge base.

The cycle described above may be repeated a number of times until the performance of the system has reached some acceptable level, until a known learning goal has been reached, or until changes ceases to occur in the knowledge base after some chosen number of training examples have been observed.

There are several important factors which influence a system's ability to learn in addition to the form of representation used. They include the types of training provided, the form and extent of any initial background knowledge , the type of feedback provided, and the learning algorithms used.

The type of training used in a system can have a strong effect on performance, much the same as it does for humans. Training may consist of randomly selected instance or examples that have been carefully selected and ordered for presentation. The instances may be positive examples of some concept or task a being learned, they may be negative, or they may be mixture of both positive and negative. The instances may be well focused using only relevant information, or they may contain a variety of facts and details including irrelevant data.

There are Many forms of learning can be characterized as a search through a space of possible hypotheses or solutions. To make learning more efficient. It is necessary to constrain this search process or reduce the search space. One method of achieving this is through the use of background knowledge which can be used to constrain the search space or exercise control operations which limit the search process.

Feedback is essential to the learner component since otherwise it would never know if the knowledge structures in the knowledge base were improving or if they were adequate for the performance of the given tasks. The feedback may be a simple yes or no type of evaluation, or it

may contain more useful information describing why a particular action was good or bad. Also , the feedback may be completely reliable, providing an accurate assessment of the performance or it may contain noise, that is the feedback may actually be incorrect some of the time. Intuitively , the feedback must be accurate more than 50% of the time; otherwise the system carries useful information, the learner should also to build up a useful corpus of knowledge quickly. On the other hand, if the feedback is noisy or unreliable, the learning process may be very slow and the resultant knowledge incorrect.

Learning Neural Network

Perceptron

- The perceptron an invention of (1962) Rosenblatt was one of the earliest neural network models.
- Also, It models a neuron by taking a weighted sum of its inputs and sending the output 1 if the sum is greater than some adjustable threshold value (otherwise it sends 0).

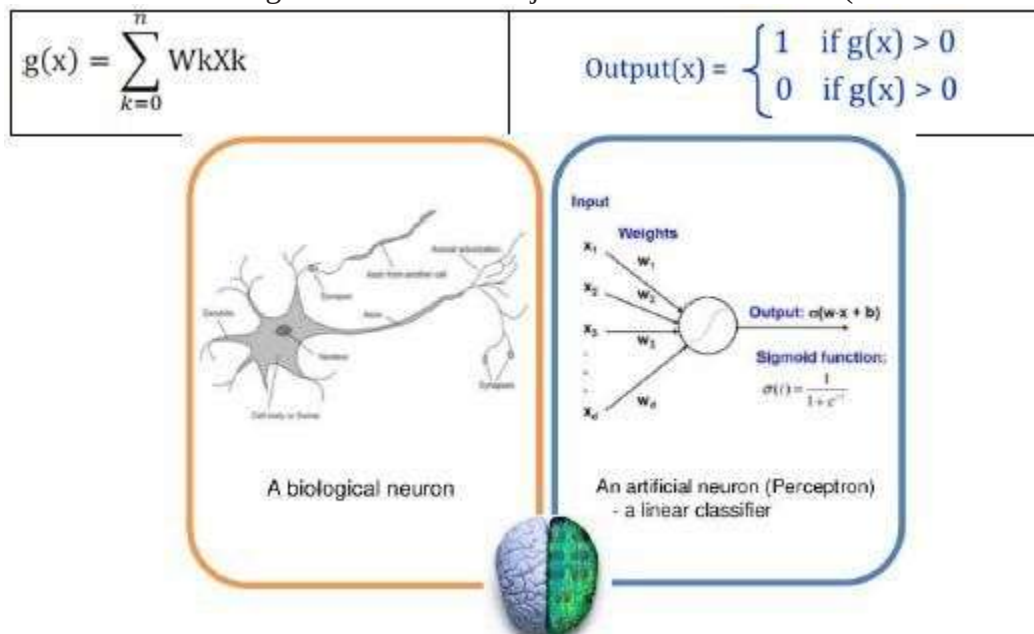


Figure: A neuron & a Perceptron

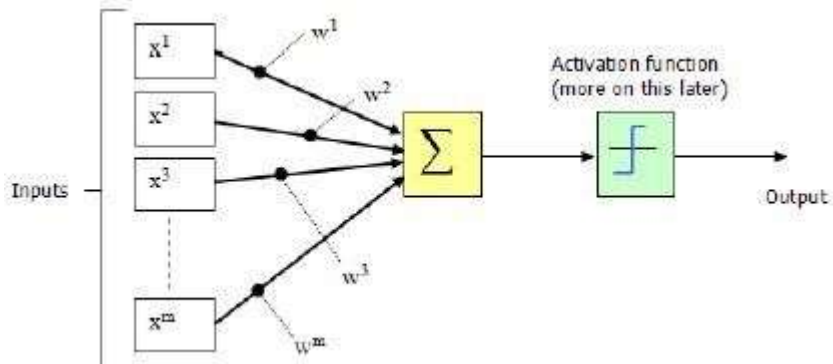


Figure: Perceptron with adjustable threshold

- In case of zero with two inputs $g(x) = w_0 + w_1x_1 + w_2x_2 = 0$

- $x_2 = -(w_1/w_2)x_1 - (w_0/w_2) \rightarrow$ equation for a line
- the location of the line is determined by the weight w_0 , w_1 and w_2
- if an input vector lies on one side of the line, the perceptron will output 1
- if it lies on the other side, the perceptron will output 0
- Moreover, Decision surface: a line that correctly separates the training instances corresponds to a perfectly function perceptron.

Perceptron Learning Algorithm

Given: A classification problem with n input feature ($x_1, x_2, \dots, x_n$) and two output classes.

Compute A set of weights ($w_0, w_1, w_2, \dots, w_n$) that will cause a perceptron to fire whenever the input falls into the first output class.

1. Create a perceptron with $n+1$ input and $n+1$ weight, where the x_0 is always set to 1.
 2. Initialize the weights ($w_0, w_1, \dots, w_n$) to random real values.
 3. Iterate through the training set, collecting all examples *misclassified* by the current set of weights.
 4. If all examples are classified correctly, output the weights and quit.
 5. Otherwise, compute the vector sum S of the misclassified input vectors where each vector has the form ($x_0, x_1, \dots, x_n$). In creating the sum, add to S a vector x if x is an input for which the perceptron incorrectly fails to fire, but $-x$ if x is an input for which the perceptron incorrectly fires. Multiply sum by a scale factor η .
 6. Moreover, Modify the weights ($w_0, w_1, \dots, w_n$) by adding the elements of the vector S to them.
 7. Go to step 3.
- The perceptron learning algorithm is a search algorithm. It begins with a random initial state and finds a solution state. The search space is simply all possible assignments of real values to the weights of the perception, and the search strategy is gradient descent.
 - The perceptron learning rule is guaranteed to converge to a solution in a finite number of steps, so long as a solution exists.
 - Moreover, This brings us to an important question. What problems can a perceptron solve? Recall that a single-neuron perceptron is able to divide the input space into two regions.
 - Also, The perception can be used to classify input vectors that can be separated by a linear boundary. We call such vectors linearly separable.
 - Unfortunately, many problems are not linearly separable. The classic example is the XOR gate. It was the inability of the basic perceptron to solve such simple problems that are not linearly separable or non-linear.

Genetic Learning

Supervised Learning

Supervised learning is the machine learning task of inferring a function from labeled training data.

Moreover, The training data consist of a set of training examples.

In supervised learning, each example a pair consisting of an input object (typically a vector) and the desired output value (also called the supervisory signal).

Training set

A training set a set of data used in various areas of information science to discover potentially predictive relationships.

Training sets used in artificial intelligence, machine learning, genetic programming, intelligent systems, and statistics.

In all these fields, a training set has much the same role and often used in conjunction with a test set.

Testing set

A **test set** is a set of data used in various areas of information science to assess the strength and utility of a predictive relationship.

Moreover, Test sets are used in artificial intelligence, machine learning, genetic programming, and statistics. In all these fields, a test set has much the same role.

Accuracy of classifier: Supervised learning

In the fields of science, engineering, industry, and statistics. The accuracy of a measurement system is the degree of closeness of measurements of a quantity to that quantity's actual (true) value.

Sensitivity analysis: Supervised learning

Similarly, Local Sensitivity as correlation coefficients and partial derivatives can only use, if the correlation between input and output is linear.

Regression: Supervised learning

In statistics, **regression analysis** is a statistical process for estimating the relationships among variables.

Moreover, It includes many techniques for modeling and analyzing several variables. When the focus on the relationship between a dependent variable and one or more independent variables.

More specifically, regression analysis helps one understand how the typical value of the dependent variable (or 'criterion variable') changes when any one of the independent variables varied. Moreover, While the other independent variables held fixed.

Expert systems:

Expert system = knowledge + problem-solving methods. ... A knowledge base that captures the domain-specific knowledge and an inference engine that consists of algorithms for manipulating the knowledge represented in the knowledge base to solve a problem presented to the system.

Expert systems (ES) are one of the prominent research domains of AI. It is introduced by the researchers at Stanford University, Computer Science Department.

What are Expert Systems?

The expert systems are the computer applications developed to solve complex problems in a particular domain, at the level of extra-ordinary human intelligence and expertise.

Characteristics of Expert Systems

- High performance
- Understandable
- Reliable
- Highly responsive

Capabilities of Expert Systems

The expert systems are capable of –

- Advising
- Instructing and assisting human in decision making
- Demonstrating
- Deriving a solution
- Diagnosing
- Explaining
- Interpreting input
- Predicting results
- Justifying the conclusion
- Suggesting alternative options to a problem

They are incapable of –

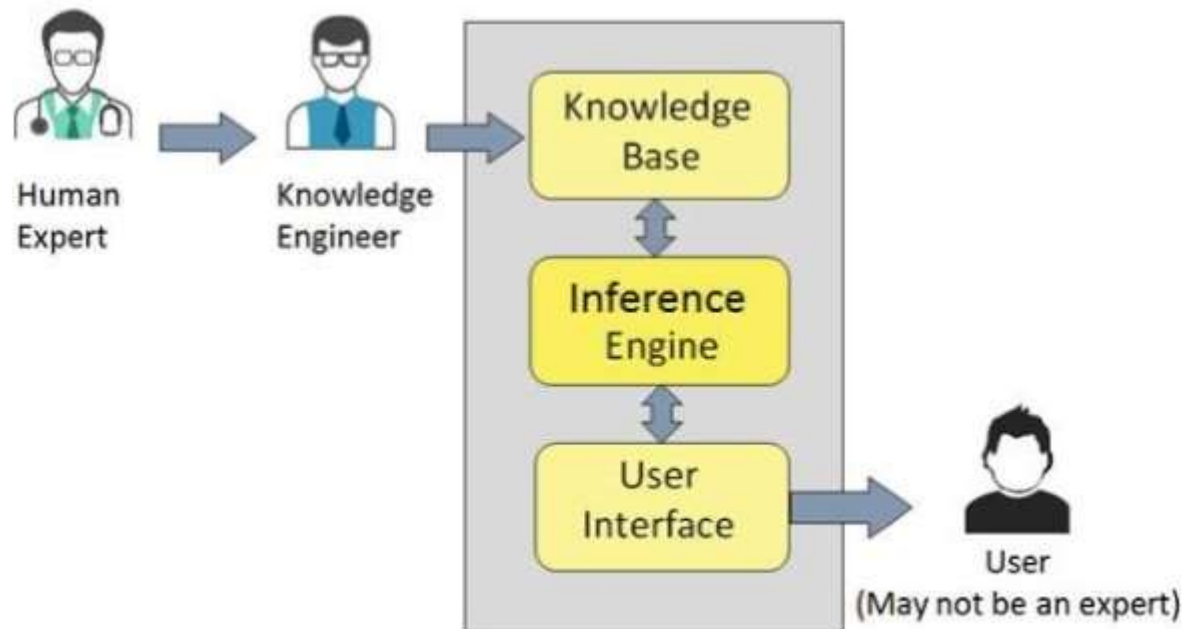
- Substituting human decision makers
- Possessing human capabilities
- Producing accurate output for inadequate knowledge base
- Refining their own knowledge

Components of Expert Systems

The components of ES include –

- Knowledge Base
- Inference Engine
- User Interface

Let us see them one by one briefly –



Knowledge Base

It contains domain-specific and high-quality knowledge. Knowledge is required to exhibit intelligence. The success of any ES majorly depends upon the collection of highly accurate and precise knowledge.

What is Knowledge?

The data is collection of facts. The information is organized as data and facts about the task domain. Data, information, and past experience combined together are termed as knowledge.

Components of Knowledge Base

The knowledge base of an ES is a store of both, factual and heuristic knowledge.

- Factual Knowledge – It is the information widely accepted by the Knowledge Engineers and scholars in the task domain.
- Heuristic Knowledge – It is about practice, accurate judgement, one's ability of evaluation, and guessing.

Knowledge representation

It is the method used to organize and formalize the knowledge in the knowledge base. It is in the form of IF-THEN-ELSE rules.

Knowledge Acquisition

The success of any expert system majorly depends on the quality, completeness, and accuracy of the information stored in the knowledge base.

The knowledge base is formed by readings from various experts, scholars, and the Knowledge Engineers. The knowledge engineer is a person with the qualities of empathy, quick learning, and case analyzing skills.

He acquires information from subject expert by recording, interviewing, and observing him at work, etc. He then categorizes and organizes the information in a meaningful way, in the form of IF-THEN-ELSE rules, to be used by interference machine. The knowledge engineer also monitors the development of the ES.

Inference Engine

Use of efficient procedures and rules by the Inference Engine is essential in deducting a correct, flawless solution.

In case of knowledge-based ES, the Inference Engine acquires and manipulates the knowledge from the knowledge base to arrive at a particular solution.

In case of rule based ES, it –

- Applies rules repeatedly to the facts, which are obtained from earlier rule application.
- Adds new knowledge into the knowledge base if required.
- Resolves rules conflict when multiple rules are applicable to a particular case.

To recommend a solution, the Inference Engine uses the following strategies –

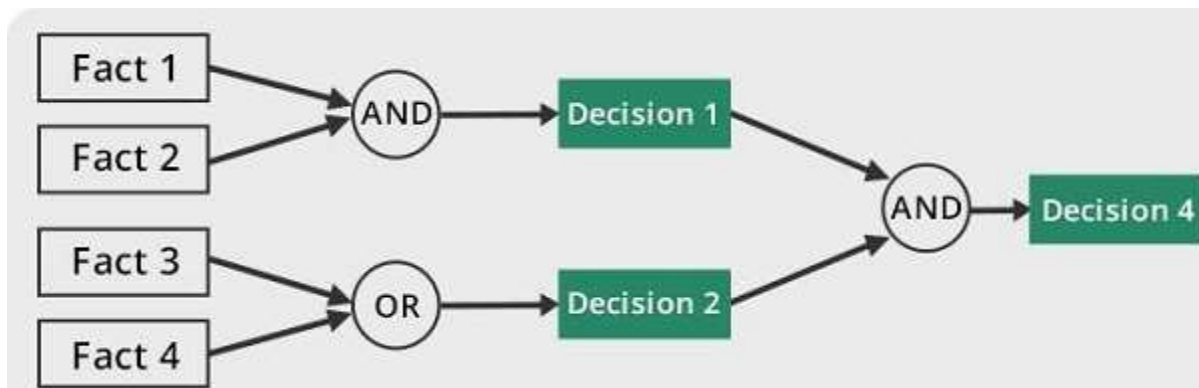
- Forward Chaining
- Backward Chaining

Forward Chaining

It is a strategy of an expert system to answer the question, “What can happen next?”

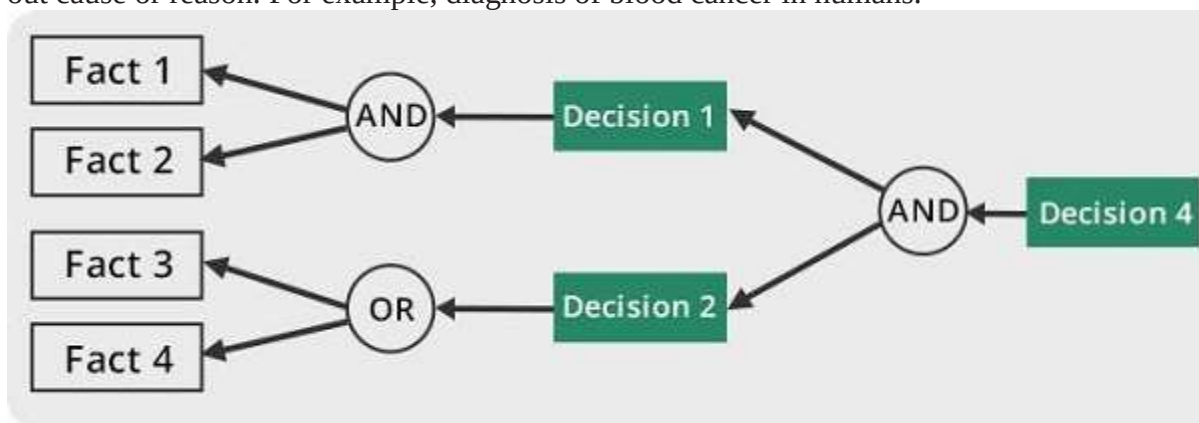
Here, the Inference Engine follows the chain of conditions and derivations and finally deduces the outcome. It considers all the facts and rules, and sorts them before concluding to a solution.

This strategy is followed for working on conclusion, result, or effect. For example, prediction of share market status as an effect of changes in interest rates.



Backward Chaining

With this strategy, an expert system finds out the answer to the question, “Why this happened?” On the basis of what has already happened, the Inference Engine tries to find out which conditions could have happened in the past for this result. This strategy is followed for finding out cause or reason. For example, diagnosis of blood cancer in humans.



User Interface

User interface provides interaction between user of the ES and the ES itself. It is generally Natural Language Processing so as to be used by the user who is well-versed in the task domain. The user of the ES need not be necessarily an expert in Artificial Intelligence. It explains how the ES has arrived at a particular recommendation. The explanation may appear in the following forms –

- Natural language displayed on screen.
- Verbal narrations in natural language.
- Listing of rule numbers displayed on the screen.

The user interface makes it easy to trace the credibility of the deductions.

Requirements of Efficient ES User Interface

- It should help users to accomplish their goals in shortest possible way.
- It should be designed to work for user's existing or desired work practices.
- Its technology should be adaptable to user's requirements; not the other way round.
- It should make efficient use of user input.

Expert Systems Limitations

No technology can offer easy and complete solution. Large systems are costly, require significant development time, and computer resources. ESs have their limitations which include –

- Limitations of the technology

- Difficult knowledge acquisition
- ES are difficult to maintain
- High development costs

Applications of Expert System

The following table shows where ES can be applied.

Application	Description
Design Domain	Camera lens design, automobile design.
Medical Domain	Diagnosis Systems to deduce cause of disease from observed data, conduction medical operations on humans.
Monitoring Systems	Comparing data continuously with observed system or with prescribed behavior such as leakage monitoring in long petroleum pipeline.
Process Control Systems	Controlling a physical process based on monitoring.
Knowledge Domain	Finding out faults in vehicles, computers.
Finance/Commerce	Detection of possible fraud, suspicious transactions, stock market trading, Airline scheduling, cargo scheduling.

Expert System Technology

There are several levels of ES technologies available. Expert systems technologies include –

- Expert System Development Environment – The ES development environment includes hardware and tools. They are –
 - Workstations, minicomputers, mainframes.
 - High level Symbolic Programming Languages such as LISP Programming (LISP) and PROgrammation en LOGique (PROLOG).
 - Large databases.
- Tools – They reduce the effort and cost involved in developing an expert system to large extent.
 - Powerful editors and debugging tools with multi-windows.
 - They provide rapid prototyping
 - Have Inbuilt definitions of model, knowledge representation, and inference design.
- Shells – A shell is nothing but an expert system without knowledge base. A shell provides the developers with knowledge acquisition, inference engine, user interface, and explanation facility. For example, few shells are given below –
 - Java Expert System Shell (JESS) that provides fully developed Java API for creating an expert system.
 - Vidwan, a shell developed at the National Centre for Software Technology, Mumbai in 1993. It enables knowledge encoding in the form of IF-THEN rules.

Development of Expert Systems: General Steps

The process of ES development is iterative. Steps in developing the ES include –

Identify Problem Domain

- The problem must be suitable for an expert system to solve it.
- Find the experts in task domain for the ES project.
- Establish cost-effectiveness of the system.

Design the System

- Identify the ES Technology
- Know and establish the degree of integration with the other systems and databases.
- Realize how the concepts can represent the domain knowledge best.

Develop the Prototype

From Knowledge Base: The knowledge engineer works to –

- Acquire domain knowledge from the expert.
- Represent it in the form of If-THEN-ELSE rules.

Test and Refine the Prototype

- The knowledge engineer uses sample cases to test the prototype for any deficiencies in performance.
- End users test the prototypes of the ES.

Develop and Complete the ES

- Test and ensure the interaction of the ES with all elements of its environment, including end users, databases, and other information systems.
- Document the ES project well.
- Train the user to use ES.

Maintain the ES

- Keep the knowledge base up-to-date by regular review and update.
- Cater for new interfaces with other information systems, as those systems evolve.

Benefits of Expert Systems

- Availability – They are easily available due to mass production of software.
- Less Production Cost – Production cost is reasonable. This makes them affordable.
- Speed – They offer great speed. They reduce the amount of work an individual puts in.
- Less Error Rate – Error rate is low as compared to human errors.
- Reducing Risk – They can work in the environment dangerous to humans.
- Steady response – They work steadily without getting motional, tensed or fatigued.

Expert System.

DEFINITION - An expert system is a computer program that simulates the judgement and behavior of a human or an organization that has expert knowledge and experience in a particular field. Typically, such a system contains a knowledge base containing accumulated experience and a set of rules for applying the knowledge base to each particular situation that is described to the program. Sophisticated expert systems can be enhanced with additions to the knowledge base or to the set of rules.

Among the best-known expert systems have been those that play chess and that assist in medical diagnosis.

An **expert system** is [software](#) that attempts to provide an answer to a problem, or clarify uncertainties where normally one or more human [experts](#) would need to be consulted. Expert systems are most common in a specific [problem domain](#), and is a traditional application and/or

subfield of artificial intelligence (AI). A wide variety of methods can be used to simulate the performance of the expert; however, common to most or all are: 1) the creation of a knowledge base which uses some knowledge representation structure to capture the knowledge of the Subject Matter Expert (SME); 2) a process of gathering that knowledge from the SME and codifying it according to the structure, which is called knowledge engineering; and 3) once the system is developed, it is placed in the same real world problem solving situation as the human SME, typically as an aid to human workers or as a supplement to some information system. Expert systems may or may not have learning components.

factors

The MYCIN rule-based expert system introduced a quasi-probabilistic approach called certainty factors, whose rationale is explained below.

A human, when reasoning, does not always make statements with 100% confidence: he might venture, "If Fritz is green, then he is probably a frog" (after all, he might be a chameleon). This type of reasoning can be imitated using numeric values called confidences. For example, if it is known that Fritz is green, it might be concluded with 0.85 confidence that he is a frog; or, if it is known that he is a frog, it might be concluded with 0.95 confidence that he hops. These certainty factor (CF) numbers quantify uncertainty in the degree to which the available evidence supports a hypothesis. They represent a degree of confirmation, and are not probabilities in a Bayesian sense. The CF calculus, developed by Shortliffe & Buchanan, increases or decreases the CF associated with a hypothesis as each new piece of evidence becomes available. It can be mapped to a probability update, although degrees of confirmation are not expected to obey the laws of probability. It is important to note, for example, that evidence for hypothesis H may have nothing to contribute to the degree to which Not_h is confirmed or disconfirmed (e.g., although a fever lends some support to a diagnosis of infection, fever does not disconfirm alternative hypotheses) and that the sum of CFs of many competing hypotheses may be greater than one (i.e., many hypotheses may be well confirmed based on available evidence).

The CF approach to a rule-based expert system design does not have a widespread following, in part because of the difficulty of meaningfully assigning CFs a priori. (The above example of green creatures being likely to be frogs is excessively naive.) Alternative approaches to quasi-probabilistic reasoning in expert systems involve fuzzy logic, which has a firmer mathematical foundation. Also, rule-engine shells such as Drools and Jess do not support probability manipulation: they use an alternative mechanism called salience, which is used to prioritize the order of evaluation of activated rules.

In certain areas, as in the tax-advice scenarios discussed below, probabilistic approaches are not acceptable. For instance, a 95% probability of being correct means a 5% probability of being wrong. The rules that are defined in such systems have no exceptions: they are only a means of achieving software flexibility when external circumstances change frequently. Because rules are stored as data, the core software does not need to be rebuilt each time changes to federal and state tax codes are announced.

Chaining

Two methods of reasoning when using inference rules are forward chaining and backward chaining.

Forward chaining starts with the data available and uses the inference rules to extract more data until a desired goal is reached. An inference engine using forward chaining searches the inference rules until it finds one in which the if clause is known to be true. It then concludes the then clause and adds this information to its data. It continues to do this until a goal is reached. Because the data available determines which inference rules are used, this method is also classified as data driven.

Backward chaining starts with a list of goals and works backwards to see if there is data which will allow it to conclude any of these goals. An inference engine using backward chaining would search the inference rules until it finds one which has a then clause that matches a desired goal. If the if clause of that inference rule is not known to be true, then it is added to the list of goals.

SW Architecture.

The following general points about expert systems and their architecture have been outlined:

1. The sequence of steps taken to reach a conclusion is dynamically synthesized with each new case. The sequence is not explicitly programmed at the time that the system is built.
2. Expert systems can process multiple values for any problem parameter. This permits more than one line of reasoning to be pursued and the results of incomplete (not fully determined) reasoning to be presented.
3. Problem solving is accomplished by applying specific knowledge rather than specific technique. This is a key idea in expert systems technology. It reflects the belief that human experts do not process their knowledge differently from others, but they do possess different knowledge. With this philosophy, when one finds that their expert system does not produce the desired results, work begins to expand the knowledge base, not to re-program the procedures.

End user

There are two styles of user-interface design followed by expert systems. In the original style of user interaction, the software takes the end-user through an interactive dialog. In the following example, a backward-chaining system seeks to determine a set of restaurants to recommend:

Q. Do you know which restaurant you want to go to?

A. No

Q. Is there any kind of food you would particularly like?

A. No

Q. Do you like spicy food?

A. No

Q. Do you usually drink wine with meals?

A. Yes

Q. When you drink wine, is it French wine?

A. Yes

Participants

There are generally three individuals having an interaction in an expert system. Primary among these is the end-user, the individual who uses the system for its problem solving assistance. In the construction and maintenance of the system there are two other roles: the problem domain expert who builds the system and supplies the knowledge base, and a knowledge engineer who assists the experts in determining the representation of their knowledge, enters this knowledge into an explanation module and who defines the inference technique required to solve the problem. Usually the knowledge engineer will represent the problem solving activity in the form of rules. When these rules are created from domain expertise, the knowledge base stores the rules of the expert system.

Inference rule

An understanding of the "inference rule" concept is important to understand expert systems. An inference rule is a conditional statement with two parts: an if clause and a then clause. This rule is what gives expert systems the ability to find solutions to diagnostic and prescriptive problems. An example of an inference rule is:

If the restaurant choice includes French and the occasion is romantic,

Then the restaurant choice is definitely Paul Bocuse.

Procedure node interface

The function of the procedure node interface is to receive information from the procedures coordinator and create the appropriate procedure call. The ability to call a procedure and receive information from that procedure can be viewed as simply a generalization of input from the external world. In some earlier expert systems external information could only be obtained in a

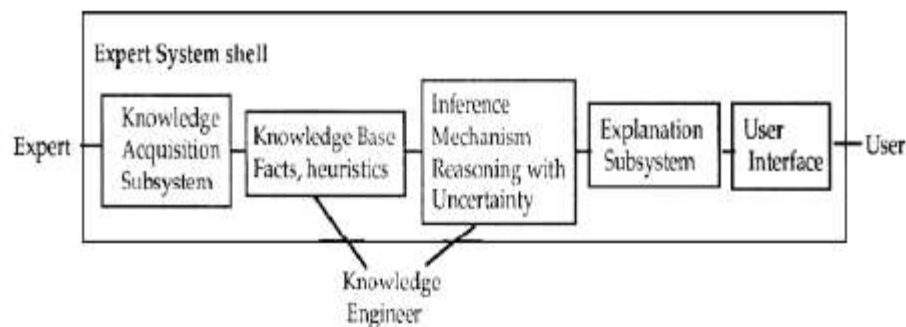
predetermined manner, which only allowed certain information to be acquired. Through the knowledge base, this expert system disclosed in the cross-referenced application can invoke any procedure allowed on its host system. This makes the expert system useful in a much wider class of knowledge domains than if it had no external access or only limited external access.

In the area of machine diagnostics using expert systems, particularly self-diagnostic applications, it is not possible to conclude the current state of "health" of a machine without some information. The best source of information is the machine itself, for it contains much detailed information that could not reasonably be provided by the operator.

The knowledge that is represented in the system appears in the rulebase. In the rulebase described in the cross-referenced applications, there are basically four different types of objects, with the associated information:

1. Classes: Questions asked to the user.
2. Parameters: Place holders for character strings which may be variables that can be inserted into a class question at the point in the question where the parameter is positioned.
3. Procedures: Definitions of calls to external procedures.
3. Rule nodes: Inferences in the system are made by a tree structure which indicates the rules or logic mimicking human reasoning. The nodes of these trees are called rule nodes. There are several different types of rule nodes.

Expert Systems/Shells. The E.S **shell** simplifies the process of creating a knowledge base. It is the **shell** that actually processes the information entered by a user relates it to the concepts contained in the knowledge base and provides an assessment or solution for a particular problem.



Knowledge Acquisition

Knowledge acquisition is the process used to define the rules and ontologies required for a knowledge-based system. The phrase was first used in conjunction with expert systems to describe the initial tasks associated with developing an expert system, namely finding and interviewing domain experts and capturing their knowledge via rules, objects, and frame-based ontologies.

Expert systems were one of the first successful applications of artificial intelligence technology to real world business problems. Researchers at Stanford and other AI laboratories worked with doctors and other highly skilled experts to develop systems that could automate complex tasks such as medical diagnosis. Until this point computers had mostly been used to automate highly data intensive tasks but not for complex reasoning. Technologies such as inference engines allowed developers for the first time to tackle more complex problems.

As expert systems scaled up from demonstration prototypes to industrial strength applications it was soon realized that the acquisition of domain expert knowledge was one of if not the most critical task in the knowledge engineering process. This knowledge acquisition process became an intense area of research on its own. One of the earlier works on the topic used Batesonian theories of learning to guide the process.

One approach to knowledge acquisition investigated was to use natural language parsing and generation to facilitate knowledge acquisition. Natural language parsing could be performed on manuals and other expert documents and an initial first pass at the rules and objects could be developed automatically. Text generation was also extremely useful in generating explanations for system behavior. This greatly facilitated the development and maintenance of expert systems. A more recent approach to knowledge acquisition is a re-use based approach. Knowledge can be developed in ontologies that conform to standards such as the Web Ontology Language (OWL). In this way knowledge can be standardized and shared across a broad community of knowledge workers. One example domain where this approach has been successful is bioinformatics.

Refferences

1. Elaine Rich, Kevin Knight, & Shivashankar B Nair, Artificial Intelligence, McGraw Hill, 3rd ed.,2009

References:

1) Introduction to Artificial Intelligence & Expert Systems, Dan W Patterson, PHI.,2010

2) S Kaushik, Artificial Intelligence, Cengage Learning, 1st ed.2011